BHT Berliner Hochschule für Technik

Entwicklung der smarten IOS-App "BerlinCharge" zur intelligenten Ladesäulen-Navigation

vorgelegt von

Tobias Gustav Hendrik Daiger

EDV.Nr.: 927607

dem Fachbereich VII – Elektrotechnik - Mechatronik - Optometrie
der Berliner Hochschule für Technik Berlin
vorgelegte Bachelorarbeit
zur Erlangung des akademischen Grades

Bachelor of Engineering (B.Eng.)

im Studiengang

Elektromobilität

Tag der Abgabe 22. Dezember 2025



Studiere Zukunft

Gutachter

Prof. Dr. S. Graupner	Berliner Hochschule für Technik
Prof. Dr. D. Macos	Berliner Hochschule für Technik

Kurzfassung

Die Bachelorarbeit thematisert die Entwicklung einer neuen iOS-App namens *BerlinCharge*, die auf den Kernfunktionen der bestehenden Vattenfall InCharge-App aufbaut und diese um eine intelligente Ladesäulen-Navigation erweitert.
Im Gegensatz zur ursprünglichen App, die das Auffinden von Ladesäulen auf der Karte und das Starten von Ladevorgängen ermöglicht, bietet die neu entwickelte Anwendung eine Navigationsfunktion vom aktuellen Standort.
Der Nutzer kann direkt zu einer ausgewählten Ladesäule navigieren. Als Kernfeature besteht zudem die Möglichkeit, eine Zieladresse einzugeben und zu wählen, ob die Route direkt dorthin oder mit einem Ladezwischenstopp an einer AC- oder DC-Ladestation führen soll.
Nach der Einleitung mit Motivation, Problemstellung und Zielsetzung werden im Grundlagenkapitel Elektromobilität, Ladeinfrastruktur sowie die Prinzipien der Routenplanung erläutert. Anschließend folgt die Anforderungsanalyse, in der die bestehenden Funktionen der InCharge-App untersucht und daraus funktionale Anforderungen an die neue App abgeleitet werden. Darauf aufbauend werden im Entwurfskapitel Benutzeroberfläche und Softwarearchitektur der App konzipiert, bevor im Implementierungskapitel die praktische Umsetzung in Xcode mit SwiftUI beschrieben wird. Ein Validierungskapitel prüft die Umsetzung anhand der definierten Anforderungen. Das abschließende Fazit fasst die zentralen Ergebnisse zusammen und gibt einen Ausblick auf mögliche Erweiterungen.

Abstract

The Bachelor's thesis addresses the development of a new iOS application called *BerlinCharge*, which builds upon the core functionalities of the existing Vattenfall InCharge app and extends them with an intelligent charging station navigation feature.
In contrast to the original app, which enables users to locate charging stations on the map and start charging sessions, the newly developed application additionally provides an integrated navigation feature from the current location.
Users can navigate directly to a selected charging station. As a core feature, it is also possible to enter a destination address and choose whether the route should lead directly to that destination or include an intermediate charging stop at an AC or DC charging station.
Following the introduction, which presents the motivation, problem statement, and objectives, the fundamentals chapter explains electromobility, charging infrastructure, and the principles of route planning. This is followed by a requirements analysis that examines the existing functions of the InCharge app and derives functional requirements for the new application. Based on these requirements, the design chapter outlines the user interface and software architecture of the app, before the implementation chapter describes the practical realization in Xcode using SwiftUI. A validation chapter evaluates the implementation based on the defined requirements. The concluding chapter summarizes the main results and provides an outlook on potential future enhancements of the application.

Inhaltsverzeichnis

Abbildungsverzeichnis **1**

1 Einleitung **2**
1.1 Motivation . . . 2
1.2 Problemstellung . . . 4
1.3 Zielsetzung . . . 5
1.4 Vorgehensweise . . . 6

2 Grundlagen **7**
2.1 Grundlagen und Entwicklung der Elektromobilität . . . 7
2.1.1 Technische Grundlagen der Elektromobilität . . . 8
2.1.2 Batterie- und Speichertechnologien . . . 11
2.1.3 Historische Entwicklung der Elektromobilität in Deutschland . . . 13
2.2 Ladeinfrastruktur . . . 15
2.2.1 Technische Grundlagen der Ladestationen . . . 15
2.3 Prinzipien intelligenter Routenplanung . . . 17
2.3.1 Dijkstra-Algorithmus . . . 18

3 Anforderungsanalyse **20**
3.1 Analyse der aktuellen Incharge-App . . . 20
3.2 Use Cases . . . 25
3.3 User Stories . . . 25
3.4 Funktionale Anforderungen . . . 26
3.5 Nicht-funktionale Anforderungen . . . 27

4 Entwurf **28**
4.1 Entwurf der Benutzeroberfläche . . . 28
4.1.1 Grunddesign . . . 28
4.1.2 HomeView . . . 29
4.1.3 MapView . . . 29
4.1.4 StationView . . . 30
4.2 Entwurf der Softwarearchitektur . . . 31
4.2.1 Ladestationen einbinden . . . 31
4.2.2 Ausgewählte Elektroautos . . . 32
4.2.3 Umlaufgrenze Berlins . . . 32
4.2.4 Apple Symbols Bibliothek . . . 33
4.2.5 Funktionsablauf der App . . . 33
4.2.6 Systemarchitektur . . . 35

5 Implementierung **36**
5.1 Projektaufbau . . . 36
5.2 Einbindung der Daten . . . 37
5.3 Implementierung der Benutzeroberfläche . . . 38
5.3.1 InchargeView . . . 38
5.3.2 HomeView . . . 39
5.3.3 MapView . . . 41
5.3.4 StationView . . . 47

6 Validierung **48**
6.1 Ergebnisse der Implementierung in der Demo-App . . . 48
6.1.1 HomeView . . . 48
6.1.2 MapView . . . 50
6.1.3 StationView . . . 56
6.2 Verifizierung der Anforderungen . . . 58

7 Fazit **59**
7.1 Ausblick . . . 59

A Diverses **60**
A.1 Demonstrationsvideo . . . 60
A.2 Quellcode . . . 60
A.3 Kürzungen . . . 61
A.3.1 Aus dem Kapitel Grundlagen und Entwicklung der Elektromobilität . . . 61
A.3.2 Aus dem Kapitel Prinzipien intelligenter Routenplanung . . . 65

Literaturverzeichnis **68**

Abbildungsverzeichnis

1.1 Startbildschirm der InCharge-App . . . 3
1.2 Detailansicht der Karte nach einem Zoom . . . 4
1.3 Auswahl eines Ladepunkts durch Klick auf der Karte . . . 5

2.1 Grundprinzip Elektromotor [Eigene Darstellung; Aufbau nach Doppelbauer (2025)] 9
2.2 Grundprinzip Elektromotor Wechselstrom [Eigene Darstellung; Aufbau nach Doppelbauer (2025)] . . . 9
2.3 Grundprinzip Elektromotor Drehstrom [Eigene Darstellung; Aufbau nach Doppelbauer (2025)] . . . 10
2.4 Ladung mit 1C [Eigene Darstellung] . . . 11
2.5 Pulsladung Close-Up [Eigene Darstellung] . . . 13
2.6 Übersicht Ladeinfrastrukturtechnologien [Eigene Darstellung; Aufbau nach Kampker (2024)] . . . 16
2.7 Ladetechnologie AC [Eigene Darstellung; Aufbau nach Kampker (2024)] . . . 16
2.8 Ladetechnologie DC [Eigene Darstellung; Aufbau nach Kampker (2024)] . . . 17
2.9 Dijkstras Algorithmus [Eigene Darstellung] . . . 18
2.10 Dijkstras Algorithmus Limitierung [Eigene Darstellung] . . . 19

3.1 Map-Tab Vattenfall Incharge-App . . . 20
3.2 Filterfunktion Suchleiste im Map-Tab Vattenfall Incharge-App . . . 21
3.3 Legende Map-Tab Vattenfall Incharge-App . . . 22
3.4 Ladepunkt Map-Tab Vattenfall Incharge-App . . . 23
3.5 Detailansicht Ladepunkt Map-Tab Vattenfall Incharge-App . . . 23
3.6 Stations-Tab Vattenfall Incharge-App . . . 24
3.7 UML-Use-Case-Diagramm [Eigene Darstellung] . . . 25

4.1 App Icon [Eigene Darstellung; Das App-Icon wurde mit Seedream 4.0 von Bytedance erstellt] . . . 28
4.2 Home-Tab Mock-Up [Eigene Darstellung] . . . 29
4.3 Map-Tab Mock-Up [Eigene Darstellung; Karte nach inside digital (2025)] . . . 30
4.4 Stations-Tab Mock-Up [Eigene Darstellung] . . . 31
4.5 Abzug aus dem MyIncharge-Portal von Vattenfall [Eigene Darstellung] . . . 32
4.6 Ortsteile von Berlin odis berlin (2025) . . . 33
4.7 Sitemap-Navigationsdiagramm [Eigene Darstellung] . . . 34
4.8 UML-Komponentendiagramm [Eigene Darstellung] . . . 35

5.1 Kürzeste Distanz Station zum Segment [Eigene Darstellung; Koordinatensystem nach Mathestunde.com (2025)] . . . 43
5.2 Kürzeste Distanz Station zur Polylinie [Eigene Darstellung; Koordinatensystem nach Mathestunde.com (2025)] . . . 44

6.1 HomeView Ergebnis Listenansicht der Elektroautos [Eigene Darstellung] . . . 48

6.2 HomeView Ergebnis Detailansicht VW ID.7 [Eigene Darstellung] 49
6.3 Standort Abfrage [Eigene Darstellung] 50
6.4 Kartenansicht [Eigene Darstellung] 51
6.5 DetailView der Station [Eigene Darstellung] 52
6.6 Marker der Hochschule auf der Karte [Eigene Darstellung] 53
6.7 Berechnete Route mit einem AC-Ladestop [Eigene Darstellung] 54
6.8 Berechnete Route mit einem DC-Ladestop [Eigene Darstellung] 55
6.9 Ladestationen im StationView [Eigene Darstellung] 56
6.10 DetailView einer Ladestation [Eigene Darstellung] 57
6.11 Finaler Stand der funktionalen Anforderungen [Eigene Darstellung] 58
6.12 Finaler Stand der nicht-funktionalen Anforderungen [Eigene Darstellung] . . . 58

A.1 BHT Cloud Code und Videos [Eigene Darstellung] 60
A.2 Lohner-Porsche mit Allradantrieb (1900) Wikipedia (2025c) 61
A.3 Entladung mit 1C [Eigene Darstellung] 62
A.4 Ruhespannungskennlinie durch Pulsmessung [Eigene Darstellung] 62
A.5 Passives Balancing [Eigene Darstellung] 63
A.6 Aktives Balancing [Eigene Darstellung] 63
A.7 KEBA KeContact P30 X-Series Green Edition Memodo (2025) 64
A.8 Negativer Zyklus [Eigene Darstellung] 65
A.9 Bellman-Ford-Algorithmus [Eigene Darstellung] 66
A.10 Tabelle zu Iteration 1 [Eigene Darstellung] 66
A.11 Tabelle zu Iteration 2 [Eigene Darstellung] 67
A.12 Tabelle zu Iteration 3 [Eigene Darstellung] 67

Kapitel 1

Einleitung

Um das Grundproblem der Elektromobilität zu adressieren, investieren Unternehmen verstärkt in den Ausbau von Ladesäulen, um der steigenden Nachfrage gerecht zu werden. Elektroautos gelten dabei als zentraler Ansatz für eine fossilfreie Zukunft im Verkehrssektor. Ob sie tatsächlich ‚sauberer' als herkömmliche Verbrennungsmotoren sind, wird weiterhin kontrovers diskutiert. Dafür ist es unerlässlich, Ladeinfrastruktur bereitzustellen, damit die Energiespeicher der Fahrzeuge bedarfsgerecht geladen werden können. Der Ausbau dieser Infrastruktur ist in vollem Gange, wie sich etwa an der wachsenden Zahl von Ladesäulen in Berliner Innenstädten beobachten lässt electrive (2025). So finden sich Ladepunkte inzwischen in nahezu jedem Parkhaus, als Wallbox-Lösungen an Gebäudewänden, auf Parkplätzen von Supermärkten und Schnellrestaurants oder als öffentlich zugängliche Stationen an Straßen. Viele Anbieter werben zudem mit kostenlosem Laden oder reservierten Parkplätzen für Elektroautos. Mittlerweile wird die Ladeinfrastruktur deutschlandweit kontinuierlich attraktiver gestaltet. Trotz der Tatsache, dass das Land historisch als Wiege der Verbrennertechnik gilt und das Thema Elektromobilität nach wie vor besonders kritisch betrachtet wird. Die Bedeutung liegt nicht nur im Ladevorgang selbst. Vor allem in Städten ist die Navigation zur Ladesäule und weiter zum nächsten Ziel von großer Relevanz. Nur so lassen sich Umwege, Staus und damit verbundene Emissionen vermeiden - ein Aspekt, der eng mit dem aktuellen Strommix in Deutschland verknüpft ist.

Der gesamte Text wurde zur sprachlichen und grammatikalischen Korrektur mithilfe von Perplexity überarbeitet. Inhalt, Struktur und Ausarbeitung stammen vollständig vom Autor selbst.

Zur besseren Lesbarkeit wird in dieser Bachelorarbeit das generische Maskulinum verwendet. Die in dieser Arbeit verwendeten Personenbezeichnungen beziehen sich - sofern nicht anders kenntlich gemacht - auf alle Geschlechter.

1.1 Motivation

Während meines Studiums im Bereich Elektromobilität begann ich ab dem dritten Fachsemester eine Werkstudententätigkeit bei der Vattenfall Smarter Living GmbH. Das Unternehmen bietet Ladelösungen für Elektrofahrzeuge an privaten Wohnhäusern, Geschäftsstandorten und im öffentlichen Raum in Großstädten an. Dort war ich insbesondere im technischen Projektmanagement für die Ladeinfrastruktur tätig, welche sowohl öffentliche als auch private Ladepunkte umfasst. Mein Aufgabenbereich umfasste primär die Planung und Auslegung von Ladeclustern sowie im späteren Verlauf auch die Wartung und Instandhaltung bereits installierter Ladestationen. Da das Thema Elektromobilität für mich sehr vielseitig und einzigartig ist, fand ich neben der technischen Planung vor allem die Fahrzeuge selbst spannend. In Berlin ist das Bild von Ladestationen und E-Autos inzwischen alltäglich geworden. Viele Automobilhersteller werben mittlerweile mit günstigen Leasingangeboten. Und da im Rahmen des Fit-for-55-

Klimaschutzprogramms ab 2035 keine neuen Fahrzeuge mit Verbrennungsmotor mehr zugelassen werden dürfen, ist der Ausbau der Ladeinfrastruktur unabdingbar geworden EUR-Lex (2025).

Neben den bei Vattenfall installierten Ladestationen, die über ein internes Backend-System von Service-Mitarbeitern verwaltet werden, gibt es auch die InCharge-App von Vattenfall. Diese ermöglicht es den Endnutzern, sämtliche Ladestationen deutschlandweit zu finden und direkt über die App - etwa durch das Scannen eines QR-Codes - einen Ladevorgang zu starten. Sowohl wichtige Informationen zur jeweiligen Ladesäule, als auch der Zahlungsverlauf und eine Favoritenliste sind in der App hinterlegt.

Bei Tesla-Fahrzeugen fällt auf, dass das integrierte Navigationssystem bei niedrigem Ladezustand auf dem Display zur nächsten Ladesäule führt und sogar während der Fahrt alternative Routen und Ladepunkte in Echtzeit berechnet. Dieses innovative, zielgerichtete Routing war letztlich der Auslöser für die Themenwahl meiner Arbeit. Im Folgenden ist ein Bildschirmfoto aus der Incharge-App abgebildet. Die Ansicht öffnet sich direkt nach dem Start der App Abbildung 1.1.

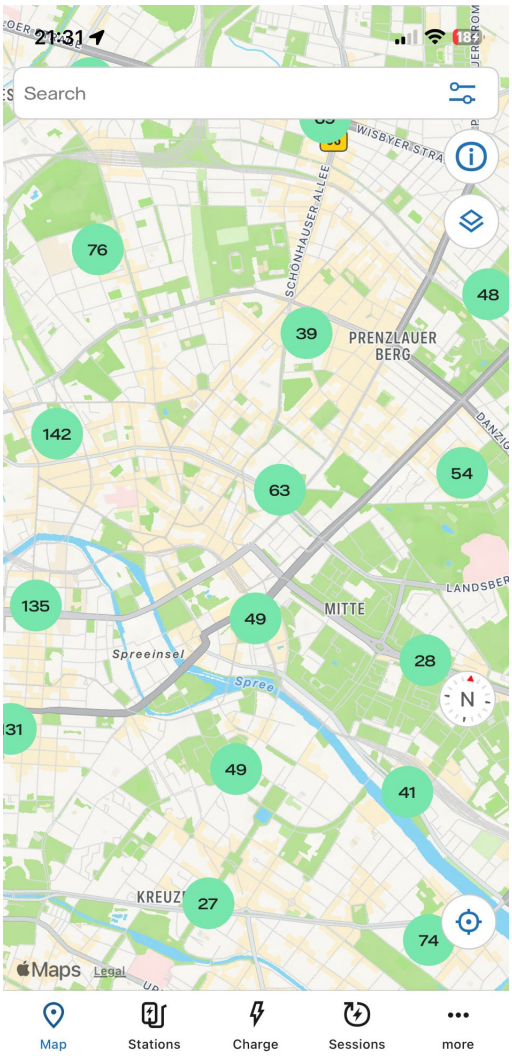


Abbildung 1.1: Startbildschirm der InCharge-App
Vattenfall (2025)

1.2 Problemstellung

Das Feature der intelligenten Ladesäulen-Navigation fehlt bislang in der InCharge-App von Vattenfall. Die InCharge-App erlaubt das Anzeigen von Ladesäulen und das Starten von Ladevorgängen darüber, jedoch keine Navigation zu diesen. Es ist demnach im realen Szenario eine zweite zusätzliche App nötig. Der Nutzer sollte die Möglichkeit haben, die Vattenfall InCharge-App im Fahrzeug als Navigationsanwendung zu starten, eine Zieladresse einzugeben und anzugeben, ob er auf dem Weg laden oder direkt zu einer Ladestation navigieren möchte. Somit stellt dieses Feature nicht bloß ein technisches Add-on dar, sondern einen vollwertigen Ersatz für konventionelle Routenplaner und letztlich einen potentiellen dauerhaften Begleiter für alle Elektroautofahrer als Vattenfall-Navigations-App.

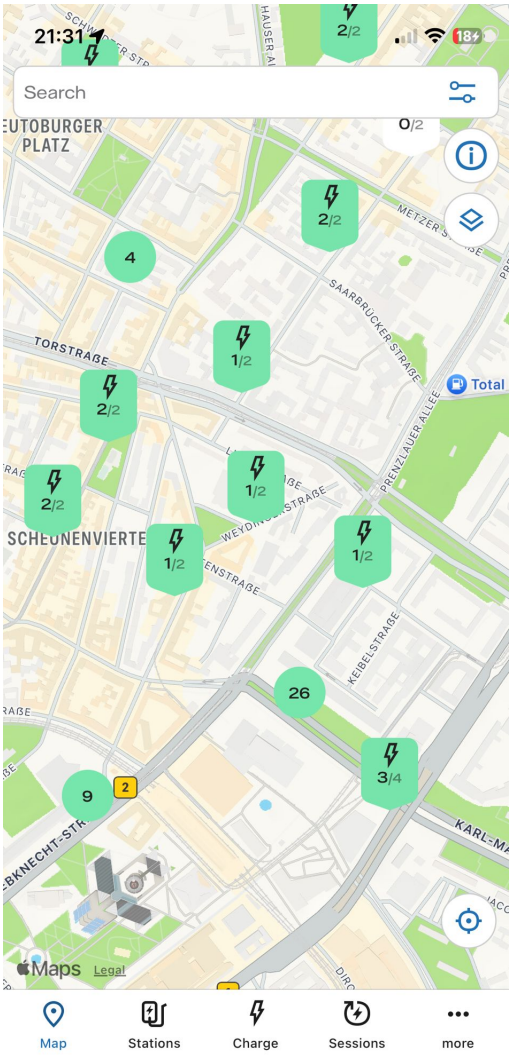


Abbildung 1.2: Detailansicht der Karte nach einem Zoom
Vattenfall (2025)

Beim weiteren Hineinzoomen in die Karte der bestehenden InCharge-App von Vattenfall werden die einzelnen Ladepunkte am jeweiligen Standort angezeigt Abbildung 1.2.

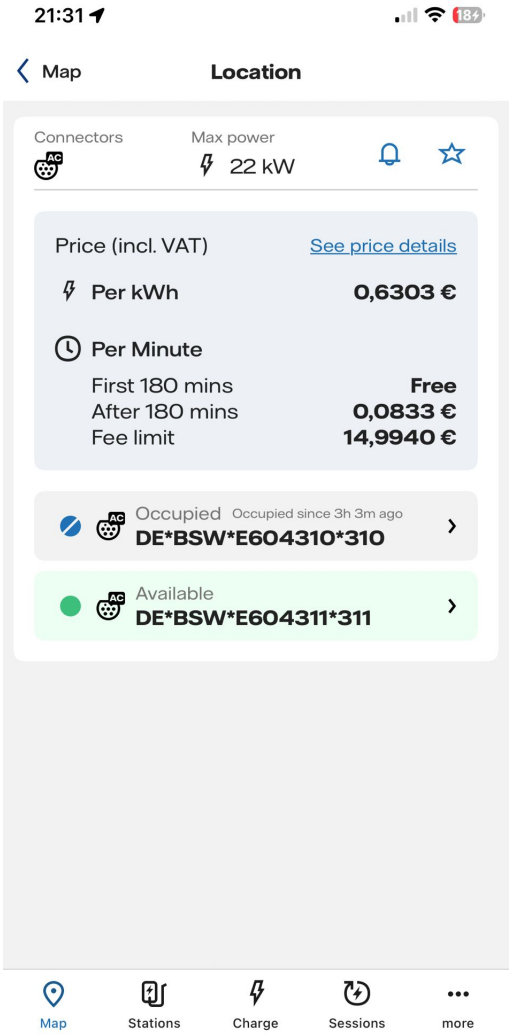


Abbildung 1.3: Auswahl eines Ladepunkts durch Klick auf der Karte
Vattenfall (2025)

Durch einen Klick auf einen Ladepunkt werden weitere Informationen wie die Ladeleistung angezeigt. Zusätzlich besteht die Möglichkeit, den Ladepunkt als Favoriten zu speichern Abbildung 1.3.

1.3 Zielsetzung

Ziel dieser Arbeit ist es, eine neue iOS-App zu entwickeln, die die wichtigsten Funktionen der bestehenden InCharge-App übernimmt und zusätzlich eine intelligente Ladesäulen-Navigation bietet. Nutzer sollen weiterhin auf eine Ladekarte zugreifen können, die alle öffentlich zugänglichen Ladestationen innerhalb Berlins anzeigt. Sie haben die Möglichkeit, eine Adresse auf der Karte einzugeben und sich vom aktuellen Standort dorthin navigieren zu lassen sowie zu wählen, ob sie auf dem Weg eine Lademöglichkeit benötigen. Alternativ ist auch eine direkte Navigation zu einer ausgewählten Ladestation möglich. Zusätzlich kann die Zieladresse frei gewählt werden. Der Nutzer kann dabei gezielt auswählen, ob er eine AC- oder eine DC-Ladestation ansteuern möchte.

Die App ermöglicht es, auf einen Ladepunkt auf der Karte zu klicken und alle relevanten Informationen wie Ladeleistung, Steckertyp, Preis und Verfügbarkeit einzusehen sowie diesen Ladepunkt als Favoriten zu speichern. Als weiteres Feature wird eine Simulation integriert. Dabei können verschiedene Fahrzeugprofile mit unterschiedlicher Batteriekapazität und Motorleistung ausgewählt werden. Die Karte zeigt die Distanz und die geschätzte Fahrzeit für die Route an. Darüber hinaus wird berechnet, ob die Strecke mit dem gewählten Profil möglich ist und welcher geschätzte Ladezustand am Ende der Route verbleibt.
In der InCharge-App von Vattenfall sind für einen Standort häufig mehrere Ladestationen hinterlegt. Da im Rahmen dieser Arbeit der Fokus auf die Navigation gelegt wird, wird pro Standort lediglich ein einzelner Ladepunkt ausgewählt. Sollten an einem Standort sowohl AC- als auch DC-Ladestationen verfügbar sein, werden diese exemplarisch gemeinsam auf denselben Standort gesetzt, um die Navigationsfunktion klar darzustellen.

1.4 Vorgehensweise

Zu Beginn wurde im ersten Kapitel die Einleitung präsentiert, die den thematischen Hintergrund, die zugrundeliegende Problemstellung, die Zielsetzung sowie die Motivation für die Bearbeitung erläutert.
Darauf aufbauend vermittelt das zweite Kapitel die theoretischen Grundlagen, die für das Verständnis der anschließenden Entwicklungsschritte relevant sind. Kapitel drei widmet sich der Analyse der Problemstellung, in deren Rahmen der aktuelle Stand betrachtet, bestehende Herausforderungen identifiziert und daraus die Anforderungen an die zu entwickelnde Anwendung abgeleitet werden.
Im Anschluss erfolgt im vierten Kapitel die Lösungskonzeption, in der die grundlegende Idee, die Softwarearchitektur und das Design der Benutzeroberfläche konzipiert werden. Darauf basiert der Umsetzungsplan, der den strukturierten Ablauf der Implementierung sowie die verwendeten Technologien und Werkzeuge beschreibt.
Kapitel fünf behandelt das Programmieren, also die praktische Umsetzung der entworfenen Konzepte in funktionalen Code. Hierbei werden sowohl die Projektstruktur als auch einzelne Komponenten und Module detailliert erläutert.
Im sechsten Kapitel folgt der Test, bei dem überprüft wird, ob die zuvor definierten Anforderungen erfüllt und die Funktionalitäten korrekt umgesetzt wurden.
Den Abschluss bildet das siebte Kapitel mit dem Fazit, welches die wesentlichen Ergebnisse, Erkenntnisse sowie einen Ausblick auf mögliche Weiterentwicklungen der Anwendung zusammenfasst.

Kapitel 2

Grundlagen

Das zweite Kapitel dient dazu, einen strukturierten Überblick über das App-Entwicklungsprojekt zu geben und die dafür notwendigen fachlichen Grundlagen zu vermitteln. Dieses Kapitel gliedert sich in drei Unterabschnitte. Zunächst werden die Grundlagen der Elektromobilität behandelt, einschließlich der Speichertechnologien und der Entwicklung der Elektromobilität in Deutschland. Darauf folgt ein Abschnitt zur Ladeinfrastruktur, in dem zentrale Akteure und technische Aspekte vorgestellt werden. Abschließend widmet sich ein Abschnitt der intelligenten Routenplanung, mit besonderem Fokus auf die effiziente Navigation im städtischen Raum. Es wird ein bekannter Algorithmus zur Navigation vorgestellt, der für die intelligente Routenplanung von Bedeutung ist.

2.1 Grundlagen und Entwicklung der Elektromobilität

Unter dem Begriff Elektromobilität werden alle Fahrzeuge verstanden, die mithilfe elektrischer Energie über einen Elektromotor angetrieben werden. Nach der Definition der Bundesregierung zählt hierzu jedes Fahrzeug, das seine Energie aus dem Stromnetz bezieht Mobil (2025). Grundsätzlich lässt sich zwischen rein elektrischen Fahrzeugen und weiteren Stufen der Elektrifizierung unterscheiden. Ein Elektrofahrzeug verfügt dabei über einen elektrischen Antriebsstrang sowie einen Energiespeicher. Zur Elektromobilität gehören nicht nur Elektroautos, sondern auch elektrisch betriebene Fahrzeuge im öffentlichen Verkehr, wie beispielsweise Eisenbahnen, Straßenbahnen oder Lieferfahrzeuge der Post. Ebenso zählen elektrisch angetriebene Fahrzeuge für den privaten Gebrauch dazu (zum Beispiel E-Scooter, Elektrofahrräder oder Elektrolongboards). Es kann somit grundsätzlich zwischen Personen- und Güterverkehr unterschieden werden. Da der Schwerpunkt dieser Arbeit auf Elektroautos liegt, wird der Fokus im Folgenden auf die verschiedenen Typen von Elektrofahrzeugen innerhalb des Individualverkehrs gerichtet Karle (2022).
Wie bereits erwähnt, lassen sich unter dem Begriff Elektrofahrzeug verschiedene Typen klassifizieren. Diese unterscheiden sich primär durch ihre Antriebskonzepte und Lademöglichkeiten:

- Battery Electric Vehicle (BEV): Rein elektrisches Fahrzeug mit ausschließlich batterieelektrischem Antrieb.
- Range Extended Electric Vehicle (REEV): BEV mit zusätzlichem Verbrennungsmotor, der ausschließlich als Generator zur Reichweitenerweiterung dient. Der Verbrennungsmotor lädt den Akku und wirkt nicht direkt auf den Antriebsstrang.
- Klassisches Hybridfahrzeug: Kombinierter Antriebsstrang aus Elektro- und Verbrennungsmotor. Der Akku wird ausschließlich durch Rekuperation und während der Fahrt geladen.

- Plug-in-Hybrid (PHEV): Hybridfahrzeug mit extern ladbarem Akku über das Stromnetz, ähnlich wie bei BEVs.

Es existiert somit eine Vielzahl unterschiedlicher Antriebskonzepte, die je nach Einsatzzweck spezifische Vorteile bieten Karle (2022).
Um die Diskussion über die Umweltfreundlichkeit von Elektrofahrzeugen sachlich einordnen zu können, wird deren Umweltbilanz in drei Kategorien gegliedert: Herstellung, Nutzung und Wiederverwertung. Neben der Fertigung spielt insbesondere die Nutzungsphase sowie das Recycling am Lebensende des Fahrzeugs eine entscheidende Rolle. Eine sehr differenzierte und umfassende Analyse der Umweltbilanz der Elektromobilität wurde bereits in einer frühen Phase vom ifeu-Institut für Energie- und Umweltforschung im Rahmen des vom Bundesministerium für Umwelt, Naturschutz und Reaktorsicherheit (BMU) geförderten Projekts „UMBReLa“ durchgeführt und im Oktober 2011 veröffentlicht. Die Ergebnisse zeigen:

- Bei der aktuellen Zusammensetzung des deutschen Strommixes sind Elektrofahrzeuge hinsichtlich der Klimabilanz mit Fahrzeugen mit Verbrennungsmotor vergleichbar.
- Wird jedoch überwiegend erneuerbare Energie zum Laden genutzt, verbessert sich die Klimabilanz der Elektrofahrzeuge deutlich Karle (2022).

Den größten Einfluss auf die Klimabilanz während der Herstellung hat die Batterie. Verbesserungen in Energie- und Leistungsdichte sowie eine verlängerte Lebensdauer wirken sich positiv auf das Gesamtergebnis aus. Ein weiterer Ansatz besteht im sogenannten Second-Life-Cycle, bei dem Batterien nach ihrer Nutzung im Fahrzeug als stationäre Energiespeicher mit reduzierter Kapazität weiterverwendet werden, bevor sie recycelt werden. In der Nutzungsphase werden unter anderem Lärm- und Schadstoffemissionen sowie der Energieverbrauch betrachtet. Elektrofahrzeuge sind insbesondere im Stadtverkehr deutlich leiser als Verbrenner, da keine motorbedingten Geräusche entstehen. Außerorts dominieren hingegen die Rollgeräusche der Reifen, bei denen zwischen beiden Fahrzeugtypen kaum Unterschiede bestehen. Hinsichtlich der Schadstoffemissionen zeigt sich das Elektrofahrzeug klar im Vorteil, da der Elektromotor im Betrieb keine direkten Emissionen verursacht. Wird jedoch der zugrunde liegende Strommix berücksichtigt, fällt der Unterschied in der Gesamtbilanz deutlich geringer aus. Erst bei der Nutzung von Strom aus erneuerbaren Quellen zeigt sich ein signifikanter Vorteil für das Elektrofahrzeug Karle (2022).
Um einen Beitrag zum Klimaschutz zu leisten und die Konzentration von Treibhausgasen in der Atmosphäre zu reduzieren, hat die Europäische Union eine Verordnung zur Begrenzung der CO_2-Emissionen von Kraftfahrzeugen erlassen. Seit dem Jahr 2020 gilt ein Grenzwert von $95\,\mathrm{g\,CO_2/km}$. Dieser Wert steht in direktem Zusammenhang mit dem Kraftstoffverbrauch des Fahrzeugs und entspricht bei einem Benzinfahrzeug einem Verbrauch von etwa $4{,}1\,\mathrm{L/100\,km}$. Reine Elektrofahrzeuge werden im Rahmen dieser Regelung mit $0\,\mathrm{g\,CO_2/km}$ bewertet. Plug-in-Hybride werden je nach elektrischer Reichweite in einer entsprechend niedrigeren Emissionsklasse eingestuft. Durch dieses System werden Automobilhersteller für einen höheren Anteil elektrischer Fahrzeuge in ihrem Absatz positiv angerechnet Karle (2022).

2.1.1 Technische Grundlagen der Elektromobilität

Beim klassischen Verbrennungsmotor wird die im Treibstoff enthaltene chemische Energie durch Verbrennung und mechanische Prozesse in Drehmoment umgewandelt. Dabei wirkt ein Zusammenspiel verschiedener Komponenten, die durch Kompression und kontrollierte Verbrennung die erforderliche Kraft auf die Antriebsräder übertragen. Im Gegensatz dazu nutzt der Elektromotor elektrische Energie als Antriebsquelle. Sein Funktionsprinzip basiert auf der Wirkung von Magnetfeldern. Ein Magnet besitzt stets einen Nord- und einen Südpol, gemeinsam bezeichnet als Polpaar. Es gilt das physikalische Prinzip, dass gleichnamige Pole einander abstoßen, während ungleichnamige sich anziehen Doppelbauer (2025).

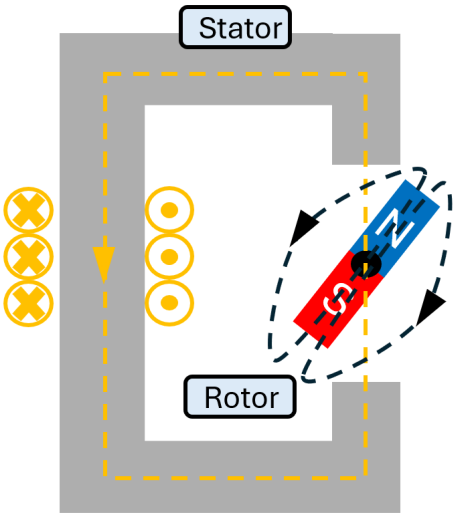


Abbildung 2.1: Grundprinzip Elektromotor [Eigene Darstellung; Aufbau nach Doppelbauer (2025)]

Grundsätzlich besteht ein Elektromotor aus einem Stator und einem Rotor. Der Rotor fungiert dabei als beweglicher Teil und trägt die magnetischen Pole. Im Stator hingegen befinden sich Spulen, die durch Stromdurchfluss ein Magnetfeld erzeugen. In Abbildung 2.1 erzeugt die Spule das gelb markierte Magnetfeld. Dadurch wird der Rotor durch das Magnetfeld ausgerichtet und bewegt sich in Richtung der Senkrechten. In dieser Position verharrt der Rotor jedoch und es entsteht keine weitere Drehbewegung. Bei einem Wechselstrommotor kommen meist zwei Spulenpaare zum Einsatz, die in Reihe um den Rotor angeordnet sind Doppelbauer (2025).

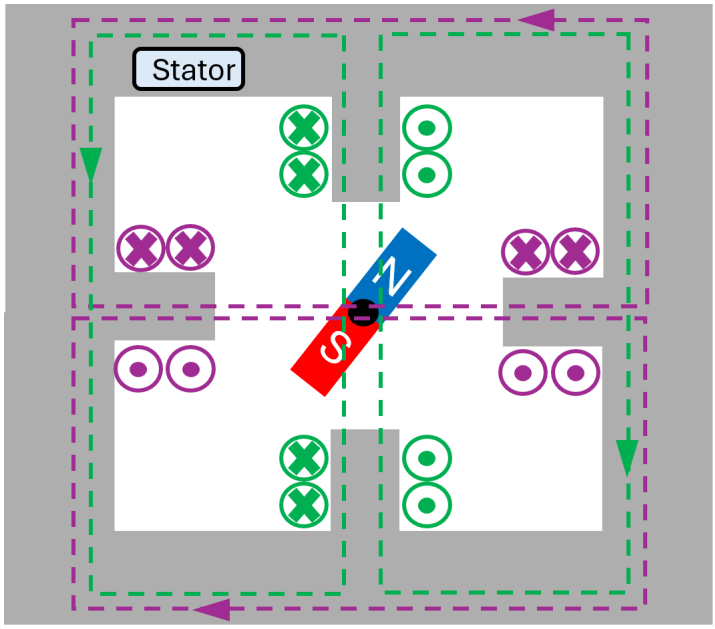


Abbildung 2.2: Grundprinzip Elektromotor Wechselstrom [Eigene Darstellung; Aufbau nach Doppelbauer (2025)]

Um eine Drehbewegung zu erzeugen, aktiviert man zunächst das erste Spulenpaar, wodurch der

Rotor aufgrund der magnetischen Wechselwirkung in Bewegung versetzt wird. Sobald er abgestoßen wird, deaktiviert man das erste Spulenpaar und schaltet das zweite ein, sodass sich der Rotor weiterdreht. Anschließend aktiviert man das erste Spulenpaar erneut, diesmal jedoch mit umgekehrter Polarität, während das zweite abgeschaltet wird. Durch die wiederholte Umschaltung der Spulenpaare entsteht eine kontinuierliche Drehbewegung, die eine vollständige Umdrehung des Rotors ermöglicht Doppelbauer (2025) Abbildung 2.2.

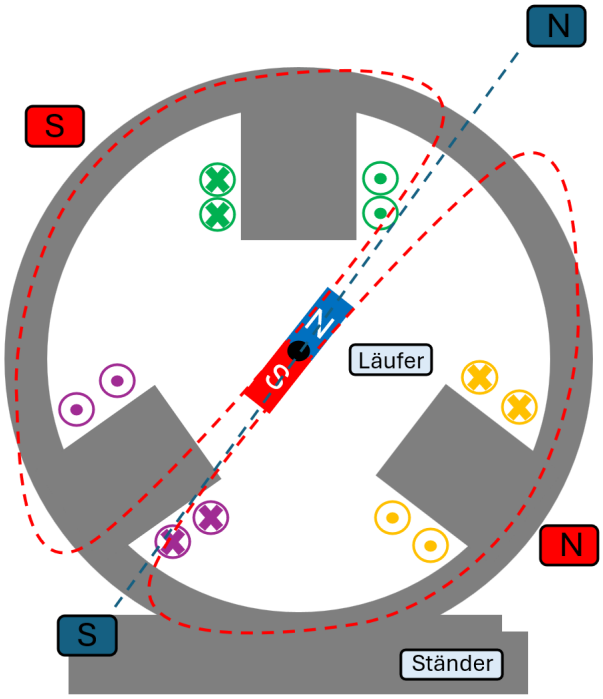


Abbildung 2.3: Grundprinzip Elektromotor Drehstrom [Eigene Darstellung; Aufbau nach Doppelbauer (2025)]

Dieses Grundprinzip lässt sich auch auf Drehstromsysteme übertragen. Dabei stehen drei Phasen zur Verfügung, die jeweils um 120° verschoben verlaufen und abwechselnd Strom an die Wicklungen liefern. Dadurch können die Spulenpaare zeitlich versetzt angesteuert werden, um dem Rotor ein kontinuierliches Drehmoment zu verleihen. Die Spulen sind räumlich um 120° versetzt angeordnet, während die Phasen des Drehstroms ebenfalls um 120° zeitlich gegeneinander verschoben sind. Auf diese Weise entsteht ein gleichmäßiges, rotierendes Magnetfeld Abbildung 2.3. Ein System mit drei Spulenpaaren besitzt somit zwei Pole. In elektrischen Antriebssystemen von Fahrzeugen werden häufig deutlich mehr Polpaare verwendet, beispielsweise neun bis zwölf Spulen, was mindestens sechs bis neun Pole ergibt. Die Drehzahl des Statorfelds hängt dabei von der Frequenz des Stroms und der Anzahl der Polpaare ab Doppelbauer (2025).

$$n_s = \frac{f}{p} \tag{2.1}$$

In Elektrofahrzeugen kommen verschiedene Varianten solcher Drehstrommaschinen zum Einsatz. Die am häufigsten verwendeten Typen sind Permanentmagneterregte Synchronmaschinen (PMSM), wie sie etwa im BMW i3 oder VW ID.3 verbaut sind, fremderregte Synchronmaschinen (FESM), die beispielsweise im Renault ZOE eingesetzt werden, sowie Asynchronmaschinen (ASM), die unter anderem im Audi e-tron Verwendung finden Doppelbauer (2025).

2.1.2 Batterie- und Speichertechnologien

In den meisten Elektrofahrzeugen kommen Lithium-Ionen-Akkumulatoren zum Einsatz, da sie sich aufgrund ihrer hohen Energie- und Leistungsdichte gegenüber anderen Batterietechnologien durchgesetzt haben. Da die Herstellung eines Akkupakets mit hohen Kosten verbunden ist, stellt dieser Bestandteil den größten Kostenfaktor eines Elektrofahrzeugs dar. Eine Reduktion der Batteriekosten wirkt sich daher direkt auf den Gesamtpreis des Fahrzeugs und somit auf die Marktnachfrage aus. Akkumulatoren basieren auf elektrochemischen Prozessen. Sowohl Batterien als auch Akkumulatoren wandeln chemische in elektrische Energie um, wobei dieser Vorgang bei Akkumulatoren reversibel ist. Sie können also entladen und anschließend wieder geladen werden. Damit ermöglichen sie die Speicherung großer Energiemengen, die bei Bedarf abgerufen werden können. Wichtige Kennwerte eines Akkumulators sind:

- die Nennspannung in Volt, die von der Zellchemie abhängt,
- die Kapazität in Amperestunden (Ah),
- die Lade- und Entladeschlussspannung, welche den zulässigen Spannungsbereich definiert.

Die gespeicherte Energie des Akkus ergibt sich aus dem Produkt aus Nennspannung und Kapazität und wird in Kilowattstunden (kWh) angegeben. Eine weitere zentrale Größe ist die sogenannte C-Rate. Sie beschreibt das Verhältnis des Lade- oder Entladestroms zur Nennkapazität und gibt somit Auskunft über die zu erwartende Lade- beziehungsweise Entladedauer Karle (2022).
Beispielsweise entspricht bei einer Zelle mit $Q = 1.4\,\mathrm{A\,h}$ eine Ladung mit einer C-Rate von $C = 1\,\mathrm{h}^{-1}$ (entspricht 1 C) einem Strom von

$$I = C \cdot Q = 1\,\mathrm{h}^{-1} \cdot 1.4\,\mathrm{A\,h} = 1.4\,\mathrm{A}. \tag{2.2}$$

Über eine Stunde ergibt sich damit eine aufgenommene Ladung von $Q = 1.4\,\mathrm{A\,h}$.
Wird dieselbe Zelle mit einer C-Rate von $C = 2\,\mathrm{h}^{-1}$ (entspricht 2 C) geladen, also mit

$$I = C \cdot Q = 2\,\mathrm{h}^{-1} \cdot 1.4\,\mathrm{A\,h} = 2.8\,\mathrm{A}, \tag{2.3}$$

verkürzt sich die theoretische Ladezeit rechnerisch auf

$$t = \frac{Q}{I} = \frac{1.4\,\mathrm{A\,h}}{2.8\,\mathrm{A}} = 0.5\,\mathrm{h} = \frac{1}{2}\,\mathrm{Stunde}. \tag{2.4}$$

Diese theoretische Berechnung gilt in der Praxis nur eingeschränkt, da der Ladeprozess stark vom State of Health (SOH) des Akkumulators abhängt Abbildung 2.4.

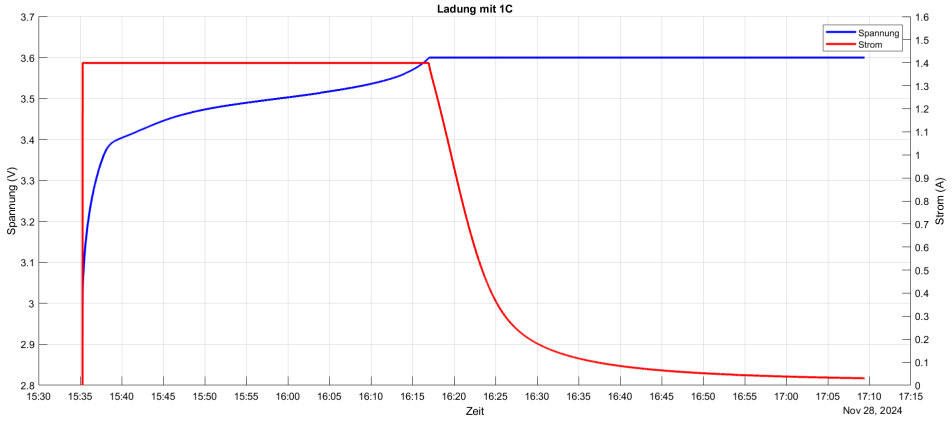


Abbildung 2.4: Ladung mit 1C [Eigene Darstellung]

Durch einen erhöhten Innenwiderstand einzelner Zellen kann die Ladeschlussspannung früher erreicht werden, wodurch weniger elektrische Ladung aufgenommen werden kann. Besonders bei großen Akkusystemen mit vielen Einzelzellen ist deshalb ein Batteriemanagementsystem (BMS) erforderlich. Dieses überwacht den Zustand der Zellen, verhindert Tiefentladung oder Überladung und stellt sicher, dass die verfügbare Energie des Gesamtsystems optimal genutzt wird. Das BMS dient somit als Schnittstelle zwischen dem Akkumulator und dem übrigen Fahrzeug. Im idealen Betrieb sollte die Zelle überwiegend zwischen 20 Prozent und 80 Prozent Ladezustand genutzt werden Doppelbauer (2025).

Im Automobilbereich besteht die Anforderung, Fahrzeuge möglichst schnell laden zu können. Insbesondere auf Langstrecken oder während kurzer Pausen, etwa beim Einkaufen. Dazu muss in kurzer Zeit eine große Strommenge in die Batterie eingespeist werden, was einer hohen C-Rate entspricht. Theoretisch verkürzt eine höhere C-Rate die Ladezeit, praktisch sind die Werte jedoch durch die Zellchemie und Temperaturgrenzen limitiert, um die Lebensdauer der Batterie zu erhalten. Bei Schnellladevorgängen, beispielsweise an einer DC-Ladestation, reagieren die Zellen besonders empfindlich, sobald der Ladezustand über 80 Prozent steigt. Aus diesem Grund ermöglichen reale Ladesysteme ein Schnellladen typischerweise von 20 Prozent auf 80 Prozent innerhalb von etwa 30 Minuten, während darüber hinaus aus Gründen des Zellschutzes die Ladeleistung reduziert wird Karle (2022).

Zum Aufladen einer Zelle existieren verschiedene Ladeverfahren. Das bekannteste ist die sogenannte IU-Ladung, auch bekannt als CCCV-Ladung (Constant Current / Constant Voltage). Dabei werden sowohl die Ladeschlussspannung als auch der maximale Ladestrom an einer elektronischen Stromquelle vorgegeben. Zu Beginn wird die entladene Zelle angeschlossen und mit konstantem Strom geladen, während die Zellspannung allmählich bis zur eingestellten Ladeschlussspannung ansteigt. Dieser erste Abschnitt wird als I-Ladung oder Konstantstromphase bezeichnet. Sobald die Ladeschlussspannung erreicht ist, schaltet die Stromquelle in den U-Betrieb (Konstantspannungsphase) um. In dieser Phase bleibt die Spannung konstant auf dem Niveau der Ladeschlussspannung, während der Ladestrom kontinuierlich abnimmt, bis der Ladevorgang abgeschlossen ist Karle (2022).

Im praktischen Versuch wurde der Ladevorgang bei einer kleinen Zelle abgebrochen, sobald der Ladestrom auf nur noch $20\,\mathrm{mA}$ gefallen war. Die Zelle war zu diesem Zeitpunkt vollständig geladen. Für den Entladevorgang werden an einer elektronischen Last die Entladeschlussspannung und der gewünschte Entladestrom eingestellt. Anschließend wird der Zelle kontinuierlich Strom entnommen. Dabei sinkt die Zellspannung allmählich ab, bis die definierte Entladeschlussspannung erreicht ist. Ab diesem Punkt darf die Zelle nicht weiter belastet werden, um Schäden zu vermeiden Karle (2022).

Da die Zellen beim tatsächlichen Betrieb nicht direkt an die IU-Ladung gekoppelt sind, sondern ihr Verhalten allein vom Ladezustand und damit von der aktuellen Zellspannung abhängt, kann das CCCV-Ladeverfahren in jedem Ladezustand angewendet werden. Selbst wenn eine Zelle nahezu vollständig geladen ist, erkennt die elektronische Quelle den entsprechenden Betriebszustand und wechselt automatisch in die Konstantspannungsphase Karle (2022).

Neben der Konstantstrom-Ladung besteht außerdem die Möglichkeit, eine Pulsladung oder Pulsentladung durchzuführen. Dabei werden kurzzeitig hohe Stromimpulse eingesetzt, um bestimmte elektrochemische Wirkungen zu erzielen oder das Ladeverhalten zu optimieren Abbildung 2.5.

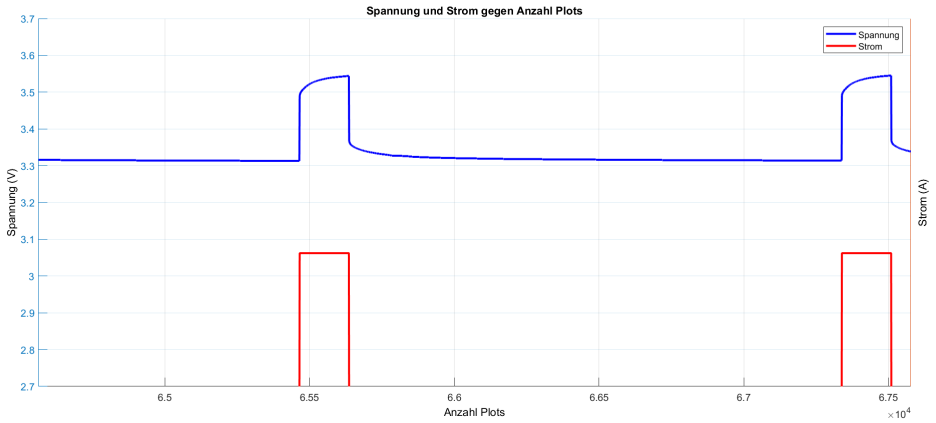


Abbildung 2.5: Pulsladung Close-Up [Eigene Darstellung]

Um eine ausreichende Energiemenge für den Betrieb eines Elektrofahrzeugs bereitzustellen, müssen zahlreiche einzelne Zellen zu einem Batteriesystem verbunden werden. Die Spannung eines solchen Systems ergibt sich aus der Anzahl der in Serie geschalteten Zellen multipliziert mit der Nennspannung einer Einzelzelle. Für ein $400\,\mathrm{V}$-System mit einer Nennspannung von $U_{\text{Zelle}} = 3.6\,\mathrm{V}$ pro Zelle ergibt sich die Anzahl der seriellen Zellen zu

$$N_s = \frac{U_{\text{System}}}{U_{\text{Zelle}}} = \frac{400\,\mathrm{V}}{3.6\,\mathrm{V}} \approx 112. \tag{2.5}$$

Neben der Spannung beeinflusst die Kapazität die Energiemenge wesentlich. Die Gesamtkapazität des Systems wird durch Parallelschaltung mehrerer Zellen erhöht. Beispielsweise soll eine Energiemenge von $E = 40\,\mathrm{kW\,h} = 40\,000\,\mathrm{W\,h}$ gespeichert werden. Die Kapazität einer einzelnen Zelle ist $Q = 1.4\,\mathrm{A\,h}$ bei Nennspannung U_{Zelle}.
Die Gesamtkapazität Q_{Gesamt} in Amperestunden für die Parallelschaltung berechnet sich aus der Energieformel:

$$E = U_{\text{System}} \cdot Q_{\text{Gesamt}} \implies Q_{\text{Gesamt}} = \frac{E}{U_{\text{System}}} = \frac{40\,000\,\mathrm{W\,h}}{400\,\mathrm{V}} = 100\,\mathrm{A\,h}. \tag{2.6}$$

Die Anzahl der parallelen Zellen N_p ergibt sich aus

$$N_p = \frac{Q_{\text{Gesamt}}}{Q} = \frac{100\,\mathrm{A\,h}}{1.4\,\mathrm{A\,h}} \approx 72. \tag{2.7}$$

Insgesamt entsteht somit ein Batteriesystem mit der Konfiguration 112S72P, also 112 Zellen in Serie mit jeweils 72 Zellen parallel, was mehr als 8000 Einzelzellen entspricht.
Die theoretisch gespeicherte Energiemenge dieses Systems beträgt

$$E_{\text{theoretisch}} = U_{\text{Zelle}} \cdot Q \cdot N_s \cdot N_p = 3.6\,\mathrm{V} \cdot 1.4\,\mathrm{A\,h} \cdot 112 \cdot 72 \approx 40.65\,\mathrm{kW\,h}. \tag{2.8}$$

2.1.3 Historische Entwicklung der Elektromobilität in Deutschland

Die Geschichte der Elektromobilität begann bereits im frühen 19. Jahrhundert mit der Entdeckung des Elektromagnetismus durch Michael Faraday im Jahr 1821. Erste Versuche, die durch elektromagnetische Kräfte erzeugte Drehbewegung praktisch zu nutzen, fanden zunächst im Modellbau und bei einfachen Spielzeugen Anwendung. So testete Thomas Davenport seinen Elektromotor an einer kleinen Modelllokomotive. In den darauffolgenden Jahrzehnten wurden Elektromotoren zunehmend in einfachen Elektrolokomotiven eingesetzt. Einen wichtigen Meilenstein

setzte Werner von Siemens mit der Vorstellung der ersten elektrischen Eisenbahn auf der Berliner Gewerbeausstellung im Jahr 1879. Drei Jahre später, 1882, präsentierte Siemens in Berlin seinen elektrischen Kutschenwagen. Im Jahr 1900 folgte in Paris das erste praxistaugliche Elektroauto, der sogenannte „Lohner-Porsche“ Karle (2022).

Dennoch verhinderten die damals schweren Bleiakkumulatoren, der begrenzte Fahrkomfort sowie die geringere Leistung im Vergleich zu Verbrennungsmotoren eine breite Akzeptanz. Elektromobilität fand daher vor allem dort Anwendung, wo Energie direkt aus dem Netz über eine Oberleitung bezogen werden konnte, beispielsweise bei Straßenbahnen und Oberleitungsbussen Doppelbauer (2025).

In bestimmten Nischen, etwa bei elektrisch betriebenen Lieferfahrzeugen wie den sogenannten „Milk Floats“ für Milch- oder Fischlieferanten, sowie bei Postfahrzeugen und Golfkarts, blieb die Technologie jedoch weiterhin in Gebrauch. Auch im Logistikbereich, insbesondere in geschlossenen Hallen, wurden Elektrofahrzeuge eingesetzt, um Abgasemissionen zu vermeiden. Mit der Ölkrise in den 1990er-Jahren und dem zunehmenden Umweltbewusstsein wuchs das Interesse an alternativen, emissionsfreien Antrieben erneut. Neue Batterietechnologien, wie die Natrium-Nickel-Batterie, fanden erstmals 1991 im BMW E1 Verwendung. Der entscheidende Durchbruch gelang jedoch mit der Entwicklung der Lithium-Ionen-Batterie, die durch ihre hohe Energiedichte, lange Lebensdauer und geringe Selbstentladung überzeugte. Diese Technologie nutzte der Automobilhersteller Tesla als einer der ersten konsequent in der Serienproduktion. Mit Modellen wie dem Tesla Roadster (2008) und dem Tesla Model S (2012) wurde die Elektromobilität alltagstauglich und zugleich emotional aufgeladen. Ergänzt durch eigene Schnellladestationen, die sogenannten Tesla Supercharger, machte das Unternehmen erstmals auch den Langstreckenbetrieb für Elektrofahrzeuge praktikabel und attraktiv Wikipedia (2025b).

Ein zentraler Vergleichsaspekt zwischen Verbrennungs- und Elektromotoren ist die Effizienz. Der Wirkungsgrad moderner Verbrennungsmotoren liegt bei etwa 40 Prozent, wobei der Großteil der im Kraftstoff enthaltenen Energie in Wärme umgewandelt und nicht in mechanisches Drehmoment umgesetzt wird. Elektromotoren hingegen erreichen Wirkungsgrade von über 90 Prozent. Zudem können sie während des Bremsvorgangs durch Rekuperation einen Teil der Bewegungsenergie zurückgewinnen, indem der Motor kurzzeitig als Generator arbeitet. Ein weiterer Vorteil liegt im Drehmomentverhalten: Elektromotoren liefern bereits bei niedrigen Drehzahlen ein hohes Drehmoment, was zu einer schnellen und gleichmäßigen Beschleunigung führt. Aufgrund der hohen möglichen Drehzahlen benötigen die meisten Elektrofahrzeuge nur einen einzigen Gang, wodurch ein klassisches Schaltgetriebe überflüssig wird. Das Resultat ist eine nahezu unterbrechungsfreie Beschleunigung.

Gegen den Elektromotor sprechen bislang die hohen Anschaffungskosten, die überwiegend auf die teure Batterieproduktion zurückzuführen sind. Die Kosten pro Kilowattstunde Energiespeicher haben sich jedoch in den letzten Jahren deutlich verringert und sich bis 2020 gegenüber 2013 nahezu auf etwa 10 000 € für eine Batteriekapazität von 20 kWh halbiert Karle (2022). Bei der wirtschaftlichen Betrachtung dürfen die im Vergleich deutlich geringeren Betriebskosten nicht außer Acht gelassen werden. Kritisch gesehen wird hingegen weiterhin die Reichweite. Zwar werben Hersteller mit Werten von bis zu knapp 800 Kilometern Chen (2025), diese werden jedoch meist unter idealen Testbedingungen erzielt. In der Praxis fällt die Reichweite - insbesondere bei niedrigen Temperaturen oder erhöhter Innenraumnutzung durch Heizung, Klimaanlage und Infotainmentsysteme - deutlich geringer aus. Für den alltäglichen Stadt- und Kurzstreckenverkehr reicht die Reichweite in der Regel aus, insbesondere wenn das Fahrzeug regelmäßig über Nacht geladen wird. Für Langstreckenfahrten und Pendelverkehre gewinnt der Ausbau schneller und leistungsfähiger Ladeinfrastruktur jedoch zunehmend an Bedeutung Kampker (2024).

2.2 Ladeinfrastruktur

Neben dem eigentlichen Fahrbetrieb stellt das Aufladen die Voraussetzung der Elektromobilität dar. Unabhängig von der Motorleistung oder der Kapazität des Akkumulators muss ein Elektrofahrzeug regelmäßig geladen werden. Während ein Fahrzeug mit Verbrennungsmotor innerhalb weniger Minuten betankt werden kann und anschließend Hunderte von Kilometern zurücklegt, steht die Elektromobilität vor der Herausforderung, vergleichbare Versorgungseigenschaften zu erreichen.
Da die begrenzte Reichweite von Elektrofahrzeugen häufig kritisiert wird, ergeben sich zwei Wege zur Verbesserung:

- Erhöhung der Batteriekapazität
- Ausbau einer leistungsfähigen und flächendeckenden Ladeinfrastruktur

Eine höhere Verfügbarkeit und schnellere Ladeoptionen können die Reichweitenproblematik erheblich entschärfen und den Komfort im Alltag deutlich erhöhen.
Bereits heute existieren verschiedene innovative Ansätze, darunter:

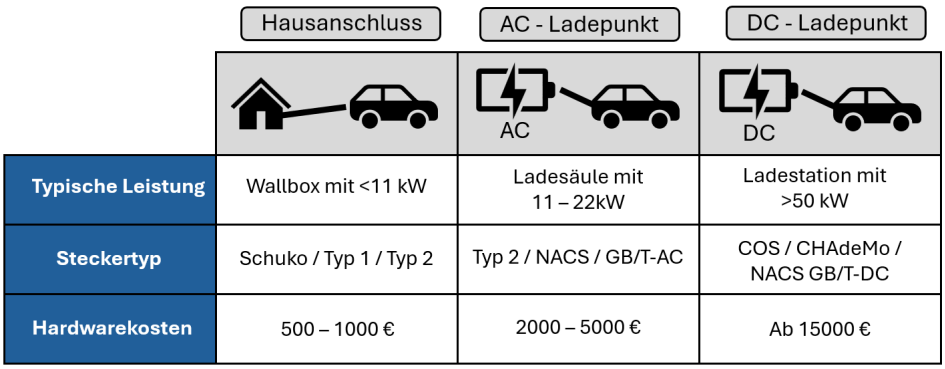
- Laden über private Photovoltaikanlagen oder sogenannte Balkonkraftwerke
- Nutzung der Fahrzeugbatterie als stationären Energiespeicher
- Bidirektionales Laden: Überschüssige Energie aus dem Fahrzeugakku kann in das häusliche Stromnetz zurückgespeist werden, um den Eigenverbrauch zu decken oder den Haushalt bei Bedarf zu versorgen.
- Integration von Photovoltaikmodulen direkt in die Fahrzeugkarosserie (z.B. auf dem Dach), um während der Fahrt zusätzliche Energie zu gewinnen und die Reichweite geringfügig zu erhöhen.

Zu den wichtigsten Akteuren der öffentlichen Ladeinfrastruktur zählen die sogenannten Charge Point Operators (CPO), die für Installation, Betrieb und Wartung der Ladesäulen verantwortlich sind. Zu den bedeutendsten Betreibern gehören unter anderem EnBW, EWE Go, E.ON Drive und Tesla. Auf der Herstellerseite sind Unternehmen wie ABB, KEBA, MENNEKES und BayWa Mobility Solutions zu nennen Neumann (2024).

2.2.1 Technische Grundlagen der Ladestationen

Im Bereich des Ladens von Elektrofahrzeugen lassen sich Ladeleistung, Ladeart und weitere technische sowie wirtschaftliche Aspekte differenziert betrachten. Ladepunkte werden in der Regel nach ihrer Ladeart, Ladeleistung, dem verwendeten Steckertyp und dem jeweiligen Stromtarif unterschieden. Für nahezu jeden Anwendungsfall - vom privaten Heimgebrauch bis hin zum öffentlichen Schnellladen - existiert die passende Lösung. Grundsätzlich kann ein Elektrofahrzeug auch über eine gewöhnliche Haushaltssteckdose geladen werden. Dabei ist die Ladeleistung jedoch durch die Auslegung des Hausnetzes stark begrenzt. Eine Dauerlast über 3,7 kW ist in der Regel nicht zulässig und selbst dieser Wert sollte nicht über längere Zeiträume erreicht werden. Typischerweise sind bis zu 16 Ampere Stromstärke technisch möglich, jedoch nicht für eine kontinuierliche Belastung ausgelegt, da die Hausinstallation keine ausreichende Wärmeabfuhr gewährleistet. Ein Dauerstrom von rund sechs bis sieben Ampere gilt als sicherer Betriebsbereich. Darüber hinaus ist das Laden unter Teillast weniger effizient, da der interne Gleichrichter des Fahrzeugs für den Betrieb bei höherer Ladeleistung optimiert ist. Das Laden über den Standard-Hausanschluss wird daher als sogenannte Notladung betrachtet. Hierfür ist ein spezieller Schuko-Adapter erforderlich, der zwischen Fahrzeugladekabel und Steckdose eingesetzt

wird. Trotz der geringen Effizienz und längeren Ladezeiten kann dieses Verfahren im Bedarfsfall eine praktikable Notlösung darstellen Kampker (2024).



Typische Leistung	Wallbox mit <11 kW	Ladesäule mit 11 – 22kW	Ladestation mit >50 kW
Steckertyp	Schuko / Typ 1 / Typ 2	Typ 2 / NACS / GB/T-AC	COS / CHAdeMo / NACS GB/T-DC
Hardwarekosten	500 – 1000 €	2000 – 5000 €	Ab 15000 €

Abbildung 2.6: Übersicht Ladeinfrastrukturtechnologien [Eigene Darstellung; Aufbau nach Kampker (2024)]

Um die technischen Einschränkungen des Ladens über eine herkömmliche Steckdose zu umgehen, werden sogenannte Wallboxen eingesetzt, die auch im privaten Bereich weit verbreitet sind. Dabei wird eine separate Leitung direkt von der Haus- oder Unterverteilung installiert, wodurch eine deutlich höhere Ladeleistung ermöglicht wird. Im Heimgebrauch sind Leistungen von bis zu 11 kW AC üblich. Die Abrechnung erfolgt in der Regel über den jeweiligen Stromanbieter, teilweise mit speziellen Elektromobilitätstarifen. Im öffentlichen Bereich finden sich ähnliche Ladeprinzipien, die jedoch in größeren Leistungsstufen realisiert werden. Grundsätzlich wird zwischen AC- (Wechselstrom-) und DC-Ladestationen (Gleichstrom) unterschieden, wobei sich beide Varianten insbesondere durch ihre Ladeleistung und technische Ausführung unterscheiden. AC-Ladestationen sind häufig am Straßenrand, in Parkhäusern oder auf den Parkplätzen von Einkaufsmärkten zu finden und bieten Ladeleistungen zwischen 11 kW und 22 kW. Die leistungsstärkste Form des Ladens stellt das DC-Schnellladen dar. Diese Stationen sind hinsichtlich Baugröße und Leistung deutlich umfangreicher und können als elektrische Entsprechung klassischer Tankstellen betrachtet werden. Je nach Standort und Infrastruktur ermöglichen sie Ladeleistungen von 50 kW bis zu 350 kW Abbildung 2.6. Öffentliche Ladesäulen sind eichrechtskonform, das heißt, jede geladene Kilowattstunde muss exakt erfasst und abgerechnet werden Kampker (2024).

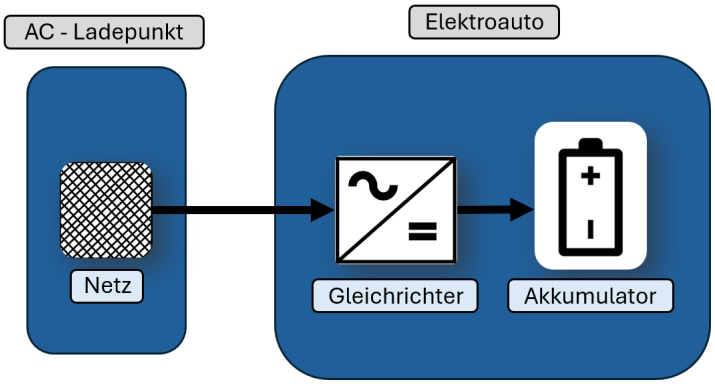


Abbildung 2.7: Ladetechnologie AC [Eigene Darstellung; Aufbau nach Kampker (2024)]

Zum Laden eines Elektrofahrzeugs wird das Fahrzeug über das Ladekabel mit der entsprechenden Ladestation verbunden. Nach der Authentifizierung oder Freischaltung beginnt der eigentliche Ladevorgang. Doch wie verläuft dieser technisch?
Der Akkumulator eines Elektrofahrzeugs arbeitet mit Gleichspannung und kann daher ausschließlich mit Gleichstrom geladen werden. Bei einer AC-Ladestation muss der aus dem Stromnetz bezogene Wechselstrom vor dem Ladevorgang in Gleichstrom umgewandelt werden. Diese Umwandlung erfolgt im Fahrzeug selbst durch den integrierten Gleichrichter, der den Wechselstrom in eine für die Batterie geeignete Gleichspannung transformiert Abbildung 2.7. AC-Ladestationen verfügen in der Regel über die Möglichkeit, ein oder zwei Fahrzeuge gleichzeitig zu laden, wobei die jeweilige Ladeleistung dynamisch auf die angeschlossenen Fahrzeuge verteilt werden kann Kampker (2024).

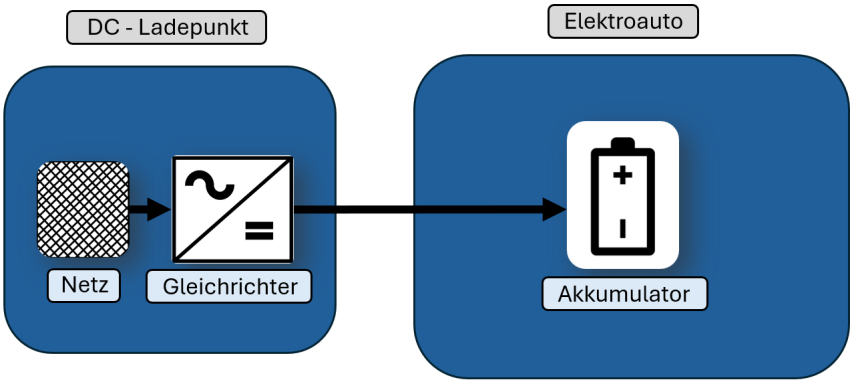


Abbildung 2.8: Ladetechnologie DC [Eigene Darstellung; Aufbau nach Kampker (2024)]

Bei DC-Ladestationen befindet sich der Gleichrichter aufgrund der höheren erforderlichen Leistung in der Ladestation selbst. Der im Fahrzeug integrierte Gleichrichter wird in diesem Fall umgangen, da der Gleichstrom direkt an den Akkumulator übergeben wird Abbildung 2.8. Da bei DC-Ladungen deutlich höhere Ströme fließen als bei AC-Ladungen, müssen sowohl der Umrichter als auch die Zuleitungen aktiv gekühlt werden. Dies zeigt sich im Betrieb insbesondere durch die hörbaren Lüftergeräusche der Station während des Ladevorgangs. Leistungen ab etwa 150 kW werden als High-Power-Charging (HPC) bezeichnet, während Ladeleistungen ab 350 kW in die Kategorie Ultra-High-Power (UHP) fallen. Typischerweise arbeiten DC-Ladesysteme im Leistungsbereich zwischen 50 kW und 350 kW bei einer Betriebsspannung von 350 bis 400 Volt. Für höhere Ladeleistungen kommen Systeme mit Spannungen von 700 bis 800 Volt zum Einsatz, um die Stromstärke zu verringern und die damit verbundene Wärmeentwicklung im Kabel sowie im System effizient zu reduzieren Karle (2022).

2.3 Prinzipien intelligenter Routenplanung

Durch den technologischen Fortschritt verfügen heute nahezu alle Smartphones über integrierte Kartenanwendungen mit Routenplanungsfunktionen. Damit lassen sich optimale Routen zwischen dem aktuellen Standort und einem gewünschten Ziel bequem berechnen. Im Sinne einer nachhaltigeren Mobilität bieten einige Kartenanbieter mittlerweile auch alternative Routen an, die zwar nicht zwingend die kürzeste Fahrzeit ermöglichen, dafür jedoch emissionsärmer sind. Bei der Berechnung werden Faktoren wie Verkehrsaufkommen, durchschnittliche Geschwindigkeit und Fahrbahnbedingungen berücksichtigt, um eine umweltfreundlichere Strecke vorzuschlagen. Gerade im städtischen Raum, wo durch ein hohes Verkehrsaufkommen eine erhöhte Abgasdichte entsteht, können solche Alternativrouten zur Verbesserung des Verkehrsflusses

beitragen und den typischen Start-Stopp-Verkehr reduzieren. Damit Karten- und Navigationssysteme optimale Routen berechnen können, muss das Verkehrsnetz mathematisch-graphisch modelliert werden. In einem solchen Modell werden Kreuzungen als Knoten und Straßenverbindungen als Kanten dargestellt. Diese Kanten können mit Gewichten versehen werden, beispielsweise basierend auf Entfernung, Fahrzeit oder Energieverbrauch. Das resultierende Netz bildet damit eine abstrahierte, aber strukturierte Darstellung der realen Infrastruktur. Auf Grundlage dieses Modells lassen sich mit verschiedenen graphbasierten Algorithmen optimale Routen berechnen. Im folgenden Abschnitt wird ein zentraler Routenplanungsalgorithmus vorgestellt und näher erläutert Hulatt (2024).

2.3.1 Dijkstra-Algorithmus

Ausgehend von dem in der Einleitung beschriebenen Modell wird für die folgenden Betrachtungen ein abstrahiertes Modell zugrunde gelegt, das aus Knoten und Kanten besteht Abbildung 2.9. Der Dijkstra-Algorithmus dient in einem solchen gewichteten Graphen zur Bestimmung des kürzesten Pfades zwischen zwei Knotenpunkten. Dabei können die Gewichte unterschiedliche Kriterien repräsentieren, beispielsweise Fahrzeit, Streckenlänge oder Energieverbrauch. Zu Beginn wird ein Startknoten festgelegt, dem eine Anfangsentfernung von null zugewiesen wird. Für alle anderen Knoten wird der Wert zunächst auf unendlich gesetzt. Anschließend werden schrittweise die minimalen Entfernungen zwischen dem aktuellen Knoten und dessen direkt verbundenen Nachbarknoten berechnet. Jeder Knoten erhält dabei den geringsten bisher bekannten Abstand zum Startpunkt Studyflix (2019).

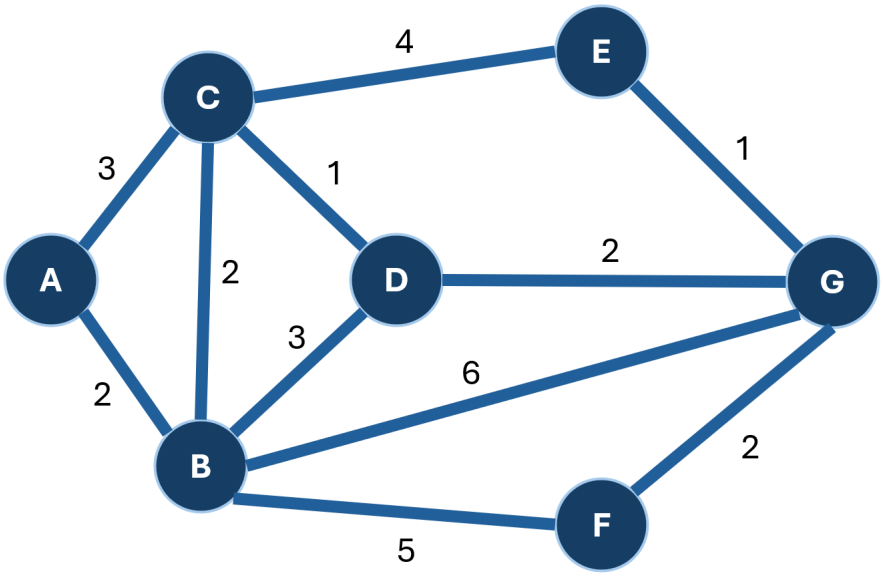


Abbildung 2.9: Dijkstras Algorithmus [Eigene Darstellung]

Ausgehend vom Startknoten wird im ersten Schritt ein benachbarter, bislang unbesuchter Knoten ausgewählt. Dabei wird stets der Knoten mit der aktuell geringsten bekannten Entfernung betrachtet. Angenommen, vom Startpunkt A aus führen zwei Verbindungen zu den Knoten B und C mit Entfernungen von 2 bzw. 3 Kilometern. Die zuvor auf „unendlich“ gesetzten Werte dieser Knoten werden nun auf 2 und 3 aktualisiert. Anschließend wird der Knoten B besucht. Vom Knoten B aus werden alle angrenzenden Knoten, beispielsweise C, D, F und G, erneut überprüft und deren Entfernungen entsprechend der Kantengewichte angepasst. Wird festgestellt, dass ein alternativer Weg - etwa A → B → C - länger ist als die bereits bekannte direkte Verbindung A →

C, bleibt der bestehende Wert unverändert. Es werden stets nur die kürzesten Weglängen gespeichert. Nachdem alle Nachbarknoten von B aktualisiert wurden, gilt B als besucht. Anschließend wird aus den verbleibenden unbesuchten Knoten derjenige mit der kleinsten aktuellen Entfernung ausgewählt, in diesem Beispiel C. Von C aus werden wiederum die Distanzen zu angrenzenden Knoten wie D und E aktualisiert. Auch dieser Knoten wird danach als besucht markiert. Das Verfahren wird fortgeführt, bis der Zielknoten G erreicht ist. Auf diese Weise ermittelt der Algorithmus den kürzesten Pfad zwischen dem Startpunkt A und dem Ziel G. Um die tatsächliche Route zu rekonstruieren, wird für jeden Knoten gespeichert, von welchem Knoten aus er erreicht wurde. Nach Beendigung des Algorithmus lässt sich die kürzeste Route bestimmen, indem diese Verbindungen rückwärts vom Ziel zum Start nachvollzogen und die Teilstrecken summiert werden. Wichtig ist, dass der Dijkstra-Algorithmus ausschließlich mit positiven Kantengewichten arbeitet. Negative Gewichte würden zu fehlerhaften Ergebnissen führen, da die zugrunde liegende Annahme der fortlaufend wachsenden Minimaldistanz verletzt würde Tree (2020).

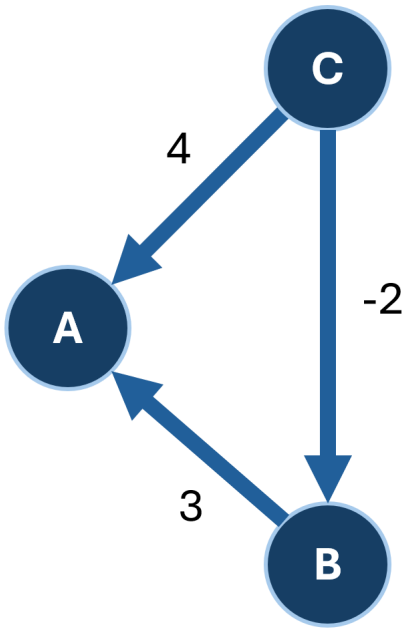


Abbildung 2.10: Dijkstras Algorithmus Limitierung [Eigene Darstellung]

Sobald der kürzeste Weg vom Startknoten A zum Knoten B gefunden wurde, wird eine alternative Teilstrecke, beispielsweise A $\rightarrow$ B über C, vom Dijkstra-Algorithmus nicht mehr berücksichtigt - selbst dann, wenn sich rechnerisch ein geringerer Wert ergeben würde, etwa wenn x = 4 - 2 < 3 wäre Abbildung 2.10. Diese Eigenschaft erklärt, warum der Dijkstra-Algorithmus ausschließlich für Graphen mit positiven Kantengewichten geeignet ist. Für Graphen, in denen negative Gewichte auftreten können, wird stattdessen der Bellman-Ford-Algorithmus angewendet, der auch bei solchen Bedingungen korrekte Ergebnisse liefert Tree (2020).

Kapitel 3

Anforderungsanalyse

Fortfolgend werden die Anforderungen der iOS-App definiert. Zuerst wird ein UML Use Case Diagramm erstellt und von dort aus die User-Stories abgeleitet. Daraufhin folgen die funktionalen und die nicht-funktionalen Anforderungen.

3.1 Analyse der aktuellen Incharge-App

Die InCharge-App von Vattenfall bietet eine Übersicht über alle zum Netzwerk gehörenden Ladestationen. Diese sind weltweit verfügbar, mit Ausnahme der Kontinente Nord- und Südamerika sowie Australien. Nach dem erstmaligen Start der App wird der Nutzer zur Freigabe des Standorts aufgefordert. Nach erfolgter Zustimmung gelangt man in den sogenannten „Map-Tab“, in dem der eigene Standort als blauer Punkt auf der Karte dargestellt wird Abbildung 3.1.

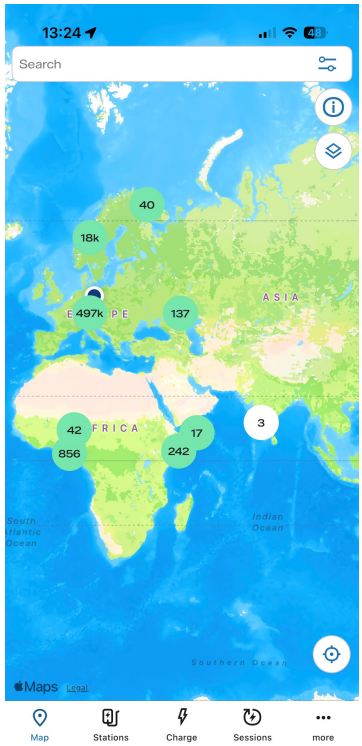


Abbildung 3.1: Map-Tab Vattenfall Incharge-App
Vattenfall (2025)

Die App bezieht ihre Kartendaten direkt aus Apple Maps, was an der Quellenangabe in der unteren linken Ecke der Karte erkennbar ist Abbildung 3.1. Am oberen Rand Abbildung 3.1 befindet sich eine Suchleiste, daneben ein Filter-Symbol Abbildung 3.2 zur Eingrenzung der Ladesuche.

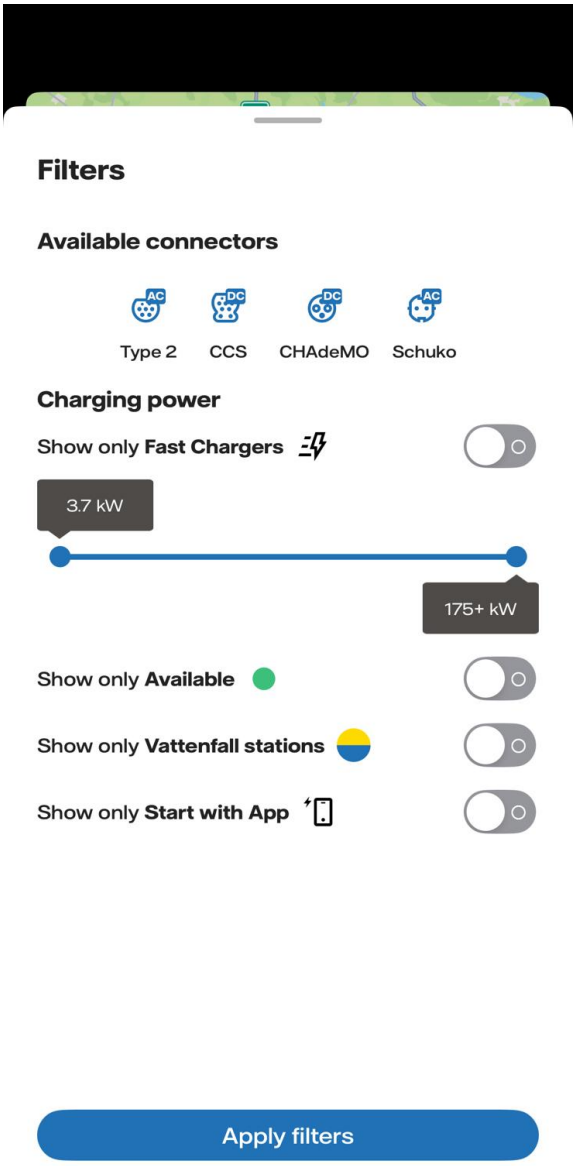


Abbildung 3.2: Filterfunktion Suchleiste im Map-Tab Vattenfall Incharge-App
Vattenfall (2025)

Direkt darunter Abbildung 3.1 sind eine Legende Abbildung 3.3 und ein Umschaltknopf Abbildung 3.1 für verschiedene Kartenansichtsmodi platziert. Ein weiterer Button unten rechts Abbildung 3.1 ermöglicht das automatische Zentrieren der Karte auf den aktuellen Standort.

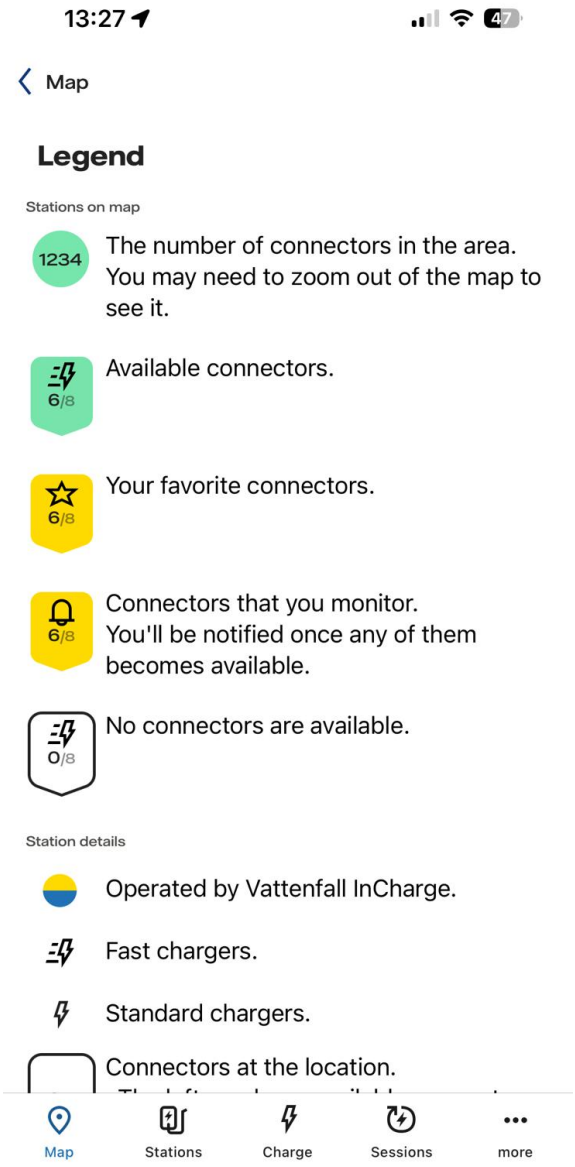


Abbildung 3.3: Legende Map-Tab Vattenfall Incharge-App
Vattenfall (2025)

Die Kartenfunktionen entsprechen dem üblichen Bedienstandard: Nutzer können hinein- und herauszoomen, die Karte drehen und verfügbare Ladepunkte betrachten. Mehrere Ladepunkte werden häufig unter einem gemeinsamen Marker zusammengefasst. Durch Antippen eines Markers öffnet sich ein Informationsfenster Abbildung 3.4 mit Details zur jeweiligen Station - darunter Steckertyp, Ladeleistung, Preis pro kWh und gegebenenfalls Preis pro Minute. Zudem werden alle verfügbaren Ladeanschlüsse der Station einzeln aufgeführt.

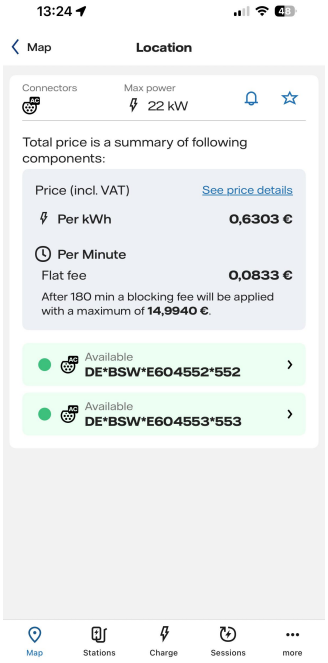


Abbildung 3.4: Ladepunkt Map-Tab Vattenfall Incharge-App
Vattenfall (2025)

Wählt der Nutzer einen Anschluss aus, öffnet sich eine Detailansicht Abbildung 3.5 mit Angaben wie Anschlussname, Straßenadresse, Netzbetreiber und weiteren technischen Daten. Über die Funktion „Navigate“ bei der Adresszeile wird ein Pop-up geöffnet, das die Wahl externer Navigations-Apps erlaubt.

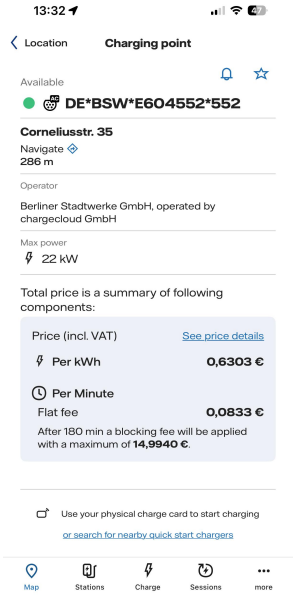


Abbildung 3.5: Detailansicht Ladepunkt Map-Tab Vattenfall Incharge-App
Vattenfall (2025)

Im Bereich „Stations“ lassen sich eigene Ladestationen einsehen, sofern der Nutzer über einen verknüpften InCharge-Account verfügt. Hier können zudem Favoriten verwaltet werden Abbildung 3.6.

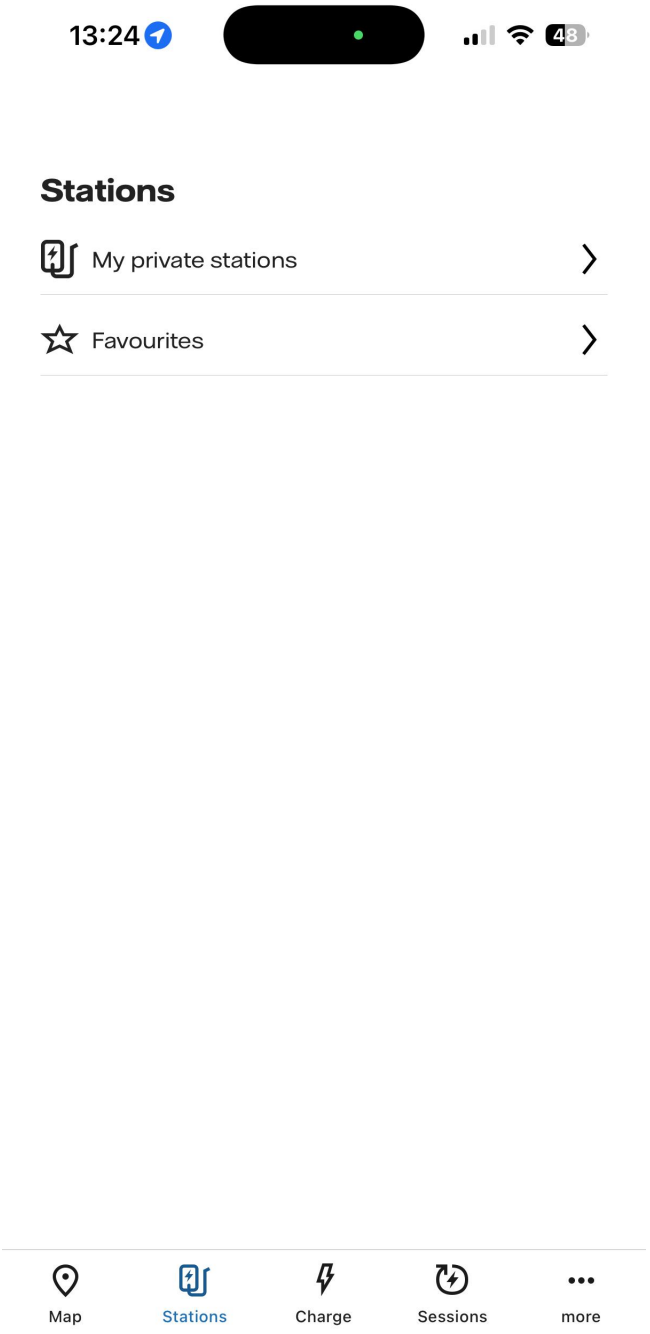


Abbildung 3.6: Stations-Tab Vattenfall Incharge-App
Vattenfall (2025)

Im „Charge-Tab“ besteht die Möglichkeit, den QR-Code einer Ladesäule direkt zu scannen, um einen Ladevorgang zu starten. Alternativ kann die Charger-ID auch manuell eingegeben werden, falls der QR-Code beschädigt oder unlesbar ist. Der „Sessions-Tab“ speichert die bisherige Ladehistorie und stellt Informationen zu vergangenen Ladevorgängen dar. Unter „More“ befindet sich

das Benutzerprofil, das nur für eingeloggte Konten verfügbar ist. Dort können Zahlungsmethoden wie Apple Pay, Bankkarten oder RFID-Ladekarten hinterlegt werden. Zusätzlich bietet dieser Bereich Optionen für Benachrichtigungen, Spracheinstellungen und weitere Kontoverwaltungsfunktionen.

3.2 Use Cases

Die folgenden Use Cases Abbildung 3.7 ergeben sich aus der Ableitung der bestehenden Incharge-App von Vattenfall und meinen gewünschten Erweiterungen zu meiner Berlincharge-App. Die Aktuere sind der Nutzer, welcher eine Ladesäule sucht und das System.

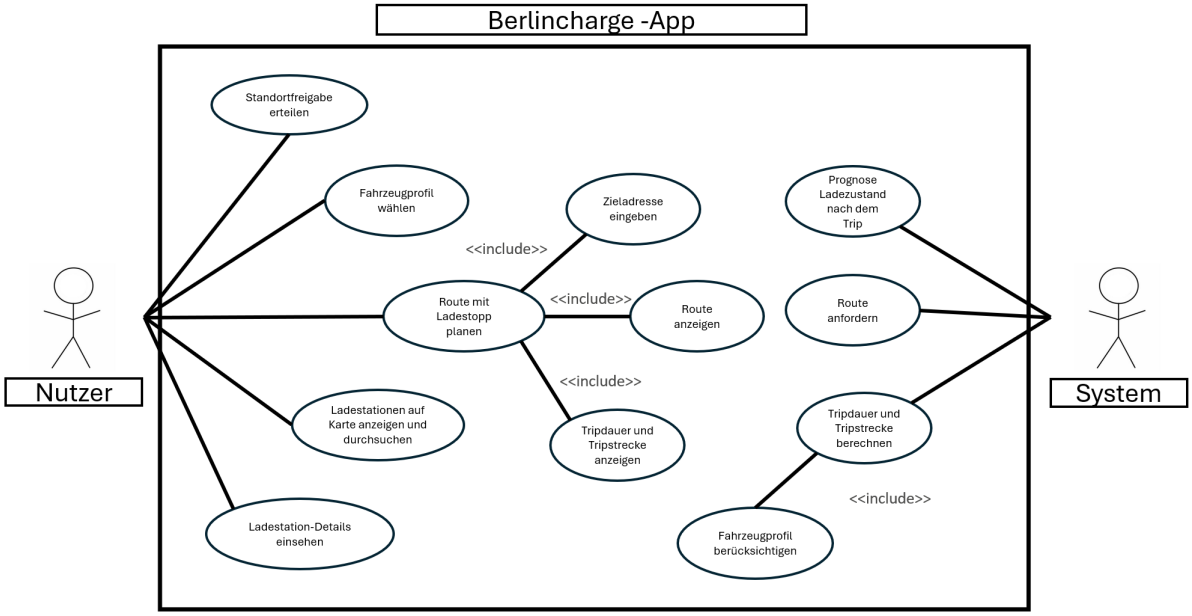


Abbildung 3.7: UML-Use-Case-Diagramm [Eigene Darstellung]

3.3 User Stories

Die folgenden User Stories ergeben sich aus den zuvor definierten Use Cases.

- US#1: Als Nutzer möchte ich die App starten und den Standort-Service akzeptieren.
- US#2: Als Nutzer möchte ich verschiedene Tabs einsehen.
- US#3: Als Nutzer möchte ich ein Fahrzeugprofil auswählen.
- US#4: Als Nutzer möchte ich auf der Karte einzelne Ladepunkte einsehen und detaillierte Informationen dazu abrufen.
- US#5: Als Nutzer möchte ich Ladepunkte als Favorit speichern.
- US#6: Als Nutzer möchte ich Dreh-, Umschau- und Zoomfunktionen nutzen.
- US#7: Als Nutzer möchte ich die Zieladresse eingeben.
- US#8: Als Nutzer möchte ich zwischen AC- und DC-Ladestopps wählen.

- US#9: Als Nutzer möchte ich die berechnete Route anzeigen lassen.
- US#10: Als Nutzer möchte ich die berechnete Tripdauer und Tripstrecke anzeigen lassen.
- US#11: Als Nutzer möchte ich eine Prognose zum Ladezustand nach Abschluss der Route erhalten.
- US#12: Als Nutzer möchte ich gezielt nach Ladestationen suchen.

3.4 Funktionale Anforderungen

Zur Validierung der Ergebnisse werden zuerst Anforderungen definiert. Diese unterteilen sich weiter in Muss- und in Sollkriterien.
Muss-Kriterien - Die Kriterien, welche die Berlincharge-App erfüllen muss:

- FA#1: Die App muss den aktuellen Nutzer-Standort abfragen.
- FA#2: Die App muss den aktuellen Nutzer-Standort auf der Karte als blauen Punkt anzeigen.
- FA#3: Die App muss alle verfügbaren Ladepunkte auf der Karte anzeigen.
- FA#4: Die App muss eine Navigationsfunktion vom aktuellen Standort zur Zieladresse bereitstellen.
- FA#5: Die App muss eine farbliche Unterscheidung von AC- und DC-Ladestationen auf der Karte haben.
- FA#6: Die App muss die Routenberechnung mit einem ausgewählten Fahrzeugprofil durchführen.
- FA#7: Die App muss Detailinformationen zu einer Ladesäule anzeigen.
- FA#8: Die App muss Detailinformationen zu einem Elektroauto anzeigen.
- FA#9: Die App muss die Auswahl und Verwaltung verschiedener Fahrzeugprofile bereitstellen.
- FA#10: Die App muss Tripdauer, Tripstrecke und den prognostizierten Ladezustand auf der Karte anzeigen.
- FA#11: Die App muss Umschau-, Dreh- und Zoomfunktionen der Karte ermöglichen.
- FA#12: Die App muss die Anzeige der berechneten Route für einen Marker ermöglichen.
- FA#13: Die App muss die Anzeige der berechneten Route für eine Ladestation ermöglichen.
- FA#14: Die App muss die Anzeige der berechneten Route mit einem Zwischenstopp AC/DC ermöglichen.
- FA#15: Die App muss die Suchfunktion für Ladestationen ermöglichen.
- FA#16: Die App muss einen Toast für die Auswahl von einem Elektroauto und einer Navigation anzeigen.

Soll-Kriterien - Die Kriterien, welche die Berlincharge-App noch zusätzlich haben kann.

- FA#17: Die App soll es ermöglichen, Benachrichtigungen zu senden, etwa bei niedrigen Ladeständen oder wichtigen Updates zu Ladepunkten.

- FA#18: Nutzer sollen eine Suchfunktion erhalten, um gezielt nach Ladepunkten anhand von Kriterien wie Standort, Steckertyp oder Verfügbarkeit filtern zu können.
- FA#19: Die App soll eine Möglichkeit zur Kontaktaufnahme mit dem Support (z.B. per E-Mail oder integriertem Kontaktformular) bieten.

3.5 Nicht-funktionale Anforderungen

- NFA#1: Die App muss die Routenberechnung jeglicher Art in weniger als 10 Sekunden bereitstellen.
- NFA#2: Die Bedienoberfläche muss intuitiv und übersichtlich gestaltet sein, maximal fünf Buttons pro Ansicht.
- NFA#3: Die Elektroautodaten dürfen nicht älter als 2 Monate sein.
- NFA#4: Die App muss für die Verwendung auf einem Smartphone optimiert sein.
- NFA#5: Die Anwendung muss mit den aktuellsten iOS-Versionen kompatibel sein.

Kapitel 4

Entwurf

In diesem Kapitel soll der Gestaltungsprozess der Berlincharge-App-Benutzeroberfläche erläutert werden.



Abbildung 4.1: App Icon [Eigene Darstellung; Das App-Icon wurde mit Seedream 4.0 von Bytedance erstellt]

4.1 Entwurf der Benutzeroberfläche

Im Rahmen der Entwicklung der Benutzeroberfläche dient die bestehende InCharge-App als konzeptionelle Orientierung. Ziel ist die Entwicklung einer kompakten Anwendung auf Basis der InCharge-App, die um eine integrierte Navigationsfunktion erweitert wird, um die parallele Nutzung externer Kartenanwendungen zu vermeiden.

4.1.1 Grunddesign

Für die Planung der Benutzeroberfläche wurden zunächst Mockups erstellt, um einen besseren Überblick über die Positionierung der einzelnen Funktionen zu erhalten und einen strukturierten Leitfaden für die Umsetzung zu entwickeln. Hierbei kam ein Template für das aktuelle iPhone 17 zum Einsatz, auf dem die ersten Konzepte entworfen wurden. Dies ermöglichte eine realistische Visualisierung und erleichtert die anschließende technische Implementierung TitanUI (2025). Der Bildschirm wird in drei Bereiche unterteilt: MapView, StationView und HomeView.

4.1.2 HomeView

Im HomeView Abbildung 4.2 wird eine Liste mit fünf ausgewählten Elektroauto-Modellen der bekanntesten deutschen Automobilhersteller angezeigt. Für jedes Modell können die wichtigsten technischen Informationen (Motorleistung, Batteriekapazität, Verbrauch auf 100 km, aktueller Ladezustand und Restreichweite) abgerufen werden. Zusätzlich besteht die Möglichkeit, ein Fahrzeug auszuwählen, das als Referenz innerhalb der Anwendung dient.

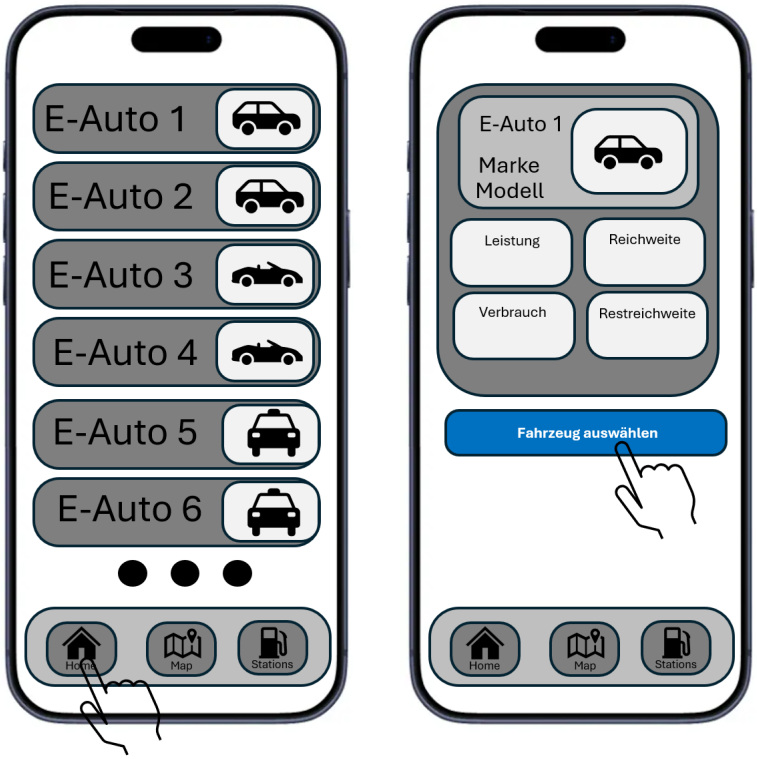


Abbildung 4.2: Home-Tab Mock-Up [Eigene Darstellung]

4.1.3 MapView

Im MapView Abbildung 4.3 wird eine Karte angezeigt, auf der sämtliche öffentlich zugänglichen Vattenfall-Ladestationen im Berliner Raum als Marker mit Seriennummer präsentiert werden. Die grundlegenden Funktionen der Karte, wie Drehen, Zoomen und Verschieben, bleiben erhalten. Zusätzlich wird die Stadtgrenze von Berlin als farblich markierte Fläche dargestellt, um den Geltungsbereich der Ladenavigation hervorzuheben. Der aktuelle Standort kann in der App angezeigt und farblich hervorgehoben werden, wobei zuvor eine Berechtigungsabfrage erfolgt. Eine Suchfunktion ermöglicht die Eingabe einer Adresse.



Abbildung 4.3: Map-Tab Mock-Up [Eigene Darstellung; Karte nach inside digital (2025)]

Die Marker auf der Karte sind farblich unterschiedlich gestaltet, abhängig davon, ob es sich um eine AC- oder DC-Ladestation handelt. Beim Klick auf einen Marker öffnet sich ein Informationskärtchen, in dem die wichtigsten Daten der jeweiligen Ladestation (Ladeleistung, Verfügbarkeit, Straßenname und Koordinaten) sowie ein Bild der Station und ein Navigationsbutton angezeigt werden. Nach dem Auswählen erscheint auf der Karte eine Route vom aktuellen Standort zur gewählten Ladestation, wobei die geschätzte Zeit und Strecke oben rechts eingeblendet werden. Eine Funktion zum Abbruch der Routenberechnung ermöglicht die Rückkehr zur Kartenansicht.

Wird eine Adresse eingegeben, kann entschieden werden, ob direkt zu diesem Ziel oder optional über eine AC- oder DC-Ladestation navigiert werden soll. Die App zeigt analog dazu die Route mit allen relevanten Details und einer Abbruch-Option an.

4.1.4 StationView

Im StationView Abbildung 4.4 werden alle verfügbaren Stationen in einer Liste als Kärtchen präsentiert. Diese Liste ist über eine Suchleiste am oberen Rand nach Stationsnamen oder Straßennamen filterbar. Ein Klick auf ein Kärtchen führt zur Anzeige sämtlicher Informationen der ausgewählten Station.

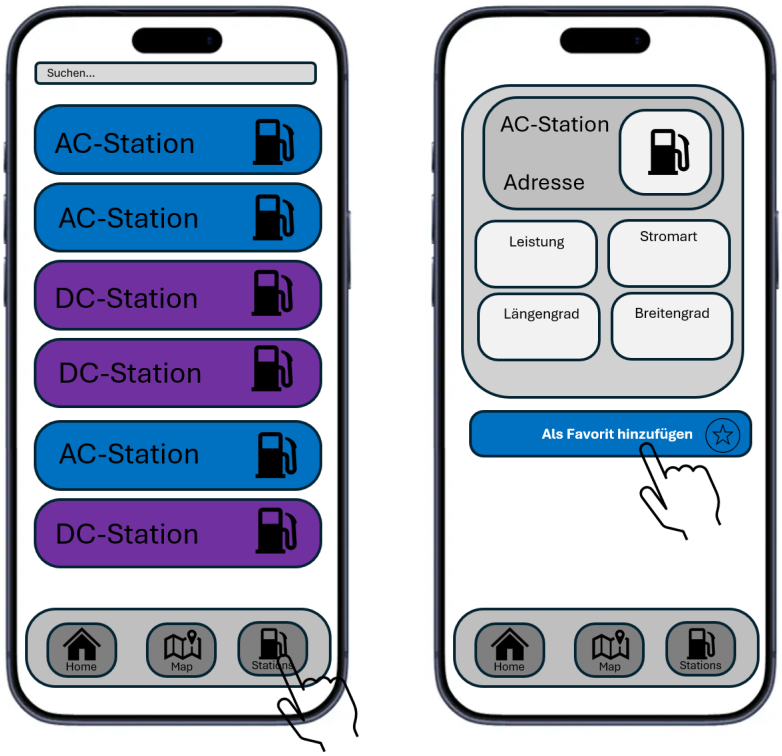


Abbildung 4.4: Stations-Tab Mock-Up [Eigene Darstellung]

4.2 Entwurf der Softwarearchitektur

4.2.1 Ladestationen einbinden

Im Rahmen der App-Entwicklung soll eine vollständige Darstellung sämtlicher öffentlich zugänglicher Ladestationen von Vattenfall im Berliner Stadtgebiet realisiert werden. Die Vattenfall Smarter Living GmbH stellt den Mitarbeitern hierfür ein Backend-System namens „MyIncharge“ zur Verfügung. Über dieses Portal ist es möglich, alle Ladestationen einzusehen, deren Konfigurationen anzupassen, Software-Updates zu installieren sowie Fehlerbehebungen durchzuführen. Zudem bietet das Portal eine Filterfunktion, mit der sich Ladestationen gezielt anzeigen lassen, beispielsweise beschränkt auf den Berliner Raum. Diese gefilterten Ergebnisse lassen sich zusätzlich als CSV-Datei exportieren. Eine CSV-Datei wurde gewählt, da keine öffentliche API für die Vattenfall-Ladestationen verfügbar ist.
Die exportierte CSV-Datei enthält umfassende Informationen zu den Berliner Ladestandorten, darunter Ladeleistung, Art des Stroms (AC/DC), Verfügbarkeitsstatus, Adresse sowie die relevanten geographischen Koordinaten in Form von Längen- und Breitengraden. Da mehrere Ladepunkte an einem Standort auftreten können, wurde die CSV-Datei zur besseren Handhabung modifiziert. Zunächst wurden alle Ladestationen entfernt, die nicht öffentlich zugänglich sind, etwa solche für Mitarbeiter oder solche, die sich in privaten Tiefgaragen von Anwohnern befinden.
Im Fokus der App steht die reine Navigation zu Ladepunkten. Daher wurde die Anzahl der Ladepunkte pro Standort reduziert. Befanden sich beispielsweise zehn Ladepunkte an einem Standort, wurde davon jeweils nur ein Ladepunkt in der App als Marker verwendet. Für Standorte mit sowohl AC- als auch DC-Optionen wurde jeweils ein Ladepunkt pro Stromart ausgewählt. Das Ergebnis ist eine Excel-Tabelle Abbildung 4.5 mit allen relevanten und öffentlich zugänglichen Ladepunkten in Berlin, die mindestens eine Stromart vertreten.

Da es sich bei den Daten um eine Momentaufnahme handelt, waren bei vielen Stationen der Status „Charging“ statt „Available“ aufgeführt; diese werden in der App dennoch als verfügbar angezeigt. Stationen mit dem Status „Unavailable“ wurden gelöscht, da hier entweder langfristige Fehler vorlagen oder die Stationen bereits außer Betrieb bzw. entfernt sind. Diese Stationen werden in der vorliegenden Arbeit nicht weiter berücksichtigt. Insgesamt ergab sich eine Anzahl von über 80 gültigen Standorten in Berlin. Im weiteren Verlauf werden für die Ladestationen folgende Daten berücksichtigt: Der Name der Station, welcher sich üblicherweise aus einer Abkürzung des Herstellers gefolgt von der Seriennummer zusammensetzt. Beispielsweise steht bei KBA-20190964 die Abkürzung KBA für den Hersteller KEBA, gefolgt von der Seriennummer. Für die ABB Tera 365 lautet der Stationsname beispielsweise t365_it1_0723_134. Des Weiteren werden die Stromart AC oder DC, die angegebene Leistung in Kilowatt, der Verfügbarkeitsstatus, die öffentliche Zugänglichkeit, die geografischen Koordinaten in Form von Längen- und Breitengrad sowie die Adresse der Station erfasst und in der Anwendung berücksichtigt.

	A	B	C	D	E	F	G	H	I	J	K
1		point name	power type	advertised power	status	public access	latitude	longitude	street	house number	postal code
2		KBA-20190964_2	AC_3_PHASE	22kW	AVAILABLE	true	525.551.196	131.980.745	Fehrbelliner Strasse	29	13585
3		KBA-25194126_1	AC_3_PHASE	11kW	AVAILABLE	true	525.347.511	132.477.366	OtternbuchtstraÃŸe	14	13599
4		t365_it1_0723_134_1	DC	360kW	AVAILABLE	true	524.533.625	133.781.224	WolframstraÃŸe	88	12105
5		KBA-29618613_1	AC_3_PHASE	11kW	CHARGING	true	525.164.537	133.933.261	Bebelplatz	2	10117
6		KBA-28922602_1	AC_3_PHASE	11kW	AVAILABLE	true	525.228.384	134.148.345	AlexanderstraÃŸe	2	10178
7		t184_it1_1623_053_1	DC	150kW	AVAILABLE	true	525.725.742	135.645.675	Am Gehrensee	2	13057
8		EVB-P1839092_1	AC_3_PHASE	11kW	AVAILABLE	true	524.796.600	133.078.200	ForckenbeckstraÃŸe	1	14199
9		t365_it1_0223_191_1	DC	360kW	AVAILABLE	true	524.341.746	135.561.645	Glienicker Weg	122	12489
10		t365_it1_0823_102_1	DC	360kW	AVAILABLE	true	525.520.387	134.552.018	PistoriusstraÃŸe	13	13086
11		KBA-21762747_1	AC_3_PHASE	11kW	AVAILABLE	true	525.103.951	133.877.910	Leipziger Str.	106	10117
12		t365_it1_0823_040_1	DC	360kW	AVAILABLE	true	525.399.446	132.957.176	Heckerdamm	240	13627
13		t184_it1_2223_004_1	DC	180kW	AVAILABLE	true	525.204.020	134.866.690	BornitzstraÃŸe	46	10365
14		t365_it1_0923_042_1	DC	360kW	AVAILABLE	true	525.828.466	134.129.919	Blankenburger Str.	61	13156
15		t365_it1_0823_073_1	DC	360kW	AVAILABLE	true	525.024.336	135.621.172	KÃ¶penicker StraÃŸe	45	12683
16		KBA-26336859_1	AC_3_PHASE	22kW	AVAILABLE	true	524.278.400	135.032.372	KanalstraÃŸe	74	12357
17		KBA-28922665_1	AC_3_PHASE	22kW	AVAILABLE	true	525.000.279	133.847.133	Hallesches Ufer	40	10963
18		KBA-27249543_1	AC_3_PHASE	11kW	AVAILABLE	true	525.640.642	132.137.642	Pepitapromenade	8	13587
19		EVB-P1844024_2	AC_3_PHASE	11kW	AVAILABLE	true	525.363.240	132.441.995	GroÃŸer Spreering	5	13599
20		KBA-25618975_1	AC_3_PHASE	11kW	AVAILABLE	true	525.434.898	135.940.647	Zossener Str.	146	12627

Abbildung 4.5: Abzug aus dem MyIncharge-Portal von Vattenfall [Eigene Darstellung]

4.2.2 Ausgewählte Elektroautos

Die Auswahl der Elektrofahrzeuge erfolgte nach eigenem Ermessen mit dem Ziel, jeweils ein Modell der bekannten deutschen Automobilhersteller aus dem Mittel- bis Oberklasse-Segment auszuwählen. Hierfür wurden die offiziellen Websites der Hersteller besucht, wobei unter der Kategorie Technische Daten sämtliche relevanten Informationen eingesehen wurden. Zu den erfassten Daten zählen der Fahrzeugname, das Modell sowie die Marke. Für den Verbrauch wurde der kombinierte Wert herangezogen, da dieser bei einigen Herstellern die einzige verfügbare Angabe ist und die maximale Reichweite ebenfalls auf Grundlage des kombinierten Verbrauchs im WLTP-Testverfahren berechnet wird. Ergänzend wurden die Netto-Batteriekapazität, die angegebene maximale Reichweite, die Batterietechnologie, die maximale Motorleistung, die Höchstgeschwindigkeit sowie die Beschleunigung von 0 auf 100 Kilometer pro Stunde berücksichtigt Volkswagen (2025) BMW (2025) Mercedes (2025) Porsche (2025) Audi (2025).

4.2.3 Umlaufgrenze Berlins

Im Rahmen der App soll klar visualisiert werden, dass der Nutzer sich ausschließlich im Berliner Raum bewegt und der Fokus auf diesem geografischen Gebiet liegt. Zur Verdeutlichung der räumlichen Eingrenzung ist geplant, neben den von Apple Maps bereitgestellten Stadtgrenzen die Umrisse der Berliner Ortsteile zusätzlich sichtbar zu machen. Während der Recherche wurde eine geeignete GeoJSON-Datei Abbildung 4.6 mit den Umgrenzungen der Ortsteile gefunden, die zur Flächenmarkierung genutzt werden kann. Diese Datei ermöglicht eine deutlichere Hervorhebung der Fläche Berlins, in der die Ladestationen später als Marker dargestellt werden. Die GeoJSON-Datei wird im weiteren Verlauf in das Xcode-Projekt eingebunden und auf der Karte visualisiert, um eine intuitive und präzise Abgrenzung der relevanten Fläche sicherzustellen.

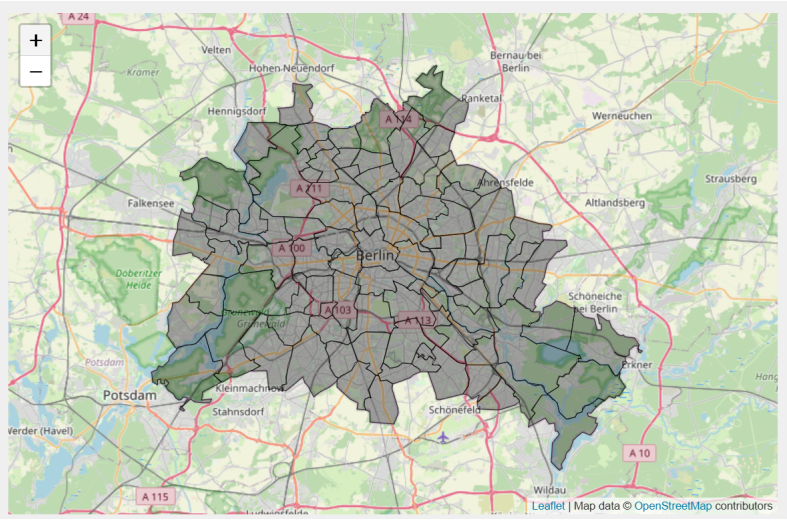


Abbildung 4.6: Ortsteile von Berlin odis berlin (2025)

4.2.4 Apple Symbols Bibliothek

Apple stellt eine umfassende Symbolbibliothek bereit, die eine Vielzahl von Icons für verschiedene Anwendungsfälle enthält, darunter Symbole für Ladestationen, Markierungen, Fahrzeugeigenschaften und vieles mehr im charakteristischen Apple-Stil. Um diese Symbole in der App verwenden zu können, muss die entsprechende Bibliothek zunächst heruntergeladen werden. In Xcode lassen sich die gewünschten Symbole anschließend bequem durch die Verwendung des „symbol“-Ausdrucks einbinden, indem der Name des gewählten Icons angegeben wird. Dadurch wird das Erscheinungsbild der App stärker an den Apple-Designrichtlinien ausgerichtet und die Benutzeroberfläche wirkt konsistent und professionell. Mittlerweile gibt es die SF Symbols Version 7, welche es ermöglicht, auch Draw-Animationen für Symbole darzustellen, sodass diese wie gezeichnet erscheinen. Darüber hinaus lassen sich die Symbole an Farbthemen anpassen, indem sie in einer festen Farbe oder sogar mit Farbverläufen, einem Gradienten, angezeigt werden, was die visuelle Gestaltungsmöglichkeiten erheblich erweitert und dynamischere Benutzeroberflächen ermöglicht.

4.2.5 Funktionsablauf der App

Der Ablauf der App gestaltet sich wie folgt Abbildung 4.7: Beim Start wird der Nutzer im Map-Tab beim Öffnen nach der Berechtigung für den Standort gefragt. Diese Erlaubnis ist notwendig, um eine Route vom aktuellen Standort beispielsweise zu einer AC-Ladestation berechnen zu können. Im Home-Tab kann der Nutzer eines der fünf in der Liste angezeigten Elektrofahrzeuge auswählen und weitere Details dazu einsehen. Über einen Button „Fahrzeug auswählen“ wird seine Auswahl gespeichert.
Im Map-Tab wird der aktuelle Standort des Nutzers als blauer Punkt auf der Karte angezeigt, wie es von anderen Kartenanbietern gewohnt ist. Zugleich sind alle AC- und DC-Ladestationen als Marker oder Pins auf der Karte sichtbar. Am oberen Bildschirmrand befindet sich eine Suchleiste. Wenn der Nutzer eine Ladestation antippt, öffnet sich eine Card-Ansicht, die die Hälfte des Bildschirms einnimmt und weitere Details der Ladestation bereitstellt. Ein Navigationsbutton ermöglicht, eine Anfrage zur Berechnung einer Route vom aktuellen Standort zum Zielpunkt zu senden. Die Route wird als blaue Polylinie auf der Karte dargestellt. Die prognostizierte Zeitdauer und die Gesamtstrecke werden oben rechts in der Ecke angezeigt. Ein X-Knopf ermöglicht das Löschen der Route von der Karte. Nach Eingabe einer Adresse wird dieser Punkt als Pin markiert.

Es stehen mehrere Navigationsoptionen zur Verfügung: Der Nutzer kann direkt zum Punkt navigieren oder einen Ladestopp über eine AC- oder DC-Ladestation einplanen. Die App berechnet die Ladestation, die am besten auf dem Weg liegt und möglichst keinen Umweg verursacht. Sollte keine Ladestation auf dem Weg oder in Reichweite sein, wird dies auf dem Bildschirm angezeigt und keine Route berechnet.
Im Stations-Tab kann der Nutzer alle Ladestationen in einer Liste einsehen und durch Anklicken weitere Details abrufen. Eine Suchfunktion ermöglicht das Filtern der Stationen nach Namen oder Adresse. Zudem besteht die Möglichkeit, Stationen in der MapView als Favoriten zu markieren, wodurch deren Marker gelb eingefärbt werden. Diese Favoriten werden in einem vierten Tab, dem Favoriten-Tab, gesammelt angezeigt. Die vorliegende Planungsphase wird im folgenden Kapitel technisch umgesetzt und detailliert beschrieben.

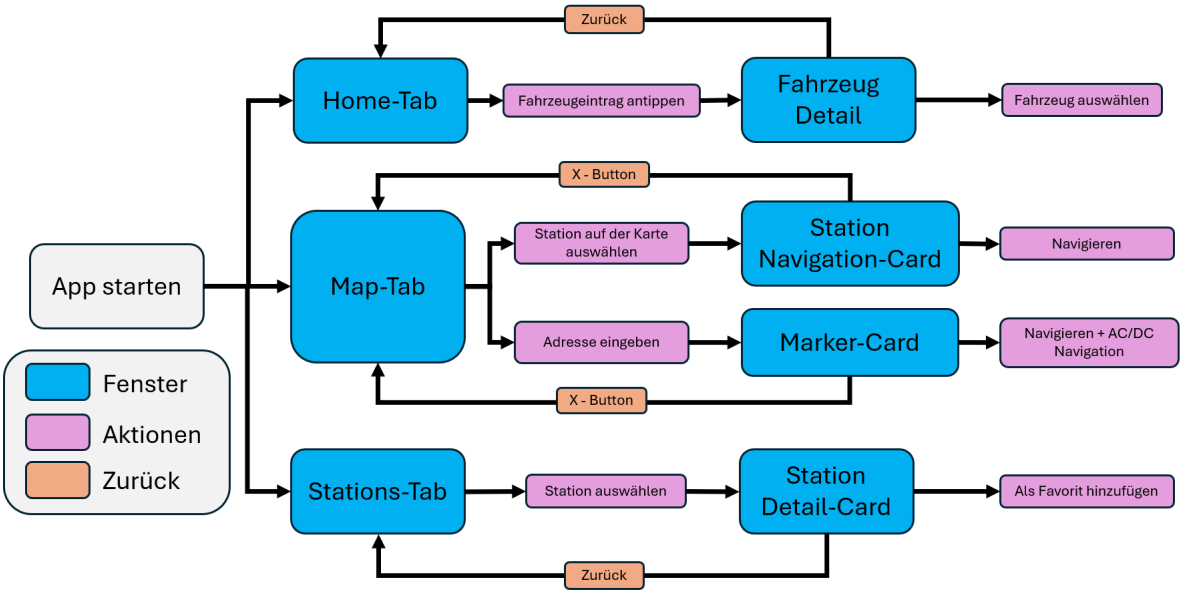


Abbildung 4.7: Sitemap-Navigationsdiagramm [Eigene Darstellung]

4.2.6 Systemarchitektur

Im Folgenden wird die Systemarchitektur der App beschrieben Abbildung 4.8. Die Systemarchitektur der App folgt einem klar strukturierten Aufbau. Ganz oben stehen die UI-Views wie InchargeView, MapView, StationView und HomeView, die nur für Darstellung und Benutzerinteraktion zuständig sind. Das ViewModel holt Standortinformationen über die Standortdienste und bereitet die Daten für die Views auf. Die tatsächlichen Daten werden von der DataAccess-Schicht über spezialisierte Loader aus Dateien oder APIs eingelesen.
Ganz unten befindet sich die Domain-Schicht mit den reinen Modelltypen sowie die genutzten Apple-Frameworks wie MapKit, SwiftUI, Foundation, Combine und CoreLocation, die von den oberen Schichten verwendet werden, ohne dass diese ihre Implementierungsdetails kennen. SwiftUI eignet sich besonders gut für das MVVM-Pattern, das eine klare Trennung zwischen Datenlogik (Model/ViewModel) und UI (View) ermöglicht. Hudson (2025a)

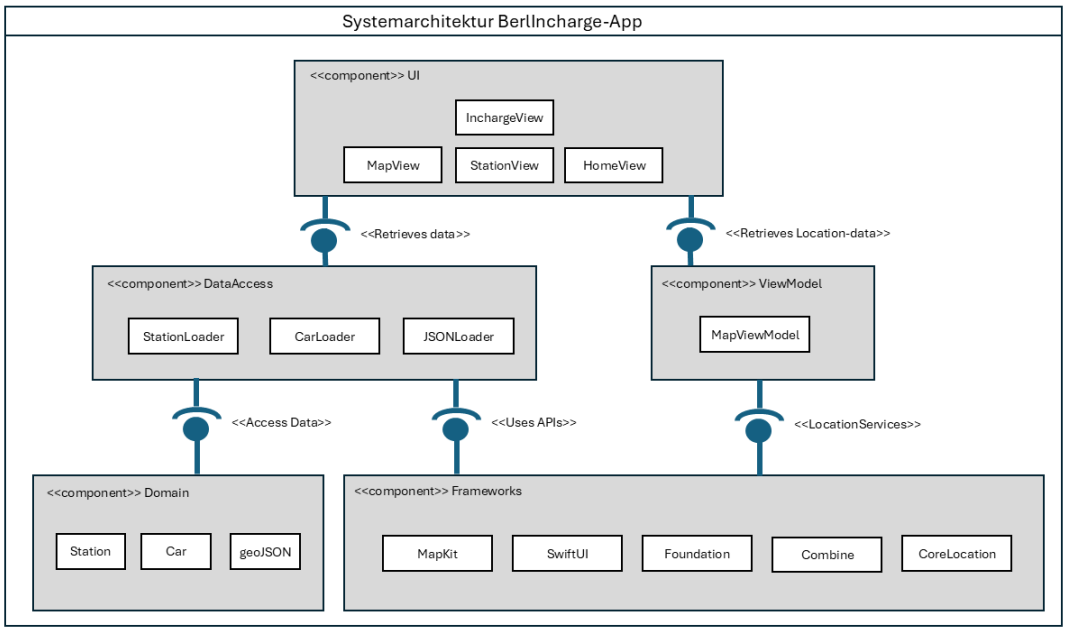


Abbildung 4.8: UML-Komponentendiagramm [Eigene Darstellung]

Kapitel 5

Implementierung

Dieses Kapitel beschreibt die schrittweise Umsetzung der Berlincharge-App für die iOS-Plattform. Die App wird in der Programmiersprache Swift unter Verwendung von SwiftUI entwickelt, wobei Apple Xcode als Entwicklungsumgebung zum Einsatz kommt. Für die Umsetzung wurde ein MacBook verwendet, wie es bereits im Kurs „App-Entwicklung“ im Bachelorstudium praktiziert wurde.

Zur Unterstützung bei der Code-Generierung und -Optimierung in Xcode wurde Apple Intelligence eingesetzt. Dies umfasste die Erstellung von Code-Vorschlägen für SwiftUI-Komponenten und Tests, die anschließend manuell überprüft, angepasst und validiert wurden. Die finale Implementierung stellt eine eigenständige Leistung dar, wobei die KI ausschließlich unterstützend diente.

5.1 Projektaufbau

Das Projekt wurde in mehrere `View`-Dateien unterteilt, um die Struktur und Übersichtlichkeit zu verbessern. Neben den Assets existieren verschiedene Ordner, in denen die unterschiedlichen Daten organisiert und in die App eingebunden werden.

Aufbau der Views

Das Projekt umfasst insgesamt vier zentrale `View`-Dateien:

- **HomeView**
- **MapView**
- **StationView**
- **InchargeView**

Die `InchargeView` fungiert als zentrale Komponente der App, da sie die drei anderen Views mittels eines `TabView`-Containers zusammenführt. Die konkrete Einbindung dieser Views wird im weiteren Verlauf mithilfe passender Code-Snippets detailliert erläutert.

Assets

In den Assets sind alle Bilddateien und Farbinformationen gespeichert, darunter:

- Ausgeschnittene Fotos der verwendeten Automodelle
- Bilder der verschiedenen Ladestationen
- Definition der Primär- und Sekundärfarben für den Light- und Dark-Modus

Für dieses Projekt wurde eine Farbkombination aus Blau und Lila gewählt, da der Kontrast in beiden Modi als optimal wahrgenommen wurde.

Datenstrukturen

Darüber hinaus existieren Unterordner, die folgende Datensätze enthalten:

- `JSON`-Dateien der Ladestationen
- `JSON`-Dateien der Automodelle
- eine `GeoJSON`-Datei mit den Berliner Stadtgrenzen

5.2 Einbindung der Daten

Die App benötigt eine Vielzahl von Daten, die visualisiert werden müssen. Diese Daten liegen ursprünglich entweder in Form von Excel-Dateien vor oder sind als Rohdaten von Websites verfügbar. Zusätzlich wird eine GeoJSON-Datei eingelesen, um geografische Informationen darzustellen und auf der Karte anzuzeigen.
Für die Elektrofahrzeuge wurden die notwendigen Daten von den jeweiligen Herstellerseiten bezogen. Ziel ist es, die typischen Kennwerte in mehreren statistischen Boxen übersichtlich darzustellen. Da mehrere Automodelle berücksichtigt werden sollen, wurde eine strukturierte Datenhaltung für die verschiedenen Fahrzeugtypen entworfen. Hierfür wurde eine JSON-Datei mit dem Namen *cars.json* erstellt, die die wichtigsten Informationen zu den einzelnen Modellen speichert.
Diese Datei dient als Datenquelle zur weiteren Verarbeitung und wird schließlich innerhalb der App visualisiert. Zu Beginn erfolgte die Festlegung einer passenden Datenstruktur, die anschließend in der JSON-Datei umgesetzt wurde.
Als nächster Schritt erfolgt die Umwandlung der JSON-Daten in nutzbare Datenobjekte Listing 5.1. Hierfür stellt Xcode eine Funktion namens `JSONDecoder` Developer (2025a) bereit, die für die Dekodierung von JSON-Dateien verwendet wird. Mittels dieser Funktion wird eine `load()`-Funktion implementiert, die im HomeView aufgerufen wird. Diese Funktion lädt die Daten aus der JSON-Datei in ein Array. Dabei erfolgt im Code eine Suche im App Bundle nach der Datei *cars.json*. Für den Fall eines Fehlers ist ein entsprechendes Error-Handling integriert, das die Fehlerausgabe übernimmt. Anschließend werden die Daten aus dem App Bundle geladen, durch den `JSONDecoder` dekodiert und im gewünschten Format zurückgegeben Thinking (2021).

```
import Foundation

enum CarLoader {
    static func load() -> [Car] {
        guard let url = Bundle.main.url(forResource: "cars", withExtension: "json") else {
            print("cars.json␣not␣found")
            return []
        }
        do {
            let data = try Data(contentsOf: url)
            let cars = try JSONDecoder().decode([Car].self, from: data)
            return cars
        } catch {
            print("Error␣decoding.␣\(error)")
            return []
        }
    }
}
```

Listing 5.1: Dekodierung der cars.json Datei mithilfe der CarLoader-Funktion

Beim Aufruf der `load()`-Funktion im Code wird die JSON-Datei transformiert, was bedeutet, dass die darin enthaltenen Daten in Objekte des Typs *Car* umgewandelt und in einem Array gespeichert werden. Dieses Array steht der App dann zur Verfügung, um alle Automodelle samt ihrer Eigenschaften darzustellen Mikina (2021) Hudson (2025b).
Für die Ladestationen wurde das Prinzip der JSON-Datenhaltung in *stations.json* ebenfalls übernommen, jedoch mit angepassten Attributen entsprechend der spezifischen Anforderungen. Für die GeoJSON-Datei ist ein GeoLoader implementiert, der die Datei über die MKGeoJSONDecoder()-Funktion korrekt einliest Developer (2025b) Novichkov (2025).
Zusammenfassend zeigt dieses Vorgehen, wie eine externe JSON-Datei mit Fahrzeuginformationen in strukturierte Swift-Datentypen überführt wird und anschließend innerhalb einer App nutzbar gemacht wird.

5.3 Implementierung der Benutzeroberfläche

Nachdem die Ladestationen und Automodelle mithilfe der jeweiligen Loader in nutzbare Daten transformiert wurden und die GeoJSON-Datei zur Darstellung der Stadtgrenzen auf der Karte integriert ist, erfolgt nun die Gestaltung der Benutzeroberfläche. Wie bereits erwähnt, fungiert die InchargeView als Skelett, welches die drei weiteren Views strukturell vereint. Die einzelnen Views übernehmen dabei jeweils ihre speziellen Aufgabenbereiche und sind für die darzustellenden Inhalte und Interaktionen zuständig.

5.3.1 InchargeView

Die InchargeView vereint die drei Haupt-Views innerhalb eines TabViews, welcher eine Leiste am unteren Bildschirmrand einfügt Listing 5.2. Über diese Leiste kann der Nutzer zwischen den verschiedenen Views wechseln. In Xcode erfolgt dies durch Verwendung der `TabView`-Funktion, der jeweils ein View zugeordnet wird. Mithilfe des Modifiers *.tabItem* lassen sich auf der Leiste ein Name und ein Symbol platzieren. Für die Symbole werden SF Symbols 7 verwendet, die über die `Image()`-Funktion und den entsprechenden Namen eingebunden werden. Die Texte für die Tabs entsprechen den jeweiligen Inhalten und lauten Home, Map und Station Allen (2021).

```
import SwiftUI

struct InchargeView: View {
    @State private var selectedCar: Car?

    var body: some View {
        TabView {
            HomeView(selectedCar: $selectedCar)
                .tabItem {
                    Image(systemName: "house")
                    Text("Home")
                }
            MapView(selectedCar: $selectedCar)
                .tabItem {
                    Image(systemName: "map")
                    Text("Map")
                }
            StationView()
                .tabItem {
                    Image(systemName: "ev.charger")
                    Text("Stations")
                }
        }
        .accentColor(Color("brandPrimary"))
    }
```

```
}
```

Listing 5.2: Navigation der einzelnen Views über InchargeView

Die *@State*-Variable *selectedCar* dient dazu, einen lokalen, veränderlichen Zustand innerhalb der View zu speichern und zu überwachen. Wenn sich dieser Wert ändert, aktualisiert SwiftUI automatisch die View, die davon abhängig ist. In diesem Fall wird die *selectedCar*-Variable als Binding an die beiden Views HomeView und MapView übergeben.

5.3.2 HomeView

Die HomeView ist dafür vorgesehen, die einzelnen Elektroautos darzustellen. Beim Anklicken eines Fahrzeugs öffnet sich eine Detailansicht mit weiteren Informationen zum jeweiligen Modell. Über einen Button kann der Nutzer außerdem das gewünschte Automodell auswählen, welches dann für die Berechnung der Strecke und der verbleibenden Restreichweite im weiteren Verlauf der App verwendet wird.
Im HomeView ist die Liste der Autos als Zustandsvariable definiert, sodass bei Änderungen automatisch die Benutzeroberfläche aktualisiert wird. In der Liste werden alle Autos mithilfe von NavigationLink-Elementen dargestellt, die anklickbar sind und beim Tippen zur entsprechenden Detailansicht führen. Jedes Listenelement zeigt ein Bild des Autos über CarImagePlaceholder, den Fahrzeugnamen, das Modell sowie Statusinformationen wie Ladezustand und Restreichweite. Die gesamte Navigation erfolgt innerhalb eines NavigationStacks Allen (2020).

```
private struct StatusBox: View {
    let selectedCar: Car?
    let onClear: () -> Void

    var body: some View {
        let hasSelection = (selectedCar != nil)
        let iconName = hasSelection ? "car.fill" : "car"
                ...
        HStack(alignment: .center, spacing: 12) {
            Image(systemName: iconName)
                    ...

            VStack(alignment: .leading, spacing: 2) {
                if let car = selectedCar {
                    Text("\(car.name) ausgewählt")
                    ...
                    Text("\(car.brand) x \(car.model)")
                    ...
                } else {
                    Text("Kein Fahrzeug ausgewählt")
                    ...
                    Text("Wähle ein Fahrzeug aus der Liste.")
                    ...
                }
            }
                    ...

            if hasSelection {
                Button {
                    onClear()
                } label: {
                    Image(systemName: "xmark.circle.fill")
                    ...
                }
                .buttonStyle(.plain)
            }
        }
```

```
                    ...
    }
}
```

Listing 5.3: StatusBox-View zur Anzeige des ausgewählten Autos

Der StatusBox Listing 5.3 ist eine View, die den Status der Fahrzeugauswahl anzeigt - entweder ein ausgewähltes Car oder einen Hinweis auf keine Auswahl. Sie erhält zwei Parameter: *selectedCar* und *onClear*. *selectedCar* ist eine optionale Car-Instanz zum Lesen und *onClear* ist ein Closure, welcher beim Löschen aufgerufen wird.

```
                Section {
                    StatusBox(
                        selectedCar: selectedCar,
                        onClear: { selectedCar = nil }
                    )
```

Listing 5.4: Anzeige der Statusbox in der HomeView

Die StatusBox wird in der `HomeView` innerhalb einer `Section` der `List` angezeigt, die in einem `NavigationStack` liegt Listing 5.4. Über den Parameter `onClear: () -> Void` erhält die StatusBox ein Callback-Closure, das beim Tippen auf den Clear-Button die gebundene Variable `selectedCar` auf `nil` setzt AzamSharp (2025) swiftbysundell (2025).

5.3.3 MapView

Die Karte der MapView zeigt die aktuelle Position des Nutzers, die hervorgehobene Berliner Stadtgrenzen, Marker für Ladestationen und lädt Suchergebnisse mit entsprechenden Pins. Im Fokus steht die Routenplanung und deren Darstellung, die sowohl einfache Direktverbindungen als auch optimierte Routen mit Zwischenladestationen unterstützt.
Die Routenfunktionen lassen sich gedanklich in zwei Gruppen unterteilen: einfache, direkte Verbindungen zu einer Ladestation oder zu einem *MapItem* sowie komplexere Routenplanung über eine AC- oder DC-Ladestation.
Die Funktion `requestRoute()` ist die Kernfunktion Listing 5.5. Sie fordert eine Route von einem *MapItem* zu einem anderen *MapItem* an. Dabei wird der Request mit Start- und Zielstandort, dem Transportmittel und gegebenenfalls mit der Option für eine zusätzliche alternative Route befüllt. Auf diese Anfrage erhält die Funktion eine Antwort, die in der Variablen *response* gespeichert wird. Aus dieser Antwort wird über die Eigenschaft *.first* die primäre, in der Regel kürzeste Route übernommen Allen (2025) Recke (2025).

```
private func requestRoute(from: CLLocationCoordinate2D, to: CLLocationCoordinate2D) async ->
  MKRoute? {
        let req = MKDirections.Request()
        req.source = MKMapItem(placemark: MKPlacemark(coordinate: from))
        req.destination = MKMapItem(placemark: MKPlacemark(coordinate: to))
        req.transportType = .automobile
        req.requestsAlternateRoutes = false

        do {
            let response = try await MKDirections(request: req).calculate()
            return response.routes.first
        } ...
    }
```

Listing 5.5: Anfrage Route über requestRoute-Funktion

Darauf aufbauend gibt es eine weitere Funktion, die für die einfachen Navigationsaufrufe genutzt wird. Die Funktion `requestRouteToCoord()` Listing 5.6 verwendet intern die Funktion `requestRoute()` Listing 5.5 und ergänzt diese um die Ermittlung der aktuellen Nutzerposition, die als Startkoordinate verwendet wird Ackermann (2025). Die berechnete Route wird in *activeRoute* gespeichert und ihre Geometrie als Polylinie in *activeRoutePolyline* abgelegt.

```
    private func requestRouteToCoord(to coordinate: CLLocationCoordinate2D, displayName:
      String?) async {
        guard let userLocation = viewModel.lastKnownLocation?.coordinate else {
                    ...
        }
                    ...

        guard let route = await requestRoute(from: userLocation, to: coordinate) else {
                    ...
        }
                    ...
                activeRoutePolyline = route.polyline
                    ...
    }
```

Listing 5.6: Anfrage Route vom Standort über requestRouteToCoord-Funktion

Auf Basis von `requestRouteToCoord()` Listing 5.6 ergeben sich zwei weitere Funktionen, mit denen jeweils vom aktuellen Standort zu einem Ziel navigiert werden kann. Entweder zu einem *MapItem* aus der Suchfunktion oder zu einer ausgewählten Station. Der Funktion `requestRouteToStation()` Listing 5.7 wird eine Station übergeben. Daraus werden aus Breiten- und Längengrad der Station

die Zielkoordinaten gebildet, die anschließend an `requestRouteToCoord()` Listing 5.6 weitergereicht werden C (2025).

```
private func requestRouteToStation(to station: Station) async {
                ...
    let coord = CLLocationCoordinate2D(latitude: station.latitude, longitude: station.
      longitude)
    await requestRouteToCoord(to: coord, displayName: station.pointName)
}
```

Listing 5.7: Route zur Ladestation über requestRouteToStation-Funktion

Die Funktion `requestRouteToItem()` Listing 5.8 ist bis auf den Parameter fast identisch aufgebaut: Statt einer Station erhält sie ein *MapItem*, das aus der Nutzersuche stammt. Aus dessen Koordinate wird das Ziel abgeleitet und anschließend über `requestRouteToCoord()` vom aktuellen Standort dorthin navigiert.

```
private func requestRouteToItem(to item: MKMapItem) async {
                ...
    await requestRouteToCoord(to: item.placemark.coordinate, displayName: item.name)
}
```

Listing 5.8: Route zum MapItem über requestRouteToItem

Damit sind die Funktionen für einfache Direktverbindungen definiert und es kann mit der komplexeren Routenberechnung fortgefahren werden. Im Mittelpunkt steht hierbei die Funktion `requestAssembledRoute()`, die eine Strecke vom aktuellen Nutzerstandort zu einem ausgewählten *MapItem* auf der Karte berechnet und dabei gezielt einen Umweg über eine AC- oder DC-Ladestation einplant. Die Auswahl der Route mit der geringsten Gesamtdistanz und dem kleinsten zusätzlichen Umweg über eine passende Ladestation erfolgt in dieser Funktion mithilfe mehrerer untergeordneter Hilfsfunktionen.
Zuerst wird festgelegt, dass nur Stationen betrachtet werden, die höchstens 2 Kilometer Luftlinie von der ursprünglich berechneten Hauptroute entfernt liegen. Außerdem wird definiert, ob gezielt nach AC- oder nach DC-Ladestationen gesucht werden soll. Diese Filterkriterien werden in der Funktion `filterStationsInRadius()` Listing 5.9 geprüft und angewendet.

```
private func filterStationsInRadius(stations: [Station], requireDC: Bool, polyline:
  MKPolyline, radius: CLLocationDistance) -> [Station] {
    stations.filter { s in
            let matchesType = Calc.isDC(powerType: s.powerType) == requireDC
            guard matchesType else { return false }

            let p = CLLocationCoordinate2D(latitude: s.latitude, longitude: s.longitude)
            return distanceFromPointToPolyline(point: p, to: polyline) <= radius
        }
}
```

Listing 5.9: Filter der Stationen nach Kriterien via filterStationInRadius-Funktion

Für die Berechnung des Abstands zu einem einzelnen Routensegment wird die Hilfsfunktion `distanceFromPointToSegment()` Listing 5.10 verwendet. In dieser Funktion wird der Abstand von einem Punkt, hier der Station, zu einem Streckenabschnitt zwischen den Punkten A und B mithilfe von Vektorrechnung bestimmt. Zunächst wird berechnet, an welcher Stelle entlang des Segments der senkrechte Fußpunkt der Station liegt. Dieser Punkt wird als projectedMapPoint bezeichnet. Der dazugehörige Hilfsparameter *projectedParameter* nimmt Werte zwischen 0 und 1 an und gibt an, wie weit der Fußpunkt zwischen A und B liegt, wobei 0 dem Startpunkt A und 1 dem Endpunkt B entspricht.

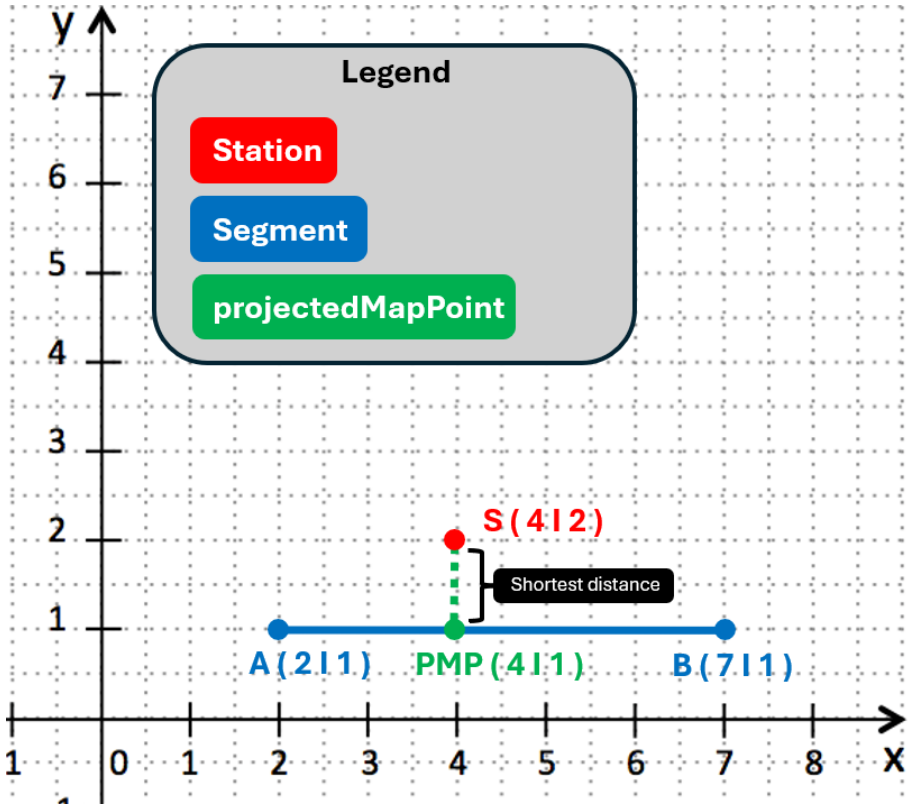


Abbildung 5.1: Kürzeste Distanz Station zum Segment [Eigene Darstellung; Koordinatensystem nach Mathestunde.com (2025)]

Liegt die Station senkrecht „über“ der Mitte des Segments, hat *projectedParameter* den Wert 0,5, unabhängig von der tatsächlichen senkrechten Entfernung. Befindet sich die Station außerhalb der durch A und B aufgespannten Strecke, wird *projectedParameter* auf den Bereich zwischen 0 und 1 begrenzt, sodass effektiv entweder der Start- oder der Endpunkt als nächstgelegener Punkt verwendet wird. Aus *projectedMapPoint* und der Stationsposition ergibt sich schließlich der gesuchte minimale Abstand zwischen Station und Segment Abbildung 5.1 geeksforgeeks (2025) Wikipedia (2025a).

```
private func distanceFromPointToSegment(_ p: CLLocationCoordinate2D, from a:
  CLLocationCoordinate2D, to b: CLLocationCoordinate2D) -> CLLocationDistance {
        let stationMapPoint = MKMapPoint(p)
        let startMapPoint = MKMapPoint(a)
        let endMapPoint = MKMapPoint(b)

        let startToStationVectorX = stationMapPoint.x - startMapPoint.x
        let startToStationVectorY = stationMapPoint.y - startMapPoint.y
        let startToEndVectorX = endMapPoint.x - startMapPoint.x
        let startToEndVectorY = endMapPoint.y - startMapPoint.y

        let startToEndVectorSquare = startToEndVectorX * startToEndVectorX + startToEndVectorY
           * startToEndVectorY
        if startToEndVectorSquare == 0 { return stationMapPoint.distance(to: startMapPoint) }

        let projectedParameter = max(0, min(1, (startToStationVectorX * startToEndVectorX +
          startToStationVectorY * startToEndVectorY) / startToEndVectorSquare))
        let projectedMapPoint = MKMapPoint(x: startMapPoint.x + projectedParameter *
          startToEndVectorX, y: startMapPoint.y + projectedParameter * startToEndVectorY)
        return stationMapPoint.distance(to: projectedMapPoint)
    }
```

Listing 5.10: Berechnung des kürzesten Abstands von Punkt zu Polylinensegment

Zur Prüfung der maximalen Luftliniendistanz nutzt `filterStationsInRadius()` Listing 5.9 die Funktion `distanceFromPointToPolyline()` Listing 5.11. Diese Funktion zerlegt die Route in ihre einzelnen Liniensegmente und berechnet den Abstand von der Station zu jedem Segment Github (2025). Für jede Station wird der kleinste dieser Abstände als Distanz zur Route zurückgegeben und später in die Bewertung der Stationen einbezogen. In Abbildung 5.2 sieht man beispielsweise, dass hier für jedes Segment individuell der kürzeste Abstand zur Station berechnet wird. Das Segment CD liegt nicht parallel zum Punkt und ist auch etwas weiter entfernt, weshalb als Referenzpunkt der Punkt C des Segments CD gewählt wurde. Der *projectedParameter* ist hierbei demnach 0 und geht in die Berechnung für den *projectedMapPoint* mit ein. Daraus resultiert der eben genannte Punkt C. Von allen Entfernungen bleibt weiterhin die Entfernung von der Station S zum Segment AB die kürzeste und wird von der Funktion `distanceFromPointToPolyline()` (Listing 5.11) zurückgegeben.

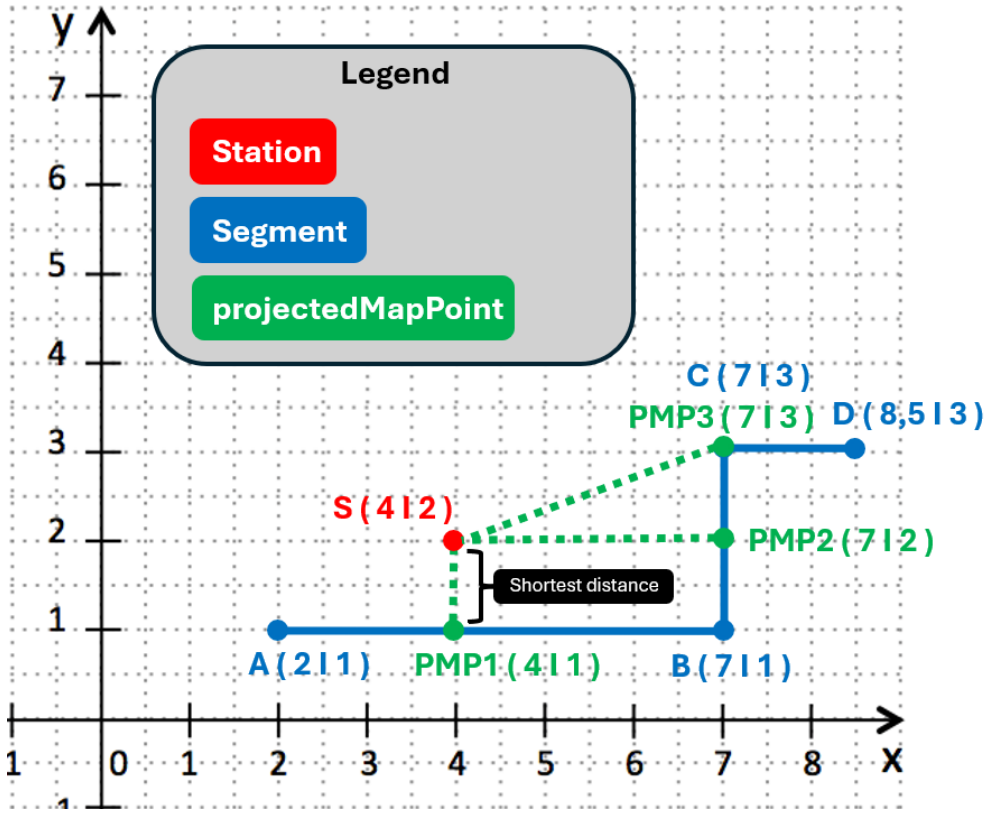


Abbildung 5.2: Kürzeste Distanz Station zur Polylinie [Eigene Darstellung; Koordinatensystem nach Mathestunde.com (2025)]

```
private func distanceFromPointToPolyline(point: CLLocationCoordinate2D, to polyline:
  MKPolyline) -> CLLocationDistance {
    let coords = polyline.coordinates
    guard coords.count > 1 else { return .greatestFiniteMagnitude }

    var minDist = CLLocationDistance.greatestFiniteMagnitude
    for i in 0..<(coords.count - 1) {
        let a = coords[i], b = coords[i + 1]
        minDist = min(minDist, distanceFromPointToSegment(point, from: a, to: b))
```

```
    }
    return minDist
}
```

Listing 5.11: Bestimmung der kürzesten Distanz zwischen einem Punkt und jedem Segment einer Polylinie

Durch die vorgelagerten Funktionen entsteht zunächst ein Container von Stationen, die maximal zwei Kilometer Luftlinie von einem Segment der direkten Route zwischen Start- und Zielpunkt entfernt liegen und gleichzeitig den Filterkriterien für AC- oder DC-Ladestationen entsprechen. Aus diesem Kandidaten-Set muss nun die Station ausgewählt werden, über die sich die insgesamt günstigste Fahrtroute ergibt, also nicht nur bezogen auf die Luftlinie.
Hier setzt die Funktion `evaluateStationsInRadius()` Listing 5.12 an. Sie erhält jeweils eine Station aus diesem gefilterten Container, berechnet zwei Teilrouten vom aktuellen Standort zur Station und von der Station zum Ziel und liefert diese zusammen mit einem Score als Tupel zurück. Der Score ist ein Distanzwert in Metern, der sich aus der gesamten Fahrstrecke über die Station sowie der minimalen Distanz der Station zur ursprünglichen Direktverbindung zwischen Standort und Ziel zusammensetzt, sodass sowohl der Umweg als auch die Einbindung in die Route berücksichtigt werden.

```
func evaluateStationsInRadius(_ s: Station) async -> (score: Double, userToStationRoute:
  MKRoute, StationToItemRoute: MKRoute)? {
            let stationCoord = CLLocationCoordinate2D(latitude: s.latitude, longitude: s.
              longitude)
            let distanceFromStationToRoute = distanceFromPointToPolyline(point: stationCoord,
              to: directRoute.polyline)

            guard
                let userToStationRoute = await requestRoute(from: userLocation, to:
                  stationCoord),
                let StationToItemRoute = await requestRoute(from: stationCoord, to: item.
                  placemark.coordinate)
            else { return nil }

            let totalDistance = userToStationRoute.distance + StationToItemRoute.distance
            let score = Double(totalDistance) + Double(distanceFromStationToRoute)
            return (score, userToStationRoute, StationToItemRoute)
        }
```

Listing 5.12: Auswertung der gefilterten Stationen

Damit wird die Station mit dem niedrigsten Score, also der geringsten Gesamtdistanz in Metern, als beste Option ausgewählt. Die Sortierung erfolgt in einer abschließenden Schleife Listing 5.13, die für jede gefilterte Station die Funktion `evaluateStationsInRadius()` Listing 5.12 aufruft und die Station mit dem kleinsten Score als Ergebnis übernimmt.

```
for s in stationContainer {
            if let observedStation = await evaluateStationsInRadius(s) {
                if let currentBestStation = bestStation {
                    if observedStation.score < currentBestStation.score {
                        bestStation = observedStation
                    }
                } else {
                    bestStation = observedStation
                }
            }
        }
        }
```

Listing 5.13: Bestimmung der Station, mit der kürzesten Gesamtstrecke

Sobald die optimalste Station gefunden wurde, wird der zugehörige Tupel an die Funktion `assembleRoute()` Listing 5.14 übergeben. Diese fügt die beiden Teilrouten vom Standort zur Station und von der Station zum Ziel zu einer gemeinsamen Polylinie zusammen, die anschließend als zusammengesetzte Route genutzt wird.

```
private func assembleRoute(routeA: MKRoute, routeB: MKRoute) -> MKPolyline {
       var coordsA = routeA.polyline.coordinates
       let coordsB = routeB.polyline.coordinates
       coordsA.append(contentsOf: coordsB)
       return MKPolyline(coordinates: coordsA, count: coordsA.count)
   }
```

Listing 5.14: Beide Teilstrecken werden zu einer gemeinsamen Strecke zusammengelegt

Die zusammengesetzte Polylinie wird dem Map-View übergeben und dort mithilfe von MapPolyline Listing 5.15 als Routenlinie auf der Karte dargestellt.

```
                    if let composite = compositePolyline {
                        MapPolyline(composite)
                    ...
```

Listing 5.15: Darstellung der berechneten Route

Damit steht eine vollständige Routinglogik zur Verfügung, die sowohl einfache Direktverbindungen als auch zusammengesetzte Routen über geeignete AC- oder DC-Ladestationen berechnet und auf der Karte darstellt. Die beschriebenen Funktionen greifen ineinander, um aus der Nutzerposition, den Kartenzielen und den verfügbaren Stationen automatisch die jeweils günstigste Route mit minimalem Umweg zu bestimmen.

5.3.4 StationView

Die StationView zeigt die Ladestationen in Form einer Liste an. Man kann diese Stationen durchsuchen und bei Auswahl zu einer Detailansicht der jeweiligen Station wechseln.
Die Variable *filteredStations* erzeugt eine gefilterte Liste der Stationen basierend auf dem Suchtext. Zuerst wird der Suchtext bereinigt, indem Leerzeichen am Anfang und Ende entfernt werden. Wenn die Suche leer ist, wird die komplette Liste zurückgegeben. Andernfalls filtert der Code die Stationen so, dass nur solche übrig bleiben, bei denen eines der Eigenschaftenfelder wie pointName, street, houseNumber etc. den Suchbegriff enthält, unabhängig von Groß- und Kleinschreibung.
Damit die Filterfunktion korrekt arbeitet, werden alle relevanten Eigenschaften der Station als einzelne Einträge im Array *container* gespeichert. Für die Suche wird dann geprüft, ob einer dieser Einträge den Suchtext *item* enthält. Ist das der Fall, wird die Station im Ergebnis zurückgegeben Listing 5.16 Allen (2023).

```
struct StationView: View {
    @State private var stations: [Station] = []
    @State private var searchText: String = ""

    private var filteredStations: [Station] {
        let query = searchText.trimmingCharacters(in: .whitespacesAndNewlines)
        guard !query.isEmpty else { return stations }

        return stations.filter { station in
            let container: [String] = [
                station.pointName,
                station.street,
                station.houseNumber,
                station.postalCode,
                station.status,
                station.powerType
            ]
            .map { $0.lowercased() }

            let item = query.lowercased()
            return container.contains(where: { $0.contains(item) })
        }
    }
```

Listing 5.16: Stationen werden auf Basis der Nutzereingabe gefiltert

Der *.searchable* Modifier Listing 5.17 integriert diese Suchleiste in die List-View innerhalb eines NavigationStack azamsharp (2025).

```
            .searchable(text: $searchText,
                        placement: .automatic,
                        prompt: "Seriennummer...")
```

Listing 5.17: Einbindung der Suchleiste

Kapitel 6

Validierung

In diesem Kapitel werden die Ergebnisse der Implementierung anhand von Bildschirmfotos der Demo dokumentiert. Dabei wurden die Haupt Use-Cases und weitere Funktionen getestet.

6.1 Ergebnisse der Implementierung in der Demo-App

6.1.1 HomeView

Die Home-View zeigt die Liste Abbildung 6.1 der fünf ausgewählten Elektroautos. Als Vorschau sieht man die zuvor eingestellte Restreichweite in km und den daraus berechneten Ladezustand in Prozent.

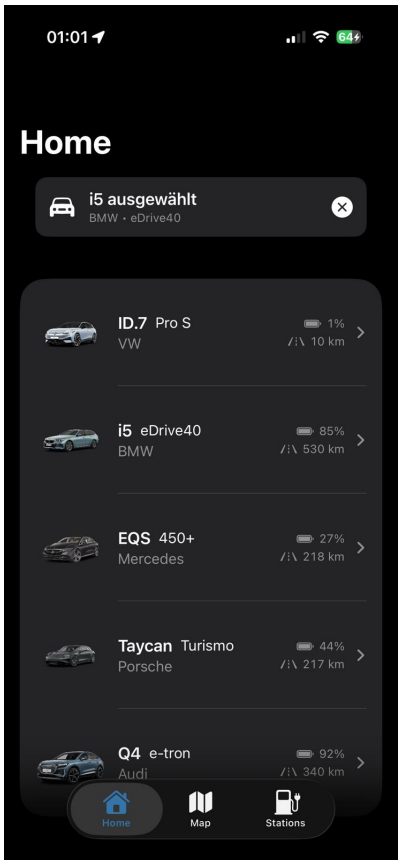


Abbildung 6.1: HomeView Ergebnis Listenansicht der Elektroautos [Eigene Darstellung]

Beim Klick auf einen Eintrag in der Liste öffnet sich die Detailansicht Abbildung 6.2, in der die Werte angezeigt werden, die in der cars.json-Datei hinterlegt sind. Hierfür sind diese Werte in Kacheln, zusammen mit einem passenden Symbol hinterlegt. Ebenfalls kann man in dieser Ansicht weiter nach unten scrollen.

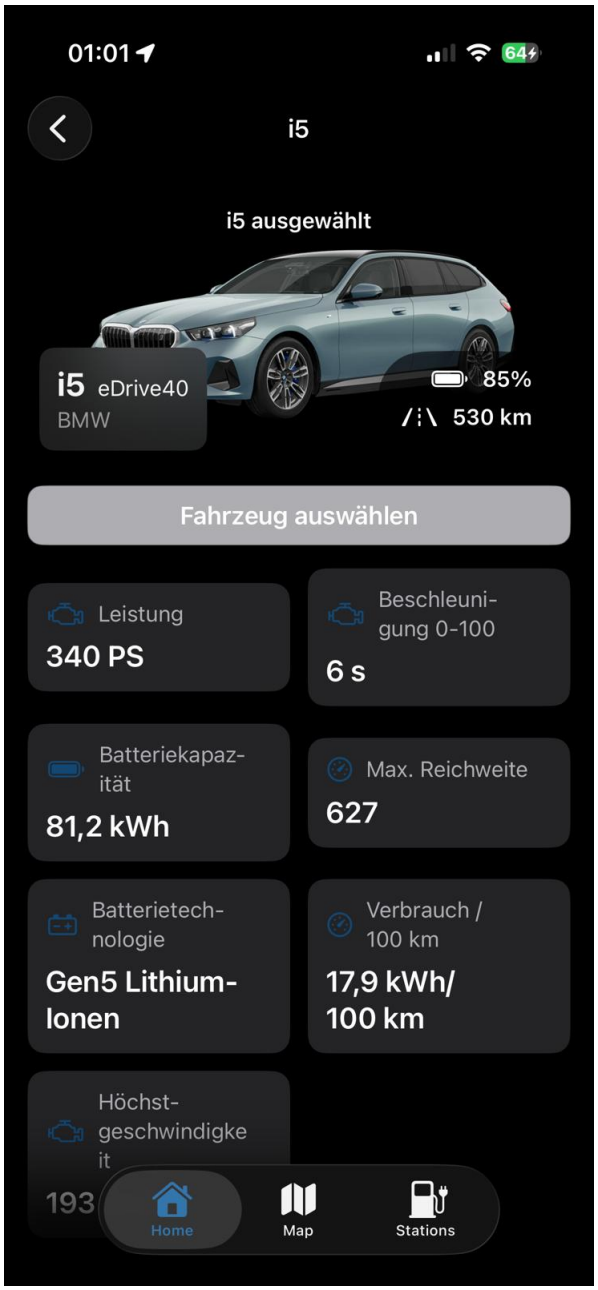


Abbildung 6.2: HomeView Ergebnis Detailansicht VW ID.7 [Eigene Darstellung]

Der Button „Fahrzeug auswählen“ ist funktionsfähig und löst den gewünschten Toast aus. In der Übersicht der HomeView kann das ausgewählte Fahrzeug gesehen werden Abbildung 6.1. In diesem Fall wurde der ID.7 von VW ausgewählt. Diese Auswahl kann über den "XButton gelöscht werden.

6.1.2 MapView

Beim Starten der App wird erfolgreich die Standortfreigabe vom Nutzer abgefragt Abbildung 6.3. Dabei erscheint die zuvor in den Target Properties eingetragene Nachricht, die den Nutzer informiert und um Erlaubnis bittet, den Standort zu verwenden. Dadurch ist sichergestellt, dass die Standortdaten nur mit Zustimmung der Nutzer erfasst werden.

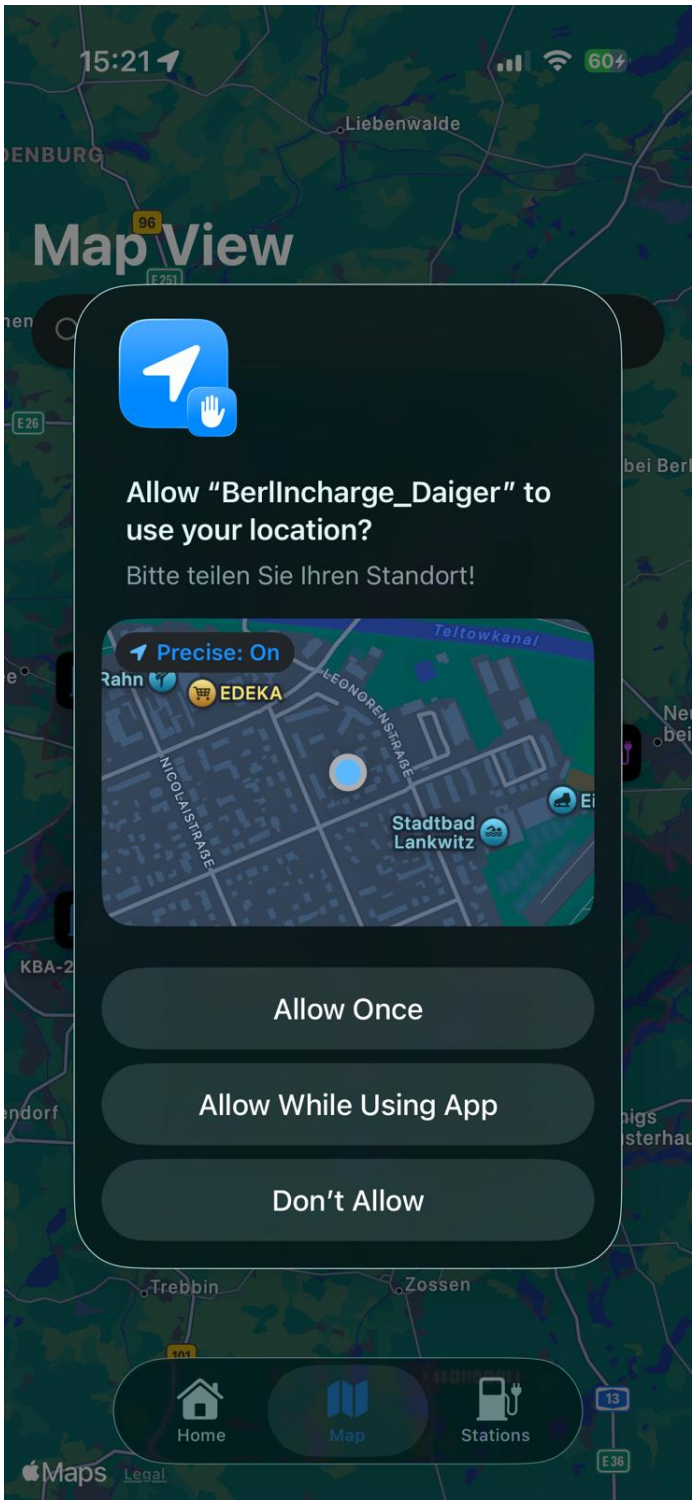


Abbildung 6.3: Standort Abfrage [Eigene Darstellung]

Auf der Karte werden alle Ladestationen als Marker dargestellt Abbildung 6.4. Die blauen Marker kennzeichnen die AC-Ladestationen, während die pinken Marker die DC-Ladestationen symbolisieren. Oberhalb der Karte ist eine Suchzeile sichtbar, und als Navigationstitel wird „Map View“ angezeigt, was zur Orientierung innerhalb der App beiträgt.

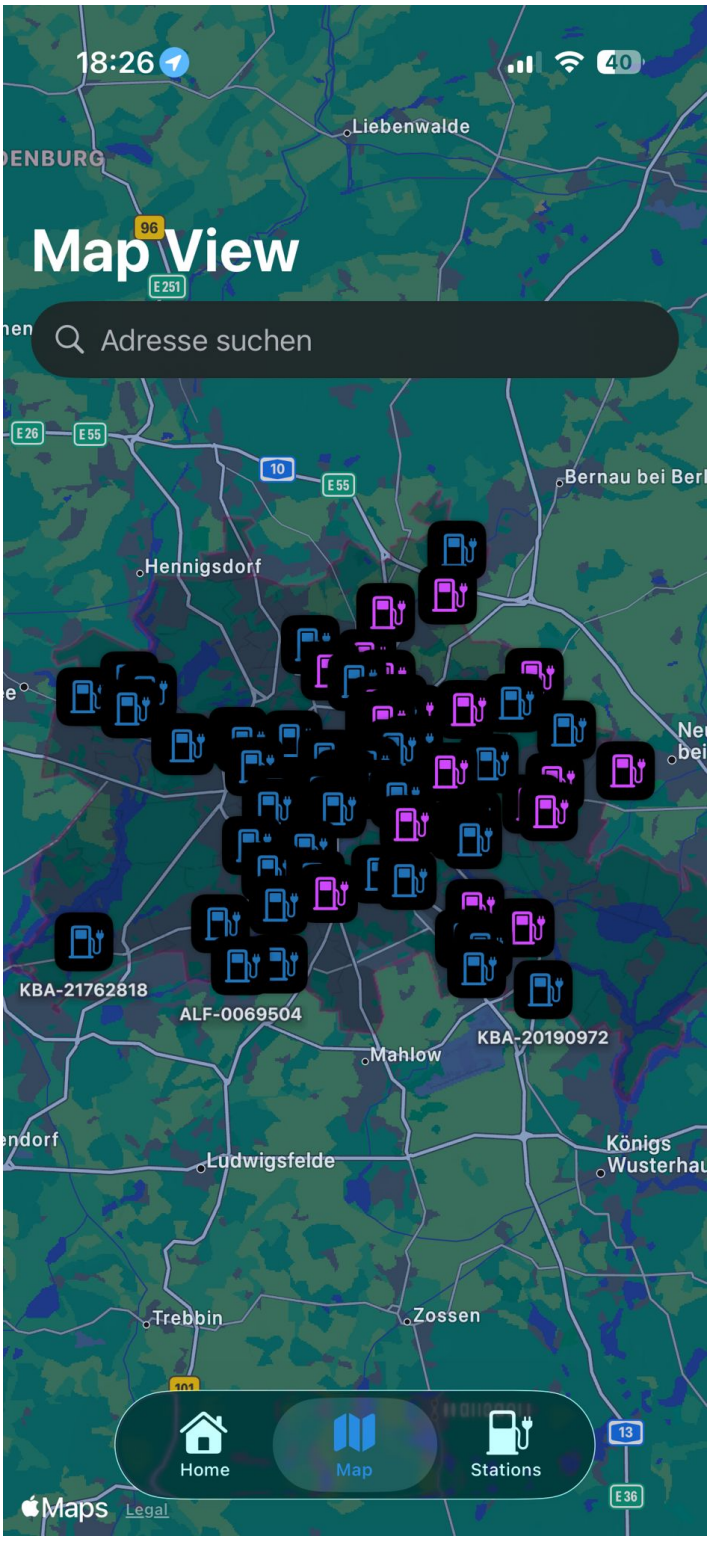


Abbildung 6.4: Kartenansicht [Eigene Darstellung]

Beim Anklicken einer Ladestation auf der Karte öffnet sich die Ladestationen DetailCard Abbildung 6.5, welche alle relevanten Informationen zur ausgewählten Station übersichtlich darstellt.

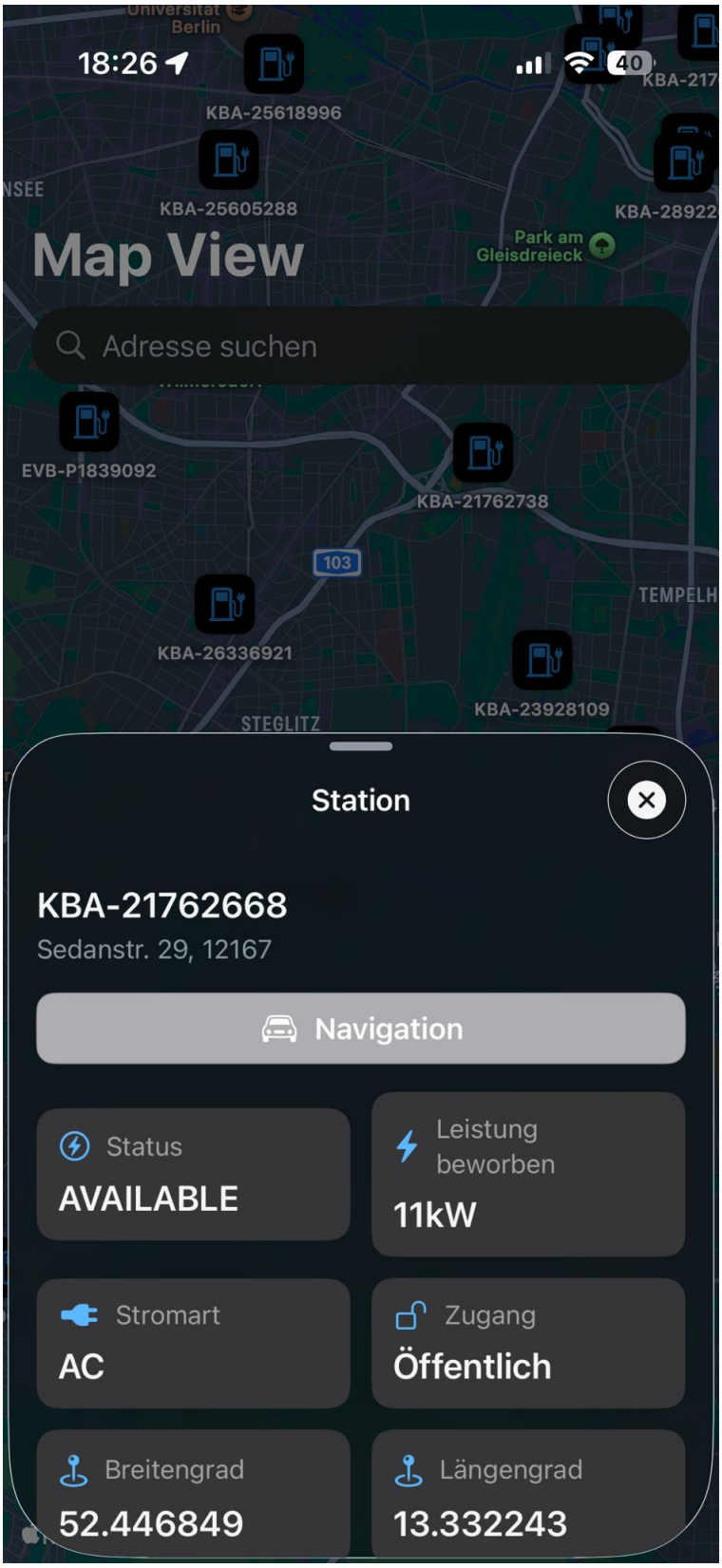


Abbildung 6.5: DetailView der Station [Eigene Darstellung]

Über die Suchzeile lässt sich eine Adresse eingeben. Im gezeigten Beispiel Abbildung 6.6 wird die Adresse unserer Hochschule eingegeben, um gezielt danach zu suchen oder die Karte darauf auszurichten.

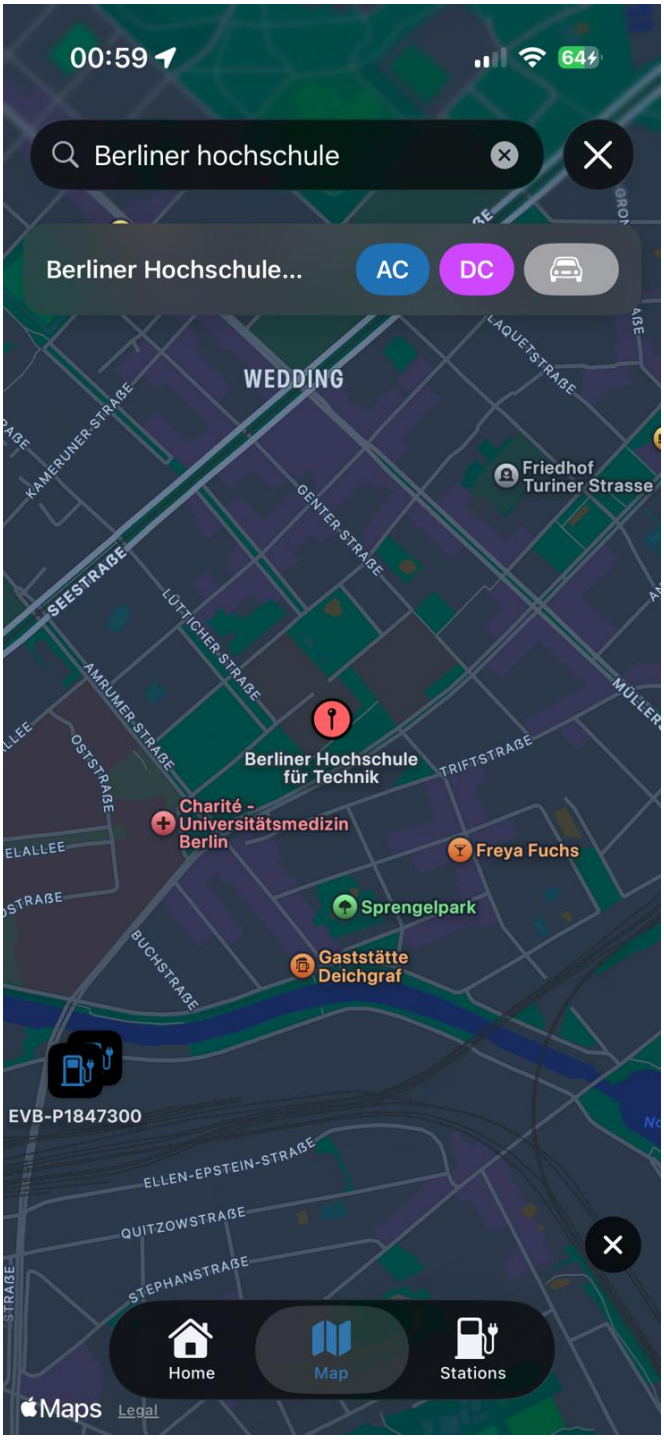


Abbildung 6.6: Marker der Hochschule auf der Karte [Eigene Darstellung]

Über den grauen „Navigieren“-Knopf oben Abbildung 6.6 kann die Route zum ausgewählten Pin berechnet und auf der Karte angezeigt werden. Oben rechts Abbildung 6.7 werden dabei die Restreichweite nach der Fahrt, die zurückzulegende Strecke sowie die prognostizierte Fahrzeit und Ladezustand angezeigt.

Für den nächsten Test wird der Ort Tempelhof eingegeben. Durch Klicken auf den blauen AC-Knopf oben Abbildung 6.6 wird dann eine Gesamtroute berechnet und angezeigt Abbildung 6.7, die den Weg inklusive der AC-Ladestation umfasst.

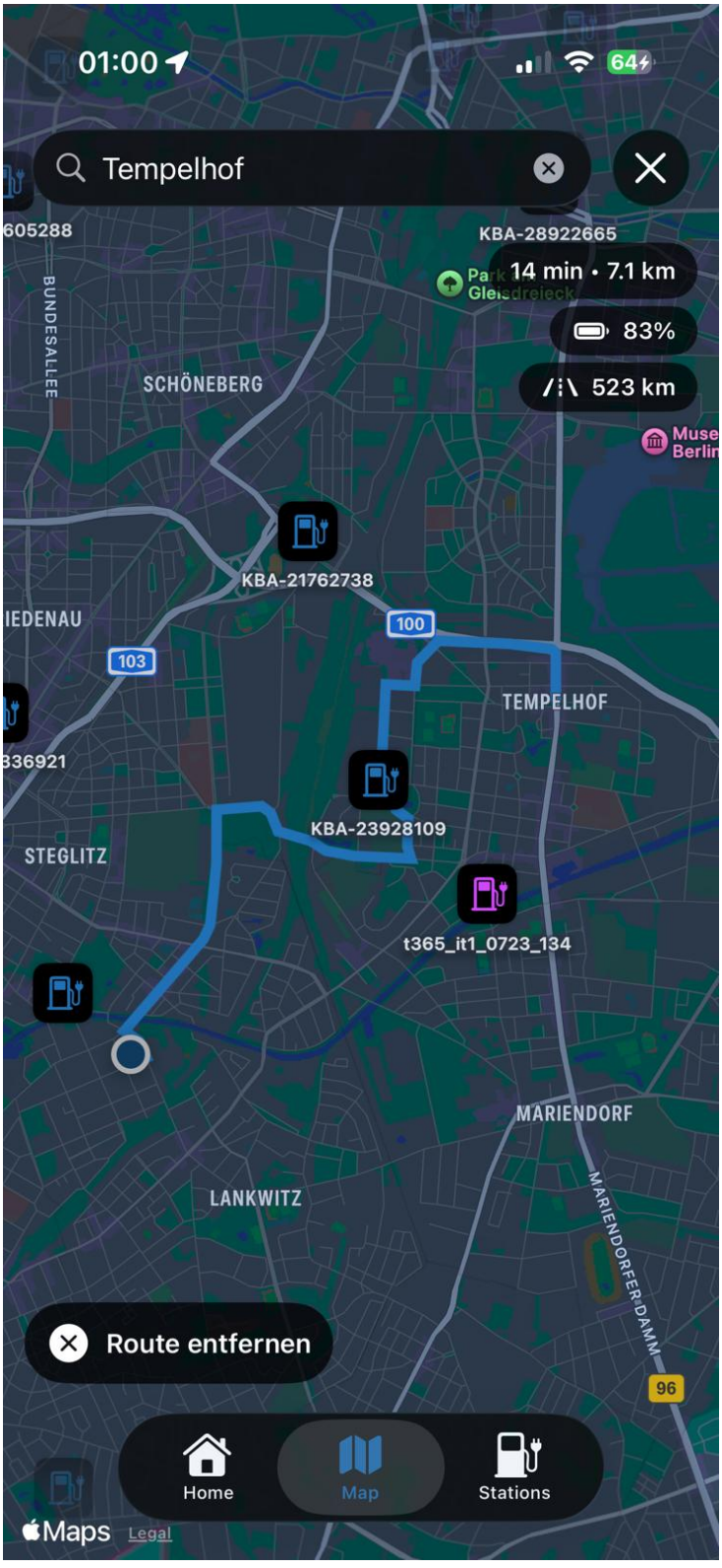


Abbildung 6.7: Berechnete Route mit einem AC-Ladestop [Eigene Darstellung]

Die gleichen Bedingungen gelten auch für eine DC-Ladestation: Es wird der Ort Mariendorf eingegeben und auf den pinken DC-Knopf oben geklickt Abbildung 6.6. Auch in diesem Fall wird die Route berechnet und angezeigt Abbildung 6.8, sofern sich eine passende DC-Ladestation auf der Route befindet.

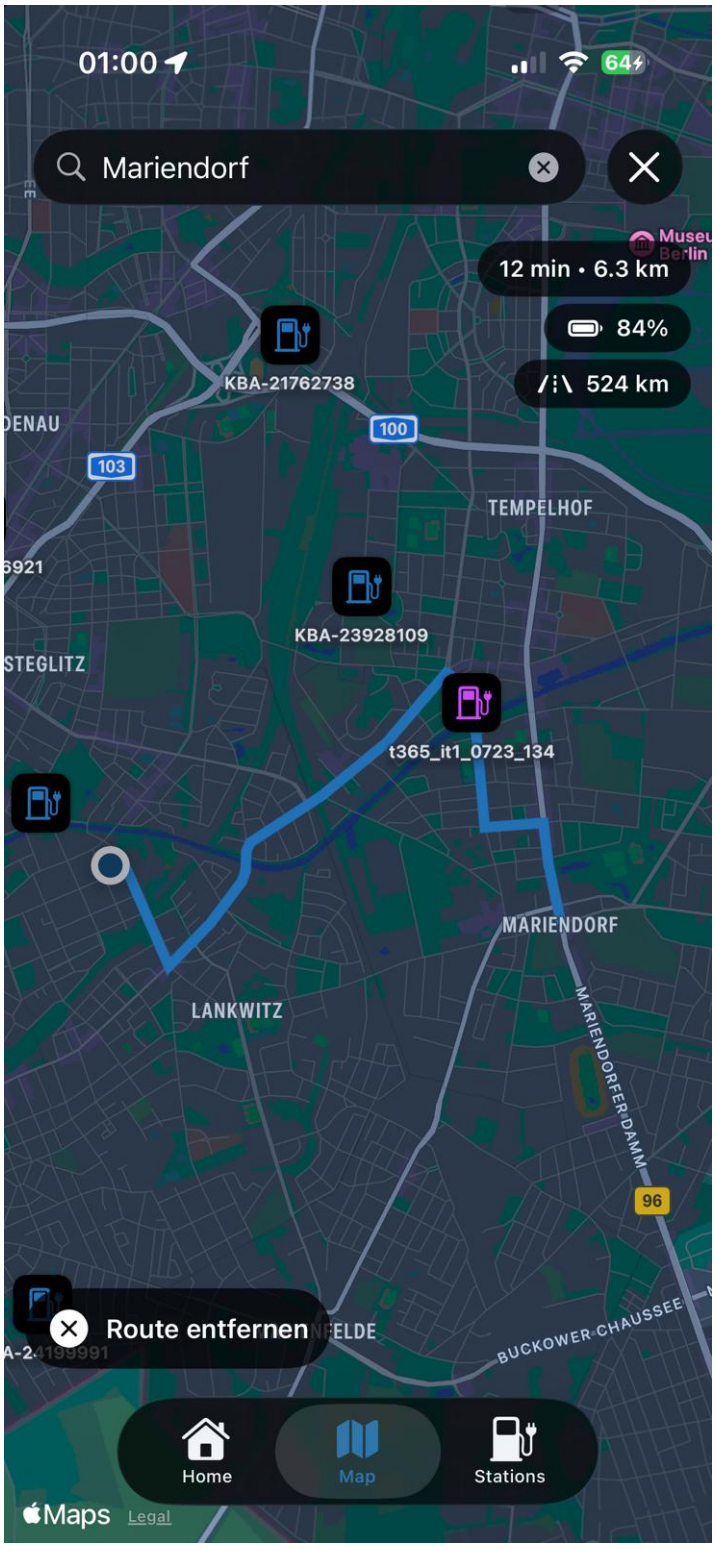


Abbildung 6.8: Berechnete Route mit einem DC-Ladestop [Eigene Darstellung]

6.1.3 StationView

Im Station-Tab werden alle Ladestationen aus der Stations.json-Datei in einer Liste Abbildung A.1 angezeigt. Dadurch hat der Nutzer einen schnellen Überblick über alle verfügbaren Ladestationen, die in der JSON-Datei hinterlegt sind.

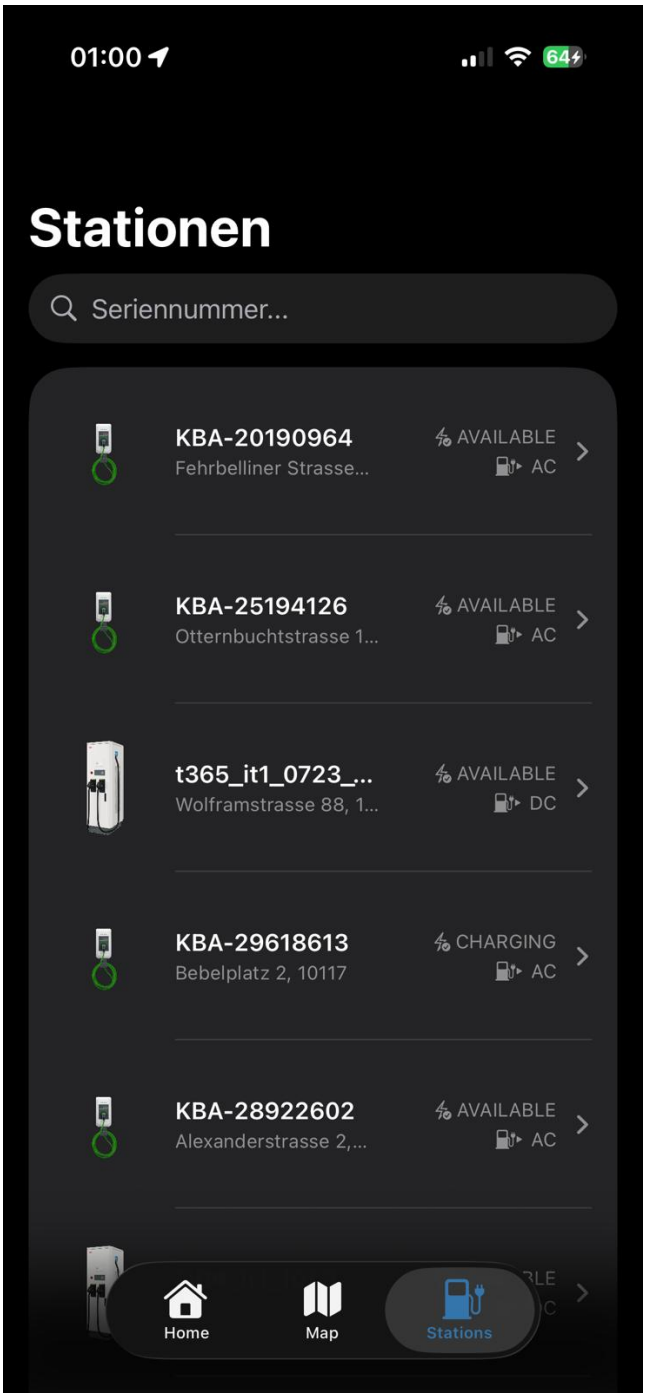


Abbildung 6.9: Ladestationen im StationView [Eigene Darstellung]

Wenn auf die gewünschte Ladestation geklickt wird, werden im DetailView Abbildung 6.10 alle weiteren Daten der Station angezeigt. Diese Informationen sind ebenfalls in der Card Abbildung 6.5 auf der MapView einsehbar, sodass der Nutzer an beiden Stellen umfassenden Zugriff auf die Details hat.

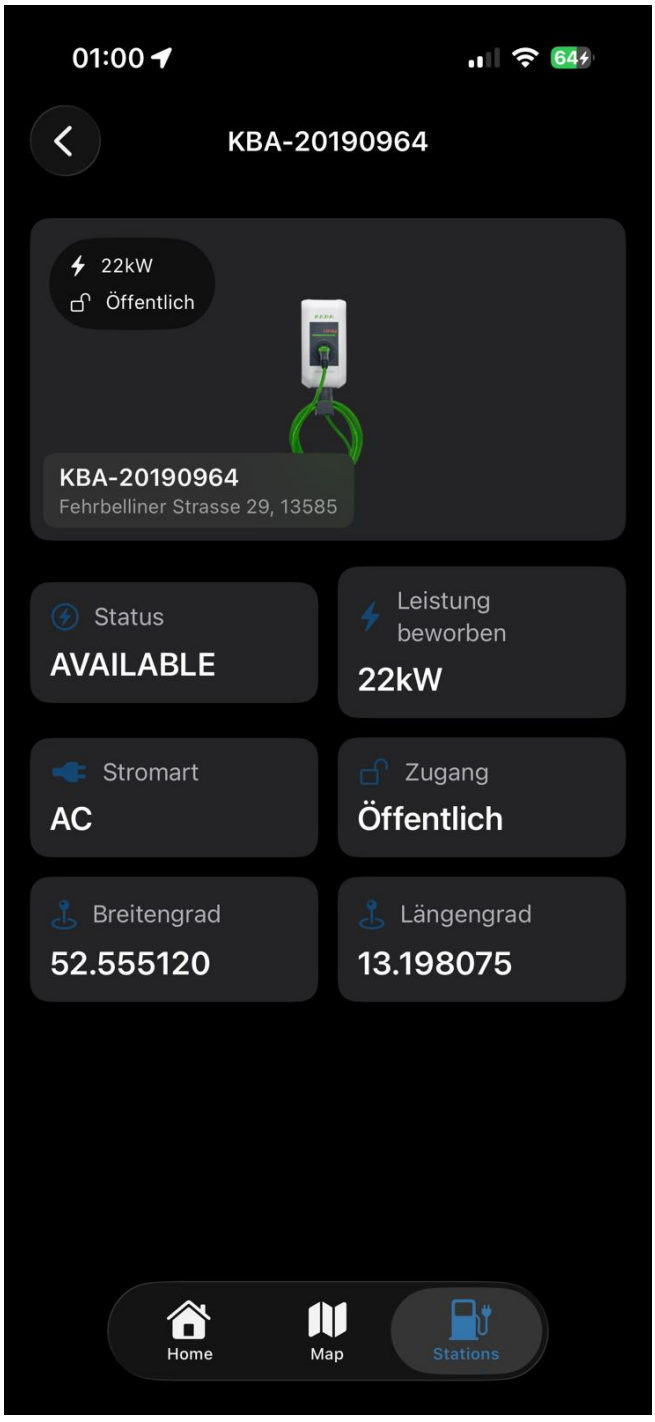


Abbildung 6.10: DetailView einer Ladestation [Eigene Darstellung]

In diesem Abschnitt wurden die zentralen Use Cases der Anwendung beschrieben. Eine vollständige Demonstration der App, einschließlich aller wesentlichen Funktionen und Abläufe, ist im beigefügten Video dargestellt. Das Demonstrationsvideo befindet sich zusammen mit dem Quellcode in der BHT-Cloud und kann dort eingesehen werden. Die Links dazu befinden sich im Anhang A.1 und A.2.

6.2 Verifizierung der Anforderungen

Die Validierung der Anforderungen erfolgte anhand der Ergebnisse im vorherigen Kapitel. In der Tabelle Abbildung 6.11 ist der finale Stand zum Projekt dokumentiert, der die Erfüllung der definierten Kriterien und den aktuellen Fortschritt übersichtlich darstellt. Der aktuellste Stand der nicht-funktionalen Anforderungen finden sich in der Tabelle darunter wieder Abbildung 6.12.

Für die funktionalen Anforderungen wurde folgender Stand erreicht:

FA#	Anforderung	Validierung
FA#1	Standort-Service Abfrage	Erfüllt
FA#2	Anzeige des aktuellen Standortes auf der Karte	Erfüllt
FA#3	Anzeige aller verfügbaren Ladepunkte	Erfüllt
FA#4	Navigationsfunktionen für Zieladresse und aktuellem Standort	Erfüllt
FA#5	Farbliche Unterscheidung AC- und DC-Ladestationen auf der Karte	Erfüllt
FA#6	Routenberechnung mit Fahrzeugprofil	Erfüllt
FA#7	Detailinformationen zu einer Ladesäule	Erfüllt
FA#8	Detailinformationen zu einem Elektroauto	Erfüllt
FA#9	Auswahl und Verwaltung verschiedener Fahrzeugprofile	Erfüllt
FA#10	Tripdauer, Tripstrecke und der prognostizierte Ladezustand	Erfüllt
FA#11	Umschau-, Dreh- und Zoomfunktionen der Karte	Erfüllt
FA#12	Anzeige der berechneten Route für einen Marker	Erfüllt
FA#13	Anzeige der berechneten Route für eine Ladestation	Erfüllt
FA#14	Anzeige der berechneten Route mit Zwischenstopp AC/DC	Erfüllt
FA#15	Suchfunktion für Ladestationen	Erfüllt
FA#16	Toast für Auswahl Elektroauto und Navigation	Erfüllt

Abbildung 6.11: Finaler Stand der funktionalen Anforderungen [Eigene Darstellung]

Für die nicht-funktionalen Anforderungen wurde folgender Stand erreicht:

NFA#	Anforderung	Validierung
NFA#1	Routenberechnung	Erfüllt
NFA#2	Intuitive Bedienoberfläche	Erfüllt
NFA#3	Aktuelle Elektroautodaten	Erfüllt
NFA#4	Smartphone-Optimierung	Erfüllt
NFA#5	Kompatibilität aktuelle iOS-Version	Erfüllt

Abbildung 6.12: Finaler Stand der nicht-funktionalen Anforderungen [Eigene Darstellung]

Kapitel 7

Fazit

Die im Rahmen dieser Arbeit entwickelte Berlincharge-App konnte erfolgreich implementiert werden. Der Nutzer hat die Möglichkeit, verschiedene Elektroautomodelle und deren Daten einzusehen sowie alle Ladepunktstandorte der Firma Vattenfall, die zum Zeitpunkt der Erstellung der Arbeit als Momentaufnahme vom MyIncharge-Portal verfügbar waren, auf der Karte anzusehen. Die Stationen können durchsucht und deren Daten eingesehen werden. Auf der Karte der App kann der Nutzer seinen aktuellen Standort sehen, einen Ladepunkt anklicken, eine Detailkarte öffnen und von dort aus zur Station navigieren sowie die Route auf der Karte anzeigen lassen. Über die Suchfunktion kann ein beliebiger Punkt auf der Karte gesetzt und entweder direkt dorthin navigiert oder eine Ladepause über AC oder DC eingelegt werden. Bei allen Routen werden oben rechts die Fahrtzeit sowie die Entfernung angezeigt, und bei der Auswahl eines Fahrzeugprofils zusätzlich die Restreichweite und der Ladezustand nach der Fahrt. Beim Start der Navigation und der Auswahl eines Fahrzeugprofils wird eine Toast-Meldung oben auf dem Bildschirm als Rückmeldung angezeigt.

7.1 Ausblick

Die App erfüllt die Grundfunktionen und funktioniert auch von unterwegs aus zuverlässig. Die Routenberechnung wurde von verschiedenen Standorten getestet, und es konnte erfolgreich eine Route angezeigt werden.
Für den weiteren Ausbau der App wären folgende Erweiterungen sinnvoll:

- Einbindung des aktuellen Verfügbarkeitsstatus der Ladestationen über eine API.
- Anzeige von aktuellen Preisen pro Kilowattstunde.
- Integration von Informationen über eventuelle Defekte der Ladestationen.
- Ergänzung von Bewertungen zur Zuverlässigkeit basierend auf vergangenen Ausfällen.
- Ausweitung der Navigationsfunktion auf ganz Deutschland.
- Verbindung der App mit dem Elektroauto über eine Schnittstelle zum Batteriesystem, um den aktuellen Batteriezustand anzeigen zu können.
- Implementierung zusätzlicher Animationen zur Verbesserung des Nutzererlebnisses.

Anhang A

Diverses

A.1 Demonstrationsvideo

Die Videos zur Demonstration der realisierten Funktionen befinden sich unter: Demonstrationsvideo. Im Ordner befinden sich das Demonstrationsvideo mit dem Dateinamen „BA_Demonstrationsvideo“ sowie zusätzliche Screenrecordings der App „BasicFeatureShowcase“ und „RoutingDetailShowcase“.

https://cloud.bht-berlin.de/index.php/s/oxS57jD6oadFwec

A.2 Quellcode

Der Quellcode befindet sich unter: Quellcode

https://cloud.bht-berlin.de/index.php/s/75osFnsB6CrTGAJ



Abbildung A.1: BHT Cloud Code und Videos [Eigene Darstellung]

A.3 Kürzungen

A.3.1 Aus dem Kapitel Grundlagen und Entwicklung der Elektromobilität

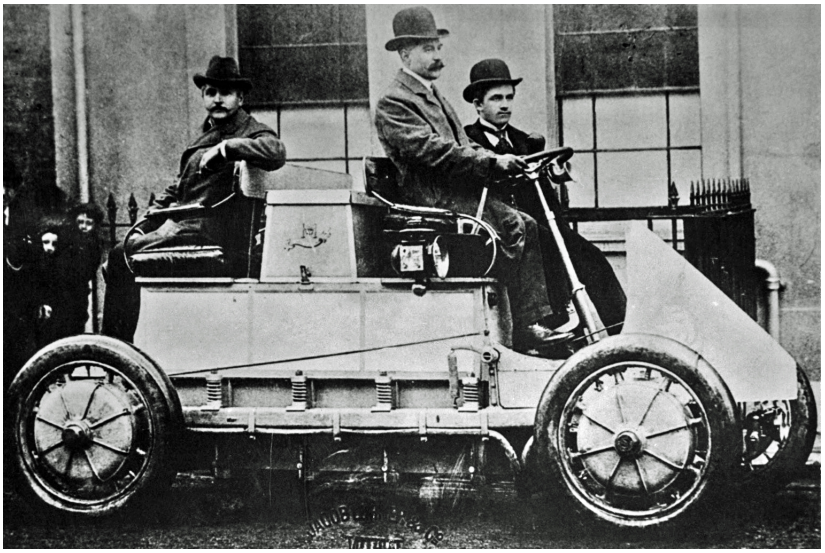


Abbildung A.2: Lohner-Porsche mit Allradantrieb (1900) Wikipedia (2025c)

Vergleicht man Verbrennungs- und Elektromotoren, zeigen sich deutliche Unterschiede in Aufbau und Funktionsweise. Reine Elektrofahrzeuge verfügen im Vergleich zu Fahrzeugen mit Verbrennungsmotor über einen wesentlich einfacheren mechanischen Aufbau. Zahlreiche Komponenten entfallen, darunter der Kraftstofftank, die Benzinpumpe, der Ölkreislauf mit Öltank sowie Bauteile wie Katalysator oder Abgasanlage. Ein weiterer Vorteil liegt in der Funktionsweise des Elektromotors selbst: Er kann ohne mechanische Umschaltung zwischen Vorwärts- und Rückwärtsbetrieb wechseln und benötigt keine komplexe Regelung hinsichtlich Zündzeitpunkt oder Kraftstoffdosierung, wie es beim Verbrennungsmotor erforderlich ist. Darüber hinaus zeichnen sich Elektrofahrzeuge durch eine deutlich geringere Geräuschentwicklung aus, insbesondere im innerstädtischen Verkehr und bei niedrigen bis mittleren Geschwindigkeiten. Dies trägt zu einer höheren Lebensqualität für Fahrer, Anwohner und andere Verkehrsteilnehmer bei. Gleichzeitig steht dieser leise Betrieb jedoch in der Kritik, da Elektrofahrzeuge von Fußgängern - insbesondere von Kindern oder sehbehinderten Personen - oft zu spät wahrgenommen werden können, was potenziell das Unfallrisiko erhöht. Die Bezahlung des Ladevorgangs kann auf unterschiedliche Weise erfolgen - etwa direkt an der Ladesäule per EC-Karte, über RFID-Karten für Vertragskunden oder über mobile Anwendungen wie die InCharge-App von Vattenfall. Die verschiedenen Ladearten und Betreiberkonzepte unterscheiden sich sowohl in ihren Investitions- und Betriebskosten als auch in der verwendeten Software- und Backend-Infrastruktur, über die Autorisierung, Abrechnung und Verwaltung der Ladevorgänge erfolgen. Ein weiterer entscheidender Faktor ist die Temperatur der Zellen. Beim Laden oder Entladen führen niedrige Zelltemperaturen zu einem früheren Erreichen der Entladeschlussspannung, wodurch weniger Energie abgegeben werden kann. Daher ist ein Temperaturmanagement integraler Bestandteil moderner Batteriesysteme. Für eine lange Lebensdauer müssen sowohl äußere Einflüsse, wie mechanische Belastungen, als auch Betriebsparameter berücksichtigt werden. Dazu zählen insbesondere das Ladeverfahren und der genutzte Ladezustand (State of Charge, SOC). Für Lithium-Ionen-Zellen liegt die Toleranz der Ladeschlussspannung bei etwa ±0,05 Volt bei einer Nennspannung von 4,2 Volt Karle (2022).

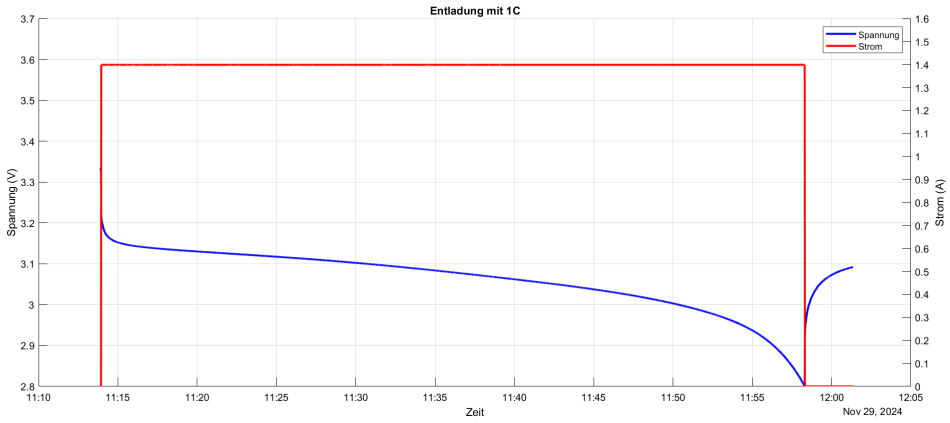


Abbildung A.3: Entladung mit 1C [Eigene Darstellung]

Während bei herkömmlichen Zellen in der Regel eine maximale Entladerate von bis zu 3C vorgesehen ist, kann bei der Verwendung einer Pulsladung kurzzeitig eine Ladeleistung von bis zu 10C erreicht werden, wie ein Versuch mit einer LFP-Zelle zeigt. Dies ist möglich, weil die hohen Stromimpulse nur über sehr kurze Zeiträume hinweg auftreten. Das Lade- und Entladeverfahren kann dabei einem PWM-Muster (Pulsweitenmodulation) folgen. Dabei werden kurze Strompulse definiert, die jeweils etwa ein Prozent der Gesamtenergiemenge übertragen. Die zugeführte Energie ergibt sich dabei aus der Stromstärke multipliziert mit der Impulsdauer. Während jedes Stromimpulses sinkt die Ruhespannung der Zelle kurzzeitig ab und erreicht eine niedrigere Spannungsebene. Diese Spannungsdifferenz stellt den Spannungsabfall dar, der durch den Innenwiderstand der Zelle verursacht wird.

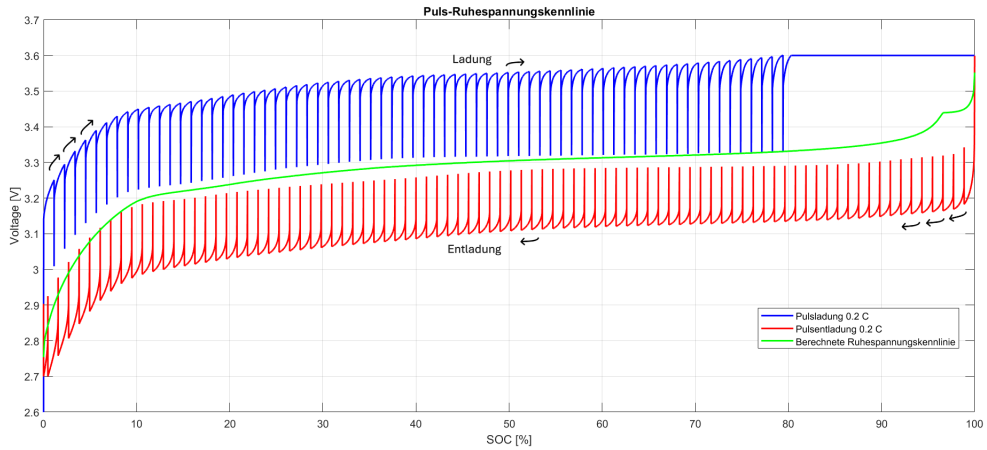


Abbildung A.4: Ruhespannungskennlinie durch Pulsmessung [Eigene Darstellung]

Nach jedem Stromimpuls steigt die Zellspannung wieder an und nähert sich allmählich ihrer Ruhespannungskennlinie an. Da dieses Ladeverfahren im öffentlichen Betrieb keine Anwendung findet, wird in der Praxis überwiegend das CCCV-Ladeverfahren eingesetzt Karle (2022).
Da sich die Zellen im Verlauf ihres Lebenszyklus unterschiedlich verhalten, ist, wie bereits erwähnt, ein Batteriemanagementsystem (BMS) von zentraler Bedeutung. Es überwacht die Spannung jeder einzelnen Zelle und gleicht Abweichungen zwischen ihnen aus („balancing“). Dies ist notwendig, da die nutzbare Gesamtenergie eines Akkupacks stets von der schwächsten Zelle

bestimmt wird. Sobald eine einzelne Zelle die Ladeschlussspannung erreicht, beendet das BMS den Ladevorgang, selbst wenn andere Zellen noch nicht vollständig geladen sind. Um eine gleichmäßige Auslastung und eine hohe Gesamteffizienz zu gewährleisten, müssen daher alle Zellen auf einen möglichst identischen Ladezustand gebracht werden. Für diesen Ausgleich existieren verschiedene Balancing-Konzepte und Technologien, die sich in Komplexität, Wirkungsgrad und Kosten unterscheiden Doppelbauer (2025).

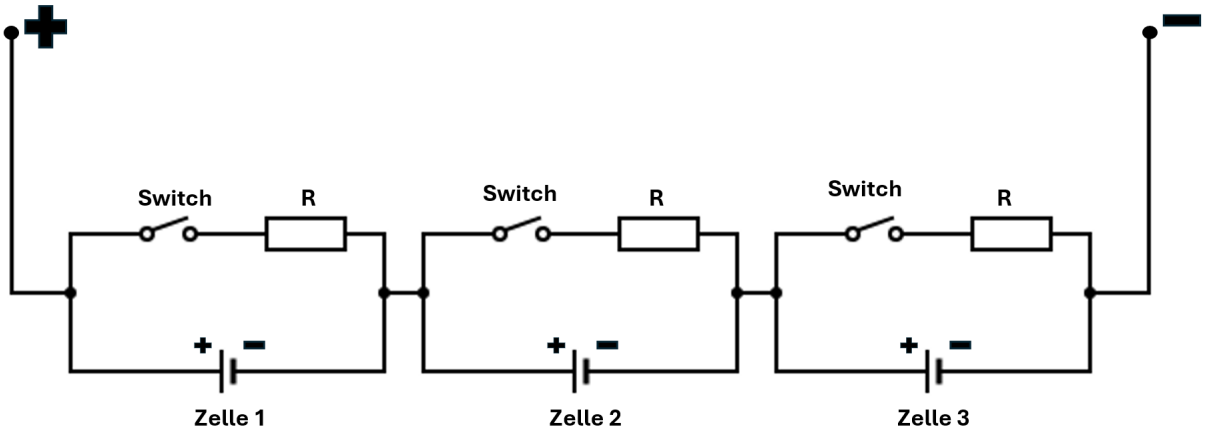


Abbildung A.5: Passives Balancing [Eigene Darstellung]

Die einfachste Form des Zellenausgleichs ist das passive Balancing. Dabei wird zu jeder Zelle ein Balancierwiderstand parallel geschaltet. Erreicht eine Zelle vor den anderen die Ladeschlussspannung, aktiviert das Batteriemanagementsystem den entsprechenden Widerstand. Dadurch fließt ein Teil des Ladestroms über den Widerstand, wodurch die betroffene Zelle weniger Strom aufnimmt. So kann der Ladevorgang fortgesetzt werden, bis auch die übrigen, schwächeren Zellen ihre Zielspannung erreichen. Diese Methode ist besonders kostengünstig, jedoch wird die überschüssige Energie in Form von Wärme über die Widerstände abgeführt. Passives Balancing wird daher überwiegend während des Ladevorgangs eingesetzt.

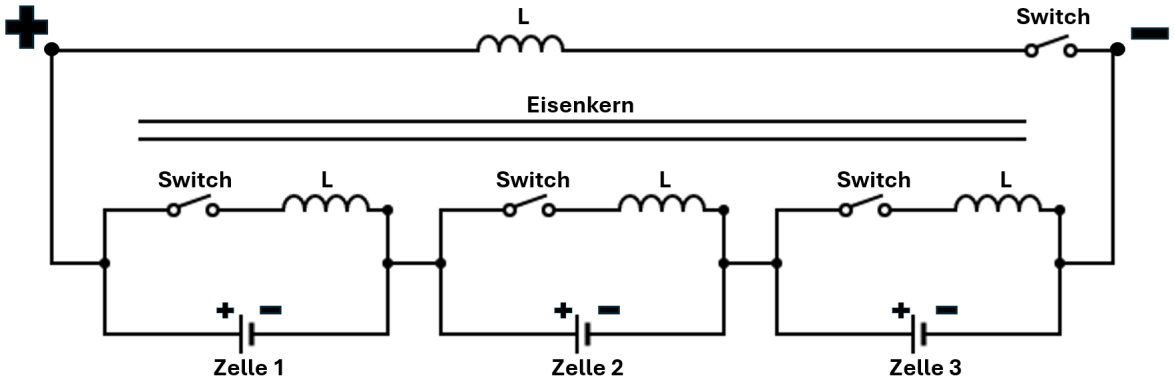


Abbildung A.6: Aktives Balancing [Eigene Darstellung]

Der komplexere Ansatz im Vergleich zum passiven Verfahren ist das aktive Balancing. Hierbei wird sowohl während des Lade- als auch während des Entladevorgangs überschüssige Energie von stärker geladenen Zellen auf schwächere Zellen übertragen. Der Energieaustausch erfolgt über elektronische Schaltungen, die Spulen, Kondensatoren und Schaltregler nutzen, um Energie

gezielt zu verteilen. Eine Möglichkeit besteht darin, mithilfe von Kondensatoren Energie schrittweise von einer Zelle zur benachbarten Zelle zu übertragen. Alternativ kann hinter jeder Zelle ein Inverswandler eingesetzt werden, der über einen Schaltregler die Energie kontrolliert an angrenzende Zellen weitergibt. Eine weitere Variante ist der Einsatz eines Sperrwandlers. Dabei wird Energie über Spulen zunächst auf eine gemeinsame Wicklung abgegeben und anschließend gezielt an schwächere Zellen zurückgeführt. Diese Verfahren ermöglichen einen deutlich effizienteren Energieausgleich, sind jedoch mit höherem Schaltungsaufwand, größerem Platzbedarf und höheren Kosten verbunden - insbesondere bei Akkupacks mit einer großen Anzahl an Zellen Khan (2024).
In Abbildung 2.11 ist eine AC-Wallbox des Herstellers KEBA zu sehen. Dieses Modell kann entweder an die Wand oder alternativ auf eine Stele montiert werden.



Abbildung A.7: KEBA KeContact P30 X-Series Green Edition Memodo (2025)

A.3.2 Aus dem Kapitel Prinzipien intelligenter Routenplanung

Der Bellman-Ford-Algorithmus berechnet die kürzeste Entfernung von einem definierten Startknoten zu allen anderen Knoten eines Graphen. Im Gegensatz zum Dijkstra-Algorithmus wird die aktuell kürzeste Entfernung hier nicht endgültig festgelegt, sondern in jeder Iteration erneut überprüft, um gegebenenfalls einen noch kürzeren Pfad zu finden. Für einen Graphen mit n Knoten werden insgesamt n - 1 Iterationen durchgeführt. In jeder Iteration werden alle Kanten in einer festen Reihenfolge überprüft. In einem Beispielnetz könnte dies bedeuten, dass zuerst die Teilstrecke A → B und anschließend B → C in genau dieser Reihenfolge untersucht wird. Die Berechnungen lassen sich tabellarisch darstellen. Typischerweise enthält die Tabelle drei Spalten: 1. den Knotennamen, 2. die aktuell kürzeste bekannte Entfernung zum Startknoten, 3. den vorherigen Knoten, über den dieser erreicht wird. Da für jeden Knoten eine feste Richtung zum Nachbarknoten definiert ist, lassen sich diese Verbindungen im Stadtverkehrsinformationssystem sinngemäß als Einbahnstraßen interpretieren. Ein wesentlicher Vorteil des Bellman-Ford-Algorithmus gegenüber dem Dijkstra-Verfahren ist seine Fähigkeit, auch negative Kantengewichte korrekt zu verarbeiten. Das Verfahren erkennt zudem negative Zyklen, also Schleifen im Graphen, bei denen die Summe der Gewichtungen kleiner als null wird. In diesem Fall wird der Algorithmus abgebrochen, da keine gültige Lösung existiert Sambol (2015).

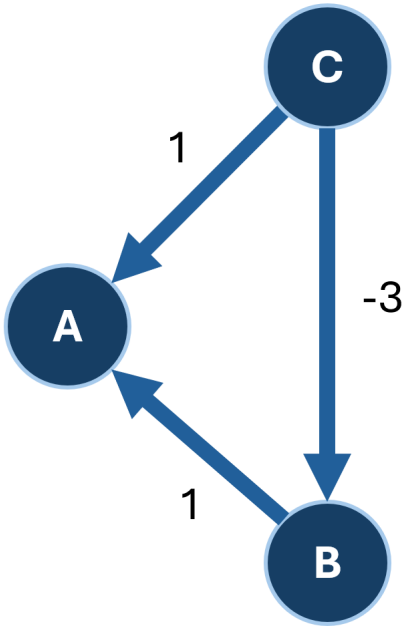


Abbildung A.8: Negativer Zyklus [Eigene Darstellung]

Ein negativer Zyklus entsteht immer dann, wenn ein Knoten über eine negative Kantenlänge mehrfach erreicht werden kann und sich dabei die Gesamtdistanz bei jeder Iteration weiter verringert. Der Algorithmus würde in diesem Fall endlos weiterlaufen, da die Entfernung theoretisch immer kleiner werden könnte. In der Realität ist ein solcher Fall jedoch nicht möglich, da negative Entfernungen physikalisch nicht existieren. Zur Veranschaulichung betrachten wir ein Beispielmodell mit n = 6 Knoten. Gemäß dem Bellman-Ford-Verfahren müssen daher n - 1 = 5 Iterationen durchgeführt werden, um für jede Kante alle möglichen Pfadverkürzungen zu prüfen. Das Beispiel kann tabellarisch oder grafisch dargestellt werden, um den Verlauf der Distanzaktualisierungen pro Iteration zu veranschaulichen KSH (2019).

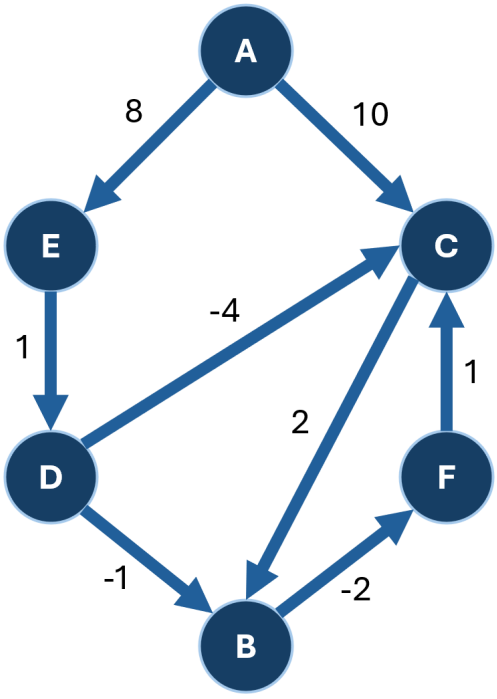


Abbildung A.9: Bellman-Ford-Algorithmus [Eigene Darstellung]

Wir beginnen im Startknoten A und fahren zunächst zum Knoten C. Die anfängliche Entfernung wird von ∞ auf 10 aktualisiert, da der Weg von A nach C diese Länge aufweist. Der Vorgänger des Knotens C ist somit A. Vom Knoten C aus erreichen wir den Knoten B, da direkte Verbindungen zu D oder F nicht bestehen - diese Kanten sind im Modell als Einbahnstraßen ausgelegt. Die Strecke von A über C nach B beträgt 12, weshalb der Wert für B entsprechend aktualisiert wird. Von B aus führt eine Verbindung zu F mit einer Kantenlänge von -2. Die neue Entfernung für F berechnet sich somit zu 10 (12 - 2), wobei B als Vorgängerknoten eingetragen wird. Nun betrachten wir den anderen Zweig des Netzwerks: Von A aus gelangen wir über eine weitere Verbindung zu E mit einer Entfernung von 8. Ausgehend von E können wir D erreichen, wodurch sich eine Gesamtdistanz von 9 ergibt. Damit wurde in der ersten Iteration jedem Knoten eine vorläufige Entfernung zugewiesen. Nach Abschluss dieser Iteration sind alle aktuell bekannten Distanzen und ihre jeweiligen Vorgängerknoten im Modell vermerkt.

Iteration 1					
0	12	10	9	8	10
A	B	C	D	E	F

Abbildung A.10: Tabelle zu Iteration 1 [Eigene Darstellung]

In der zweiten Iteration prüfen wir, ob sich die bisherigen Streckenwerte weiter optimieren lassen. Dazu werden erneut alle Kanten des Graphen in der festgelegten Reihenfolge überprüft, um mögliche Verbesserungen der Distanzen zu ermitteln. Für die Knoten F und B über C ergibt sich keine Änderung, da keine kürzere Verbindung gefunden wird. Betrachten wir nun den linken Zweig des Graphen, so ergibt sich eine Anpassung: Über den Knoten D kann C in einer kürzeren Entfernung erreicht werden. Die Gesamtdistanz über den Pfad A → E → D → C berechnet sich dabei aus der bisherigen Entfernung von 9 und der negativen Kantenlänge -4. Damit reduziert sich die Distanz zu C von 10 auf 5. Ein ähnlicher Effekt zeigt sich beim Knoten B. Über den Pfad A → E → D → B reduziert sich die bisherige Entfernung von 12 auf 8. Alle übrigen Knoten behalten ihre Distanzen aus der ersten Iteration bei, da keine Optimierung möglich ist.

Iteration 2					
0	8	5	9	8	10
A	B	C	D	E	F

Abbildung A.11: Tabelle zu Iteration 2 [Eigene Darstellung]

In der dritten Iteration wird die bisherige Vorgehensweise fortgeführt. Dabei zeigt sich erneut eine Optimierung des Knotens B: Da die Teilstrecke zum Startpunkt A über den Zwischenknoten C nun auf 5 reduziert wurde, verringert sich auch die Entfernung zu B von 8 auf 7. Diese Änderung wirkt sich unmittelbar auf den Knoten F aus. Über den aktualisierten Pfad A → C → B → F reduziert sich dessen Entfernung von 10 auf 5. Somit ergibt sich nach Abschluss der dritten Iteration folgende Gesamtsituation für die bekannten Entfernungen zu den einzelnen Knoten:

Iteration 3					
0	7	5	9	8	5
A	B	C	D	E	F

Abbildung A.12: Tabelle zu Iteration 3 [Eigene Darstellung]

In der vierten Iteration werden alle Kanten erneut geprüft. Dabei zeigt sich, dass keine weiteren Optimierungen mehr möglich sind. Alle Distanzen bleiben unverändert, was bedeutet, dass für jeden Knoten bereits die kürzeste Entfernung vom Startpunkt A ermittelt wurde. Wie zu Beginn erwähnt, sind für ein Modell mit n Knoten höchstens n - 1 Iterationen erforderlich, um das Optimum zu erreichen. In diesem Beispiel konnte die Berechnung jedoch bereits nach dem vierten Durchlauf beendet werden, da keine Verbesserungen mehr eingetreten sind Byte (2025).

Literaturverzeichnis

Ackermann, R. (2025). *Routing with mapkit and core location.* Zugriff auf https://www.kodeco.com/10028489-routing-with-mapkit-and-core-location (Zugriff am 22.12.2025)

Allen, S. (2020). *Swiftui navigation - navigationview navigationlink tutorial.* Zugriff auf https://www.youtube.com/watch?v=tnNFoZ7CkP8 (Zugriff am 22.12.2025)

Allen, S. (2021). *Swiftui - tabview tutorial.* Zugriff auf https://www.youtube.com/watch?v=tnNFoZ7CkP8 (Zugriff am 22.12.2025)

Allen, S. (2023). *Swiftui search bar - searchable.* Zugriff auf https://www.youtube.com/watch?v=iTqwa0DCIMA (Zugriff am 22.12.2025)

Allen, S. (2025). *Swiftui maps - ios 17 - look around, user location, directions, markers annotations, map styles.* Zugriff auf https://www.youtube.com/watch?v=98rQZbwxMFI (Zugriff am 22.12.2025)

Audi. (2025). *Q4 e-tron.* Zugriff auf https://www.audi.de/de/neuwagen/q4-e-tron/q4-e-tron/konfigurator/?ulid=1761392783870&pr=F4BACM0_2026%7CC2C2%7CAI#configurator (Zugriff am 22.12.2025)

azamsharp. (2025). *Filtering list in swiftui using searchable modifier.* Zugriff auf https://www.youtube.com/watch?v=gcD182QO8xg&t=1s (Zugriff am 22.12.2025)

AzamSharp. (2025). *Introduction to communication patterns in swiftui.* Zugriff auf https://azamsharp.com/2024/09/22/introduction-to-communication-patterns-in-swiftui.html (Zugriff am 22.12.2025)

BMW. (2025). *Technische daten der bmw i5 limousine.* Zugriff auf https://www.bmw.de/de/neufahrzeuge/bmw-i/i5/bmw-i5-ueberblick.html/bmw-i5-edrive40-limousine.bmw (Zugriff am 22.12.2025)

Byte, H. (2025). *Bellman-ford shortest path algorithm visually explained.* Zugriff auf https://www.youtube.com/watch?v=Mn9bFIIyXIM&t=585s (Zugriff am 22.12.2025)

C, C. (2025). *Swiftui mapkit: How to display a map and show a location.* Zugriff auf https://codewithchris.com/swiftui-mapkit/ (Zugriff am 22.12.2025)

Chen, P. (2025). *Mercedes cla 2025 – fast 800 km reichweite, jetzt ab 55.858 € bestellbar.* Zugriff auf https://www.carwow.de/mercedes/cla/auto-news/3954/mercedes-benz-cla-2025 (Zugriff am 22.12.2025)

Developer, A. (2025a). *Jsondecoder.* Zugriff auf https://developer.apple.com/documentation/foundation/jsondecoder (Zugriff am 22.12.2025)

Developer, A. (2025b). *Mkgeojsondecoder.* Zugriff auf https://developer.apple.com/documentation/mapkit/mkgeojsondecoder (Zugriff am 22.12.2025)

Doppelbauer, M. (2025). *Grundlagen der Elektromobilität Technik, Praxis, Energie und Umwelt* (2. Aufl.). Springer Vieweg. (ISBN 978-3-658-44827-1)

electrive, F. T. (2025). *Mittlerweile 6.000 öffentliche ladepunkte in berlin.* Zugriff auf https://www.electrive.net/2025/10/01/mittlerweile-6-000-oeffentliche-ladepunkte-in-berlin/ (Zugriff am 22.12.2025)

EUR-Lex. (2025). *Verordnung (eu) 2023/851 des europäischen parlaments und des rates vom 19. april 2023 zur Änderung der verordnung (eu) 2019/631 im hinblick auf eine verschärfung der co2-emissionsnormen für neue personenkraftwagen und für neue leichte nutzfahrzeuge im einklang mit den ehrgeizigeren klimazielen der union.* Zugriff auf https://eur-lex.europa.eu/legal-content/DE/TXT/?uri=CELEX:32023R0851 (Zugriff am 22.12.2025)

geeksforgeeks. (2025). *Minimum distance from a point to the line segment using vectors.* Zugriff auf https://www.geeksforgeeks.org/dsa/minimum-distance-from-a-point-to-the-line-segment-using-vectors/ (Zugriff am 22.12.2025)

Github. (2025). *Get a list of coordinates from a mkpolyline / mkroute.* Zugriff auf https://gist.github.com/freak4pc/98c813d8adb8feb8aee3a11d2da1373f (Zugriff am 22.12.2025)

Hudson, P. (2025a). *Introducing mvvm into your swiftui project.* Zugriff auf https://www.hackingwithswift.com/books/ios-swiftui/introducing-mvvm-into-your-swiftui-project (Zugriff am 22.12.2025)

Hudson, P. (2025b). *Introducing mvvm into your swiftui project.* Zugriff auf https://www.hackingwithswift.com/books/ios-swiftui/introducing-mvvm-into-your-swiftui-project (Zugriff am 22.12.2025)

Hulatt, S. L. (2024). *Navigationsalgorithmen.* Zugriff auf https://www.youtube.com/watch?v=EFg3u_E6eHU (Zugriff am 22.12.2025)

inside digital. (2025). *ios 16: Die neuheiten in apple maps im Überblick.* Zugriff auf https://www.inside-digital.de/news/ios-16-die-neuheiten-in-apple-maps-im-ueberblick (Zugriff am 22.12.2025)

Kampker, A. (2024). *Elektromobilität Grundlagen einer Fortschrittstechnologie* (3. Aufl.). Springer Vieweg. (ISBN 978-3-662-65811-6)

Karle, A. (2022). *Elektromobilität Grundlagen und Praxis* (6. Aufl.). Hanser. (ISBN 978-3-446-47508-3)

Khan, N. (2024). *A critical review of battery cell balancing techniques, optimal design, converter topologies, and performance evaluation for optimizing storage system in electric vehicles.* Zugriff auf https://www.sciencedirect.com/science/article/pii/S2352484724002506 (Zugriff am 22.12.2025)

KSH, C. H. (2019). *Bellman ford algorithmus (graphentheorie).* Zugriff auf https://www.youtube.com/watch?v=Pv9tQ7FatGY (Zugriff am 22.12.2025)

Mathestunde.com. (2025). *Koordinatensystem - alles, was du wissen musst.* Zugriff auf https://www.mathestunde.com/koordinatensystem (Zugriff am 22.12.2025)

Memodo. (2025). *Keba kecontact p30 x-series green edition inkl. mid-zähler, pnc, kabel.* Zugriff auf https://www.memodo.de/keba-kecontact-p30-x-series-green-edition-inkl.-mid-zaehler-pnc-kabel/10596 (Zugriff am 22.12.2025)

Mercedes. (2025). *Die technischen daten des neuen eqs limousine.* Zugriff auf https://www.mercedes-benz.de/passengercars/models/saloon/eqs/overview.html (Zugriff am 22.12.2025)

Mikina, M. (2021). *Parsing json that contains multiple different types of objects in swift.* Zugriff auf https://www.youtube.com/watch?v=Tz6MZE16dfE (Zugriff am 22.12.2025)

Mobil, E. (2025). *Elektromobilität (definition i.s. der bundesregierung).* Zugriff auf https://www.erneuerbar-mobil.de/glossar/elektromobilitaet-definition-der-bundesregierung (Zugriff am 22.12.2025)

Neumann, H. (2024). *Cpo report 2024.* Zugriff auf https://emobilityexcellence.com/cpo-report-07-2024#toggle-id-3 (Zugriff am 22.12.2025)

Novichkov, A. (2025). *Drawing maps with swift charts.* Zugriff auf https://artemnovichkov.com/blog/drawing-maps-with-swift-charts (Zugriff am 22.12.2025)

odis berlin. (2025). *Ortsteile von berlin.* Zugriff auf https://daten.odis-berlin.de/de/dataset/ortsteile/ (Zugriff am 22.12.2025)

Porsche. (2025). *Taycan.* Zugriff auf https://www.porsche.com/germany/models/taycan/taycan-models/taycan/#technical-data (Zugriff am 22.12.2025)

Recke, M. P. (2025). *Getting directions in mapkit with swiftui.* Zugriff auf https://www.createwithswift.com/getting-directions-in-mapkit-with-swiftui/ (Zugriff am 22.12.2025)

Sambol, M. (2015). *Bellman-ford in 5 minutes — step by step example.* Zugriff auf https://www.youtube.com/watch?v=obWXjtg0L64 (Zugriff am 22.12.2025)

Studyflix. (2019). *Dijkstra algorithmus - beispiel mit graph und tabelle veranschaulicht!* Zugriff auf https://www.youtube.com/watch?v=8NsD01kzJwg&t=1s (Zugriff am 22.12.2025)

swiftbysundell. (2025). *Passing methods as swiftui view actions.* Zugriff auf https://www.swiftbysundell.com/tips/passing-methods-as-swiftui-view-actions/ (Zugriff am 22.12.2025)

Thinking, S. (2021). *Codable, decodable, and encodable in swift | continued learning 21.* Zugriff auf https://www.youtube.com/watch?v=H9mt8WjpZgM (Zugriff am 22.12.2025)

TitanUI. (2025). *Minimal iphone 17 vector mockup set.* Zugriff auf https://www.titanui.com/128876-minimal-iphone-17-vector-mockup-set/ (Zugriff am 22.12.2025)

Tree, S. (2020). *How dijkstra's algorithm works.* Zugriff auf https://www.youtube.com/watch?v=EFg3u_E6eHU (Zugriff am 22.12.2025)

Vattenfall. (2025). *Vattenfall incharge-app.* Zugriff auf https://apps.apple.com/us/app/vattenfall-incharge/id885790133 (Zugriff am 22.12.2025)

Volkswagen. (2025). *Der id.7, technische daten.* Zugriff auf https://www.volkswagen.de/de/modelle/id7.html/__ui/technische-daten.app?campaign=10003580%7CID7_AO&language=DE&adchan=seah&publisher=GOGL&country=DE&adplt=pseac&cpid=&adver=1409674980641&format=NU&adpay=pd&adpl=adtext&adcr=vw+id7&adgroup=ID7_%5BAMT%5D&seatype=branded&gclsrc=aw.ds&gad_source=1&gad_campaignid=20064107949&gbraid=0AAAAADEbU6EY66hXZ20SSevy9J-RZ00vZ (Zugriff am 22.12.2025)

Wikipedia. (2025a). *Distance from a point to a line.* Zugriff auf https://en.wikipedia.org/wiki/Distance_from_a_point_to_a_line (Zugriff am 22.12.2025)

Wikipedia. (2025b). *Geschichte des elektroautos.* Zugriff auf https://de.wikipedia.org/wiki/Geschichte_des_Elektroautos (Zugriff am 22.12.2025)

Wikipedia. (2025c). *Lohner-porsche.* Zugriff auf https://de.wikipedia.org/wiki/Lohner-Porsche (Zugriff am 22.12.2025)